TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ HỌA MÁY TÍNH

BÀI TẬP LỚP NHÀ

Giảng viên: Thầy Lý Quốc Ngọc

Sinh viên thực hiện:

Họ và tên	Mã số sinh viên	Lớp
Tô Thành Đạt	24127153	24TGMT
Lương Hưng Phát	24127298	24TGMT
Nguyễn Nhựt Huy	24127398	24TGMT

TP. Hồ Chí Minh, tháng 10/2026

BÀI TẬP LỚP NHÀ: NHÓM GITREMOVEALL

Tô Thành Đạt – 24127153
Khoa Công nghệ Thông tin
Trường ĐH Khoa học Tự nhiên
24127153@student.hcmus.edu.vn

Lương Hưng Phát – 24127298
Khoa Công nghệ Thông tin
Trường ĐH Khoa học Tự nhiên
24127298@student.hcmus.edu.vn

Nguyễn Nhựt Huy – 24127398
Khoa Công nghệ Thông tin
Trường ĐH Khoa học Tự nhiên
24127398@student.hcmus.edu.vn

ABSTRACT

Báo cáo này tổng hợp toàn bộ nội dung bài học, câu hỏi lý thuyết và bài tập môn Đồ họa Máy tính (Computer Graphics) theo lộ trình 12 tuần học.

MỤC LỤC

1 Tuần 1: Tổng quan Khoa học Dữ liệu Thị giác & Computer Graphics **5**

1.1 Đối tượng nghiên cứu của Khoa học Dữ liệu Thị giác và Dữ liệu Đa phương thức . 5
1.2 Mục tiêu và bản chất của cuộc Cách mạng Công nghiệp 4.0 5
1.3 Tiến trình phát triển kỹ thuật qua 4 cuộc Cách mạng Công nghiệp 5
1.4 Vai trò của dữ liệu thị giác trong sản xuất và quản lý xã hội 6
1.5 Tác động của dữ liệu thị giác trên các quy trình sản xuất ngành nghề 6
1.6 Nguồn gốc dữ liệu thị giác và Sự khác biệt giữa mắt người và thiết bị thu nhận hình ảnh 7
1.7 Hệ thống các loại đặc trưng trong dữ liệu thị giác 7
1.8 Khái niệm đối tượng trong dữ liệu thị giác: Phân biệt cấp độ lớp và cấp độ thực thể 7
1.9 Bản chất và thách thức của khoảng cách ngữ nghĩa (Semantic Gap) 8
1.10 Tiến trình phát triển trong việc thu hẹp khoảng cách ngữ nghĩa 8
1.11 Động lực mở rộng từ dữ liệu hình ảnh sang dữ liệu đa phương thức 8
1.12 Phân định ranh giới giữa Đồ họa máy tính, Xử lý ảnh và Thị giác máy tính 9
1.13 Khái quát các bài toán cốt lõi trong Thị giác máy tính 9
1.14 So sánh bài toán Nhận dạng (Recognition) và Truy vấn ảnh (Image Retrieval) . . . 12
1.15 So sánh bài toán Truy vấn ảnh (Retrieval) và Theo dõi đối tượng (Tracking) 12
1.16 So sánh bài toán Đếm đối tượng (Counting) và Phát hiện đối tượng (Object Detection) 12
1.17 Mô hình Tạo sinh (Generative) và tương quan với các tác vụ Phân tích (Discriminative) 12

2 Tuần 2: Trí tuệ Nhân tạo, Học máy & Học sâu trong Khoa học Dữ liệu Thị giác **14**

2.1 Bản chất của Trí tuệ Tự nhiên, Trí tuệ Nhân tạo và Khái niệm Thông minh 14
2.2 Tác động của Trí tuệ Nhân tạo đối với sự phát triển của Trí tuệ Tự nhiên 15

2.3 Phân định bản chất giữa Thông minh (Intelligence) và Thông tuệ (Wisdom) 17
2.4 Giải thích Luận điểm: "Thông minh là Khả năng Thích ứng với sự Thay đổi" . . . 17
2.5 Bản chất của Học máy: Phân tích tư tưởng Arthur Samuel (1959) 18
2.6 Định nghĩa Hình thức về Học máy theo Tom Mitchell (1997) 18
2.7 Tiến trình Chuyển tiếp từ Học máy sang Học sâu: Giới hạn và Động lực Đột phá . . 18
2.8 Nền tảng của Học sâu: Vì sao Nguyên lý Đơn giản tạo nên Khả năng Biểu diễn Vượt trội? 19
2.9 Mô hình 5 Cấp độ Nhận thức và Dữ liệu: Ứng dụng Giám sát An ninh Trường học . 20
2.10 Mô hình Giải pháp 3 Tầng trong Khoa học Dữ liệu Thị giác 20
2.11 Khái quát các Nhóm Ứng dụng Cơ bản của Thị giác Máy tính 21
2.12 Phân tích Chuyên sâu: Ứng dụng Trí tuệ Nhân tạo trong Ngành Điện ảnh 22
2.13 Ý nghĩa Thực tiễn của Bài toán Đếm Đám đông ở Tầng Ứng dụng Nghiệp vụ . . . 22
2.14 Phân tích Hệ thống Tái tạo Khuôn mặt 3D theo Mô hình Giải pháp 3 Tầng 22

3 Tuần 3: Ứng dụng Đồ họa Máy tính và Đề tài Nghiên cứu Sinh Đồ họa Vector (Text-to-Vector) 23

3.1 Bản chất và Mục tiêu của Đồ họa Máy tính 23
3.2 Phân tích Ứng dụng của Đồ họa Máy tính trong 7 Lĩnh vực Trọng yếu 24
3.3 Đề tài Nghiên cứu Chuyên sâu: Sinh Đồ họa Vector từ Ngôn ngữ Tự nhiên (Text-to-Vector Graphics Generation) 24

4 Tuần 4: Hệ Tọa độ, Cơ chế Chiếu và Giải thuật Vẽ Đoạn thẳng, Đường tròn 27

4.1 Bản chất và Phân định Ba Hệ Tọa độ trong Đồ họa Máy tính (WCS, UCS, DCS) . . 27
4.2 Cơ sở Lý thuyết: Mô phỏng Thế giới Thực thông qua Biểu diễn Vector Kích thước Nhỏ 27
4.3 Cơ chế Mô phỏng Đối tượng Thực 3 Chiều trên Màn hình Hiển thị 2 Chiều 28
4.4 Phân tích Toàn diện Giải thuật Vẽ Đoạn thẳng và Khảo sát Độ dốc 29
4.5 Cơ sở Giới hạn Bài toán $0 < m < 1, x_a < x_b$ và Cơ chế Suy diễn Toàn diện 29
4.6 Cơ chế Khử Phép tính Số thực của Thuật toán Bresenham Đoạn thẳng 30
4.7 Chứng minh Công thức Khởi tạo Giá trị p_0 31
4.8 Cơ sở Vẽ Hình tròn trên Lưới Tọa độ Nguyên và Tính Đối xứng 8 Phần 32
4.9 Phân tích Tính Liên tục 8-Lân cận: Vì sao x phải Tăng đều trong Cung $[90°, 45°]$. 32
4.10 Chứng minh Hình học và Đại số của Thuật toán Điểm Giữa (Midpoint Circle Algorithm) 33
4.11 Chứng minh Thay thế Khử Số thực: $p_0 = \frac{5}{4} - r$ bằng $p'_0 = 1 - r$ khi r Nguyên . . 34

5 Tuần 5: Thuật toán Vẽ Đường cong Ellipse, Đường cong Bézier và Phân vùng Đa giác 36

5.1 Thuật toán Vẽ Cung tròn và Cung Ellipse trên Lưới Tọa độ Nguyên 36
5.2 Hạn chế của Phương trình Tường minh $y = f(x)$ trong Biểu diễn Đường cong . . . 37
5.3 Cơ sở Toán học và Kỹ thuật Vẽ Đường cong Bézier trên Lưới Tọa độ Nguyên 38

5.4 Kiểm soát Bậc Thấp cho Tập Điểm Điều khiển Lớn: Đường cong Ghép mảnh (Piecewise Curves) . . . 38
5.5 Ý nghĩa Hình học của Đặc tính Nội suy Điểm Mút $P(0) = p_0, P(1) = p_n$ . . . 38
5.6 Phương trình Tham số của Đường cong Bézier Bậc Ba (Cubic Bézier Curve) . . . 39
5.7 Kiểm tra Đạo hàm Bậc Hai $P''(0)$ và $P''(1)$: Độ cong tại Hai Điểm Mút . . . 39
5.8 Phương trình Đoạn thẳng theo Tham số Đi qua p_0, p_1 . . . 40
5.9 Chứng minh Thuật toán De Casteljau cho Đường cong Bézier Bậc 2 và Bậc 3 . . . 40
5.10 So sánh Bản chất giữa Spline Hermite và Đường cong Bézier Bậc Ba . . . 41
5.11 Phương pháp Xác định Điểm Nằm Trong và Ngoài Hình Đa giác . . . 42
5.12 Chứng minh Cơ sở Toán học Dựa trên Định lý Đường cong Jordan . . . 42
5.13 Nguồn gốc và Bản chất Topo của Quy tắc Chẵn Lẻ (Odd-Even Rule) . . . 42
5.14 Kỹ thuật Rút ngắn Cạnh tại các Đỉnh Không Cực trị trong Thuật toán Tô màu theo Dòng quét . . . 43

6 Tuần 6: Kỹ thuật Tô miền Nâng cao và Phép Biến đổi Hình học trong Không gian 2D và 3D 44

6.1 Thuật toán Dòng quét với Góc nghiêng α Bất kỳ và Bước nhảy A Pixel . . . 44
6.2 Thuật toán Tô màu Dòng quét cho Biên Dạng Đường tròn và Ellipse . . . 45
6.3 Bản chất và Vai trò Nền tảng của các Phép Biến đổi Hình học trong Đồ họa Máy tính 45
6.4 Thiết lập Phép Quay và Co giãn 2D quanh Tâm Bất kỳ $C(x_c, y_c)$ . . . 46
6.5 Xác định Ma trận Biến đổi Đối xứng qua một Trục Tuyến tính Bất kỳ trong Mặt phẳng 2D . . . 47
6.6 Vai trò của Phép Biến đổi từ Hệ Tọa độ Thế giới Thực sang Hệ Tọa độ Thiết bị Hiển thị . . . 48
6.7 Tính Cần thiết của Cơ chế Đồng bộ Hai chiều Giữa Ánh xạ Cửa sổ - Cổng nhìn (Win-to-View) và Cổng nhìn - Cửa sổ (View-to-Win) . . . 48
6.8 Xây dựng Ma trận Biến đổi Quay quanh một Trục Bất kỳ trong Không gian 3 Chiều 49
6.9 Chứng minh Sự Đồng nhất của Hai Góc α trong Bài toán Quay quanh Trục 3D (Slide 28) . . . 50
6.10 Cơ sở Hiệu năng và Ứng dụng Thực tiễn của Tích Vô hướng và Tích Hữu hướng trong Đồ họa . . . 51
6.11 Chứng minh Biểu thức Tích Hữu hướng $\vec{u}' \times \vec{u}_z = b \cdot \vec{u}_x$ (Slide 29) . . . 52

7 Tuần 7: Mô hình Hóa Hình học Tham số 3D, Cấu trúc Dữ liệu Bản vẽ và Kỹ thuật Mặt Cong 54

7.1 Thuật toán và Cài đặt Tính Toán x_k theo Phương pháp Sai phân Quy nạp . . . 54
7.2 Thuật toán Tìm Bước Sai phân Tối ưu Thỏa mãn Tính Liên tục 8-Lân cận . . . 55
7.3 Tiêu chuẩn Phân định Vị trí Tương quan Trái - Phải của Điểm so với Vector qua Tích Hữu hướng . . . 56
7.4 Khảo sát Hệ thống các Ứng dụng Toàn diện của Tích Vô hướng và Tích Hữu hướng trong Đồ họa . . . 57
7.5 Phương pháp Tính Chiều dài Cung của Đường cong Bézier . . . 57

7.6 Thiết kế Cấu trúc Dữ liệu Bản vẽ Đồ họa Hỗ trợ Toàn diện Thao tác CRUD và Ánh xạ Hai chiều 59
7.7 Thuật toán và Cài đặt Vẽ Mô hình Khung dây (Wireframe) Quả cầu với Lưới Tứ giác 61
7.8 Xác định và Tính toán Trường Vector Pháp tuyến của Quả cầu 62
7.9 Thuật toán và Cài đặt Tạo Mặt Trụ Trải (TabSurf) với Đường sinh Bézier Bậc Ba . 63
7.10 Thuật toán và Cài đặt Vẽ Mặt Kẻ (RuleSurf) Nối Hai Biên Cong Bézier Bậc Ba Khác nhau 64
7.11 Bản chất Hình học của Mặt Vá Coons (EdgeSurf) và Chứng minh Lượng dư Triệt tiêu Bằng P_3 65

1 Tuần 1: Tổng quan Khoa học Dữ liệu Thị giác & Computer Graphics

1.1 Đối tượng nghiên cứu của Khoa học Dữ liệu Thị giác và Dữ liệu Đa phương thức

Khoa học Dữ liệu Thị giác (Visual Data Science) là lĩnh vực liên ngành nghiên cứu các nguyên lý toán học, cấu trúc dữ liệu và mô hình tính toán nhằm thu thập, biểu diễn, trích xuất đặc trưng và suy luận tri thức từ dữ liệu trực quan. Miền dữ liệu này bao gồm hình ảnh số 2D, chuỗi khung hình video có tính tương quan thời gian, đám mây điểm 3D (Point Clouds) từ thiết bị quét laser, dữ liệu thể tích voxel trong chẩn đoán y khoa (chụp cắt lớp vi tính CT, cộng hưởng từ MRI), cho đến ảnh viễn thám đa phổ từ vệ tinh.

Nhằm vượt qua giới hạn biểu diễn của từng kênh cảm biến đơn lẻ, ngành khoa học này đã mở rộng phạm vi sang dữ liệu đa phương thức (Multimodal Data). Việc thiết lập liên kết biểu diễn giữa dữ liệu thị giác với các phương thức khác như văn bản tự nhiên, tín hiệu âm thanh hay chuỗi đo lường quán tính (IMU) cho phép giải quyết các bài toán nhận thức không gian và ngữ cảnh phức tạp. Điển hình cho hướng tiếp cận này là bài toán phân đoạn đối tượng trong video dựa trên mô tả hành động bằng văn bản hoặc âm thanh dẫn đường (Referring Motion Expression Video Segmentation) (Ding et al., 2025). Thay vì chỉ định mục tiêu bằng các chỉ số lớp cố định, mô hình tiếp nhận câu mô tả tự nhiên hoặc âm thanh đặc trưng để khoanh vùng chính xác mặt nạ pixel (segmentation mask) của đối tượng chuyển động xuyên suốt các khung hình.

1.2 Mục tiêu và bản chất của cuộc Cách mạng Công nghiệp 4.0

Trọng tâm của cuộc Cách mạng Công nghiệp 4.0 là quá trình thông minh hóa (Smartification) toàn diện các quy trình sản xuất công nghiệp và hệ thống quản trị xã hội trên nền tảng chuyển đổi số.

Quá trình thông minh hóa được hiện thực hóa thông qua việc tích hợp các mô hình học máy và mạng lưới cảm biến vạn vật (IoT) vào thiết bị phần cứng. Nhờ đó, hệ thống kỹ thuật đạt được năng lực tự thu thập dữ liệu môi trường, tự hiệu chỉnh thông số vận hành và ra quyết định tối ưu theo thời gian thực mà **không cần sự can thiệp thủ công liên tục từ con người**.

Trong lĩnh vực sản xuất công nghiệp, quá trình này giúp nâng cao năng suất lao động, giảm thiểu chi phí vận hành, loại bỏ sai sót chủ quan và hiện thực hóa mô hình cá nhân hóa sản phẩm hàng loạt (Mass Customization). Trong quản trị xã hội, việc triển khai các hệ thống điều khiển giao thông thích ứng, lưới điện thông minh và nền tảng y tế số góp phần tối ưu hóa dịch vụ công và phân bổ nguồn lực đô thị minh bạch, hiệu quả.

Tiến trình chuyển đổi công nghệ trong kỷ nguyên mới diễn ra qua ba cấp độ kế thừa:

1. **Số hóa dữ liệu (Digitization):** Quá trình chuyển đổi dữ liệu từ dạng vật lý hoặc tương tự (tài liệu giấy, hình ảnh chụp, băng từ analog) sang định dạng nhị phân số.
2. **Số hóa quy trình (Digitalization):** Việc ứng dụng công nghệ kỹ thuật số và dữ liệu đã số hóa vào việc tối ưu hóa hoặc tự động hóa các quy trình nghiệp vụ sẵn có mà chưa làm thay đổi bản chất mô hình vận hành tổng thể.
3. **Chuyển đổi số (Digital Transformation):** Sự tái cấu trúc toàn diện mô hình hoạt động, quy trình nghiệp vụ, phương pháp ra quyết định và văn hóa tổ chức dựa trên việc khai thác dữ liệu và trí tuệ nhân tạo.

Trong hệ sinh thái công nghệ 4.0, các ngành khoa học cơ bản như Toán học, Vật lý, Hóa học và Sinh học đóng vai trò nền tảng trong việc xây dựng cơ sở lý thuyết và chế tạo linh kiện bán dẫn, cảm biến quang học. Tuy nhiên, Trí tuệ nhân tạo (AI) đóng vai trò cốt lõi bởi đây là công cụ tính toán duy nhất có khả năng trích xuất tri thức từ các tập dữ liệu phi cấu trúc khổng lồ và đưa ra quyết định tự hành trong môi trường thay đổi liên tục.

1.3 Tiến trình phát triển kỹ thuật qua 4 cuộc Cách mạng Công nghiệp

Lịch sử phát triển sản xuất của văn minh nhân loại được đánh dấu qua 4 bước chuyển dịch công nghệ nền tảng:

Cách mạng Công nghiệp 1.0 (Cuối thế kỷ 18 đến đầu thế kỷ 19): Khởi đầu từ việc thương mại hóa động cơ hơi nước và ứng dụng cơ giới hóa vào ngành dệt. Bước tiến này đã thay thế sức kéo cơ bắp của con người và động vật bằng năng lượng cơ khí, đặt nền móng cho các đô thị công nghiệp đầu tiên.

Cách mạng Công nghiệp 2.0 (Cuối thế kỷ 19 đến đầu thế kỷ 20): Được thúc đẩy bởi việc phát minh ra năng lượng điện, động cơ đốt trong và dây chuyền lắp ráp liên tục (Assembly Line). Đột phá này đưa phương thức sản xuất hàng loạt với chi phí thấp trở thành chuẩn mực công nghiệp, đồng thời thúc đẩy sự bùng nổ của mạng lưới giao thông vận tải và viễn thông hiện đại.

Cách mạng Công nghiệp 3.0 (Thập niên 1960 đến cuối thế kỷ 20): Chuyển trọng tâm sang công nghệ bán dẫn, máy vi tính cá nhân và mạng Internet. Sự ra đời của các bộ điều khiển logic lập trình được (PLC) trong tự động hóa nhà máy đã thay đổi căn bản cách thức vận hành công nghiệp và mở ra kỷ nguyên số hóa thông tin.

Cách mạng Công nghiệp 4.0 (Từ đầu thế kỷ 21 đến nay): Hướng tới xây dựng các hệ thống không gian mạng - vật lý (Cyber-Physical Systems - CPS), nơi thế giới thực và mô hình số đồng bộ hai chiều thông qua AI, IoT, dữ liệu lớn và điện toán đám mây, tiến tới mô hình nhà máy tự hành và đô thị thông minh.

1.4 Vai trò của dữ liệu thị giác trong sản xuất và quản lý xã hội

Xét về mặt nhận thức sinh học, thị giác cung cấp hơn 80% tổng lượng thông tin mà con người tiếp nhận từ thế giới xung quanh. Dưới góc độ kỹ thuật, dữ liệu thị giác đại diện cho phương thức thu thập thông tin không tiếp xúc (non-contact) và không phá hủy mẫu vật (non-destructive), mang lại mật độ thông tin không gian - thời gian vượt trội so với các loại cảm biến đo điểm truyền thống.

Trong sản xuất công nghiệp, hệ thống camera thị giác máy tính quét bề mặt sản phẩm với tần số cao để đo đạc kích thước hình học, kiểm tra độ phẳng và phát hiện khuyết tật vật liệu theo thời gian thực. Trong quản lý xã hội, luồng dữ liệu hình ảnh và video từ camera giám sát đô thị cùng ảnh vệ tinh viễn thám cung cấp thông tin không gian chi tiết, phục vụ công tác điều phối giao thông, phát hiện ùn tắc, giám sát an ninh trật tự và ứng phó thiên tai kịp thời.

1.5 Tác động của dữ liệu thị giác trên các quy trình sản xuất ngành nghề

Ứng dụng của dữ liệu thị giác hiện diện sâu rộng trong nhiều lĩnh vực sản xuất và quản lý tài nguyên:

Trong công nghiệp sản xuất và chế tạo cơ khí: Dữ liệu thị giác đóng vai trò cốt lõi trong hệ thống kiểm tra quang học tự động (Automated Optical Inspection - AOI) nhằm phát hiện các vết nứt vi mô, lỗi hàn vi mạch trên bo mạch điện tử với độ chính xác micromet. Đồng thời, thị giác robot (Vision-Guided Robotics) cung cấp tọa độ không gian 3D để cánh tay robot tự động gắp thả linh kiện, hàn điểm và lắp ráp phụ tùng trên dây chuyền tốc độ cao. Ngoài ra, hệ thống camera thông minh còn giám sát việc chấp hành trang bị bảo hộ cá nhân (PPE) và cảnh báo công nhân khi bước vào vùng nguy hiểm quanh máy móc nặng.

Trong nông nghiệp công nghệ cao: Hình ảnh kỹ thuật số hỗ trợ phân loại nông sản tự động sau thu hoạch dựa trên kích thước, màu sắc, độ chín và phát hiện dị tật bề mặt. Trên cánh đồng, camera gắn trên máy bay không người lái (drone) kết hợp thuật toán thị giác giúp phát hiện sớm sâu bệnh và nấm lá, cho phép phun thuốc bảo vệ thực vật chính xác theo từng cụm cây nhiễm bệnh. Ảnh đa phổ từ drone còn được sử dụng để tính toán chỉ số sinh khối (NDVI), hỗ trợ dự báo năng suất mùa màng chính xác.

Trong lâm nghiệp và quản lý môi trường: Dữ liệu ảnh vệ tinh đa thời gian được phân tích để theo dõi diễn biến độ che phủ rừng, phát hiện kịp thời các hoạt động phá rừng hoặc lấn chiếm đất lâm nghiệp trái phép. Bên cạnh đó, hệ thống camera nhiệt và camera quang học tầm xa đặt tại các tháp quan sát hỗ trợ phát hiện sớm cột khói và các điểm cháy rừng bất thường từ khoảng cách hàng chục kilômét.

1.6 Nguồn gốc dữ liệu thị giác và Sự khác biệt giữa mắt người và thiết bị thu nhận hình ảnh

Dữ liệu thị giác được thu thập từ đa dạng các loại cảm biến quang điện tử: camera quang học dải khả kiến (cảm biến CCD/CMOS), camera nhiệt hồng ngoại, thiết bị quét laser LiDAR, cảm biến thời gian bay (Time-of-Flight - ToF), thiết bị chụp cắt lớp y tế (X-ray, CT, MRI, siêu âm) cũng như dữ liệu tổng hợp từ các engine đồ họa 3D.

Xét về mặt cơ chế thu nhận và xử lý, giữa hệ thống thị giác sinh học của con người và thiết bị thu nhận ảnh kỹ thuật số tồn tại những khác biệt nền tảng:

Cơ chế cảm nhận và xử lý tín hiệu: Võng mạc mắt người chứa hàng triệu tế bào thụ cảm quang phân bố không đồng đều, gồm tế bào que cảm nhận độ sáng trong điều kiện thiếu sáng và tế bào nón cảm nhận màu sắc tập trung với mật độ cao nhất tại vùng hoàng điểm (fovea). Tín hiệu thị giác được tiền xử lý cục bộ ngay tại các tầng tế bào thần kinh võng mạc trước khi truyền qua dây thần kinh thị giác lên vỏ não. Ngược lại, cảm biến máy ảnh kỹ thuật số sử dụng mảng lưới điểm ảnh đồng nhất được phủ bộ lọc màu Bayer (RGB), thực hiện biến đổi tuyến tính photon ánh sáng thành điện tích để tạo ra ma trận số thô.

Dải động học (Dynamic Range): Mắt người sở hữu khả năng thích ứng động học linh hoạt vượt quá 20 stop ($> 100,000 : 1$), cho phép nhận biết đồng thời các chi tiết nằm trong vùng bóng râm sâu và vùng nguồn sáng chói. Cảm biến kỹ thuật số thương mại thông thường chỉ đạt dải động khoảng 10–14 stop, dễ dẫn đến hiện tượng cháy sáng ở vùng quá sáng hoặc mất chi tiết do nhiễu ở vùng tối.

Dải phổ điện từ (Spectral Range): Thị giác con người bị giới hạn nghiêm ngặt trong dải ánh sáng khả kiến với bước sóng từ 380 nm đến 740 nm. Thiết bị thu nhận nhân tạo có thể mở rộng độ nhạy quang phổ sang tia X, tia tử ngoại, dải hồng ngoại gần, hồng ngoại nhiệt và sóng radar vô tuyến.

Độ chính xác định lượng và tính ổn định: Thiết bị quang học đo lường cường độ bức xạ ánh sáng bằng các giá trị số học tuyệt đối và hoạt động liên tục với tính ổn định cao. Ngược lại, nhận thức thị giác của con người mang tính định tính, chịu sự chi phối của tâm lý học nhận thức và dễ bị suy giảm độ chính xác do ảo ảnh thị giác hoặc mệt mỏi thể chất.

1.7 Hệ thống các loại đặc trưng trong dữ liệu thị giác

Bên cạnh các đặc trưng thị giác mức thấp (màu sắc, kết cấu texture, hình dạng đường biên) và đặc trưng ngữ nghĩa mức cao (nhãn phân loại lớp đối tượng), việc biểu diễn dữ liệu thị giác hiện đại còn khai thác nhiều nhóm đặc trưng chuyên sâu:

Đặc trưng không gian (Spatial Features): Bao gồm độ sâu từng pixel, tọa độ không gian 3D thực tế, vector pháp tuyến bề mặt, góc xoay và tịnh tiến 6 bậc tự do (6-DoF Pose), cấu trúc đám mây điểm và lưới đa giác 3D (Polygon Mesh).

Đặc trưng thời gian (Temporal Features): Bao gồm luồng quang học (Optical Flow) biểu diễn vector chuyển động của từng pixel giữa hai khung hình liên tiếp (u, v), quỹ đạo di chuyển của đối tượng qua thời gian, cùng vận tốc và gia tốc chuyển động.

Đặc trưng cấu trúc và quan hệ (Relational Features): Được mô hình hóa dưới dạng Đồ thị quan hệ cảnh (Scene Graphs), trong đó các nút đại diện cho các thực thể và các cạnh mô tả quan hệ không gian hoặc tương tác người - vật (Human-Object Interaction - HOI), giúp hệ thống nắm bắt ngữ cảnh toàn diện thay vì các đối tượng đơn lẻ.

Đặc trưng thống kê và tần số (Statistical Features): Bao gồm phổ tần số không gian thông qua biến đổi Fourier, biểu diễn đa tỷ lệ qua biến đổi Wavelet, ma trận đồng xuất mức xám (GLCM) và ma trận hiệp phương sai đặc trưng.

1.8 Khái niệm đối tượng trong dữ liệu thị giác: Phân biệt cấp độ lớp và cấp độ thực thể

Trong thị giác máy tính, một đối tượng thị giác (Visual Object) là một vùng không gian hoặc tập hợp các pixel liên thông tương ứng với một thực thể vật lý xác định trong thế giới thực, có ranh giới phân định rõ ràng với môi trường nền xung quanh.

Việc định danh đối tượng được phân chia thành hai cấp độ nhận thức:

- **Đối tượng chung (General Object / Category-level):** Nhận diện một thực thể thuộc về một lớp khái niệm chung mà không phân biệt danh tính cá thể cụ thể (ví dụ: xác định một vùng ảnh chứa "nam giới", "xe ô tô" hoặc "con mèo").
- **Đối tượng cụ thể (Specific Object / Instance-level):** Định danh chính xác một thực thể duy nhất trong cơ sở dữ liệu mẫu (ví dụ: nhận diện một khuôn mặt cụ thể là "bạn An", chiếc ô tô mang biển số riêng "30F-999.99" hoặc tòa nhà cụ thể "Landmark 81").

1.9 Bản chất và thách thức của khoảng cách ngữ nghĩa (Semantic Gap)

Khoảng cách ngữ nghĩa (Semantic Gap) là sự chênh lệch căn bản giữa **cách biểu diễn dữ liệu mức thấp** của máy tính và **cách hiểu ngữ nghĩa mức cao** của con người.

Dưới góc nhìn của máy tính, hình ảnh số chỉ là một mảng dữ liệu đa chiều chứa các giá trị cường độ điểm ảnh số thô. Khi điều kiện chiếu sáng, góc chụp, khoảng cách hoặc tư thế vật thể thay đổi, toàn bộ các giá trị số trong ma trận điểm ảnh bị biến đổi hoàn toàn dù thực thể vật lý trong ảnh không thay đổi.

Ngược lại, hệ thống thị giác con người tự động trích xuất các cấu trúc bất biến, kết hợp với tri thức nền tảng về thế giới để nhận thức ngay lập tức danh tính thực thể, ngữ cảnh không gian, cảm xúc và các mối quan hệ logic phức tạp. Việc thu hẹp khoảng cách giữa tín hiệu điểm ảnh số thô và khái niệm ngữ nghĩa trừu tượng chính là bài toán trung tâm của thị giác máy tính.

1.10 Tiến trình phát triển trong việc thu hẹp khoảng cách ngữ nghĩa

Quá trình giải quyết khoảng cách ngữ nghĩa trong thị giác máy tính đã trải qua ba giai đoạn phát triển chính:

Giai đoạn trích xuất đặc trưng thủ công (Trước năm 2012): Dựa trên các thuật toán hình học và xử lý tín hiệu xác định như SIFT (Lowe, 2004), SURF (Bay et al., 2008), HOG (Dalal & Triggs, 2005) kết hợp với các bộ phân loại nông như Support Vector Machines (SVM) (Cortes & Vapnik, 1995) hoặc Random Forest (Breiman, 2001). Phương pháp này chỉ mô tả được các mẫu cục bộ đơn giản và phụ thuộc nặng nề vào kỹ nghệ thiết kế đặc trưng bằng tay, khiến khoảng cách ngữ nghĩa vẫn còn rất lớn.

Kỷ nguyên Học sâu phân cấp (Giai đoạn 2012–2020): Sự ra đời của các mạng nơ-ron tích chập sâu như AlexNet (Krizhevsky et al., 2012), VGG (Simonyan & Zisserman, 2015) và ResNet (He et al., 2016) đã mở ra cơ chế tự động học biểu diễn đặc trưng phân cấp trực tiếp từ dữ liệu thô. Các tầng nông học các đặc trưng biên nét cục bộ, trong khi các tầng sâu trừu tượng hóa thành các bộ phận và ngữ nghĩa hoàn chỉnh của đối tượng, giúp thu hẹp đáng kể khoảng cách ngữ nghĩa trong các bài toán đóng.

Kỷ nguyên Mô hình Nền tảng và Thị giác - Ngôn ngữ (Từ 2020 đến nay): Kiến trúc Vision Transformer (ViT) (Dosovitskiy et al., 2020) kết hợp với các mô hình học biểu diễn liên kết đa phương thức như CLIP (Radford et al., 2021), BLIP (Li et al., 2022), LLaVA (Liu et al., 2023) và GPT-4V (OpenAI, 2023) đã ánh xạ trực tiếp không gian ảnh và không gian ngôn ngữ tự nhiên vào cùng một không gian nhúng chung. Nhờ đó, mô hình có khả năng suy luận ngữ cảnh mở, sinh mô tả tự nhiên (Image Captioning) và trả lời câu hỏi thị giác (Visual Question Answering) một cách linh hoạt (Dai et al., 2023).

Mặc dù đạt được những tiến bộ vượt bậc, các mô hình học sâu và mô hình nền tảng hiện nay vẫn bộc lộ điểm yếu khi đối mặt với nhiễu đối kháng (adversarial perturbations) (Li et al., 2024), dễ sinh ảo giác (hallucination) trong điều kiện thiếu sáng hoặc bị che khuất nghiêm trọng (Hendrycks & Dietterich, 2019; Li et al., 2023; Guan et al., 2024), đồng thời còn hạn chế đáng kể trong khả năng suy luận quan hệ nhân quả (causal reasoning) (Zang et al., 2023).

1.11 Động lực mở rộng từ dữ liệu hình ảnh sang dữ liệu đa phương thức

Việc giới hạn mô hình trong dữ liệu hình ảnh hoặc video đơn lẻ bộc lộ nhiều điểm nghẽn nhận thức. Về mặt hình học, hình ảnh 2D là phép chiếu thu nhỏ từ thế giới 3D, làm suy giảm thông tin chiều

sâu và gây ra tính nhập nhằng về kích thước thực tế của vật thể. Hơn nữa, nhiều hiện tượng thực tế đòi hỏi sự kết hợp của các tín hiệu bổ trợ như âm thanh (tiếng va chạm cơ học, tiếng rò rỉ khí), văn bản ngữ cảnh (biển báo giao thông, tài liệu hướng dẫn) hay dữ liệu đo lường từ cảm biến gia tốc và nhiệt độ.

Dữ liệu đa phương thức mang lại cơ chế bổ trợ ngữ nghĩa hai chiều: ngôn ngữ tự nhiên cung cấp cấu trúc ngữ nghĩa trừu tượng và tri thức logic, trong khi dữ liệu thị giác cung cấp tọa độ hình học và chi tiết không gian cụ thể. Sự kết hợp này là điều kiện tiên quyết để xây dựng các hệ thống tự hành đáng tin cậy như xe tự lái (kết hợp đồng thời Camera RGB, Radar, LiDAR và bản đồ số HD) hay robot tương tác thông minh.

1.12 Phân định ranh giới giữa Đồ họa máy tính, Xử lý ảnh và Thị giác máy tính

Ba lĩnh vực Đồ họa máy tính (Computer Graphics - CG), Xử lý ảnh (Image Processing - IP) và Thị giác máy tính (Computer Vision - CV) cùng thao tác trên dữ liệu thị giác nhưng khác biệt căn bản về mục tiêu, phạm vi bài toán, phương pháp tiếp cận và bản chất chuyển đổi dữ liệu. Bảng 1 so sánh toàn diện giữa ba lĩnh vực trên 5 tiêu chí chuẩn hóa: Mục tiêu (Goal), Phạm vi (Scope), Phương pháp tiếp cận (Approach), Dữ liệu đầu vào (Input) và Dữ liệu đầu ra (Output).

Bảng 1: So sánh toàn diện giữa Computer Graphics, Image Processing và Computer Vision theo 5 tiêu chí chuẩn hóa

Tiêu chí	**Computer Graphics (CG)**	**Image Processing (IP)**	**Computer Vision (CV)**
1. Mục tiêu (Goal)	Mô phỏng thế giới thực	Nâng cao chất lượng, khôi phục ảnh suy thoái, lọc nhiễu, nén hoặc biến đổi biểu diễn dữ liệu ảnh.	Hiểu, phân tích, trích xuất cấu trúc ngữ nghĩa, tri thức và suy luận ra quyết định từ dữ liệu thị giác.
2. Phạm vi (Scope)	Không gian mô hình hình học 2D/3D trừu tượng, thực hiện chiếu phối cảnh, đổ bóng và mô phỏng vật liệu quang học.	Không gian tín hiệu pixel và tần số mức thấp, thao tác không can thiệp hay thay đổi ngữ nghĩa mức cao của đối tượng.	Không gian ngữ nghĩa mức cao, ánh xạ từ tín hiệu pixel sang biểu diễn nhận thức và quan hệ logic.
3. Phương pháp (Approach)	Mô hình hóa toán học (Polygon Mesh, Spline, NeRF, 3D Gaussian Splatting), đường ống đồ họa (Graphics Pipeline, Shaders, Ray Tracing).	Xử lý tín hiệu số 2D (lọc không gian, biến đổi Fourier/Wavelet, cân bằng lược đồ mức xám, toán tử hình thái học).	Học máy và học sâu (CNNs, Vision Transformers, Foundation Models) kết hợp hình học thị giác đa góc nhìn.
4. Đầu vào (Input)	Mô hình hình học 2D/3D (tập điểm, mesh, vertices), thuộc tính vật liệu, vị trí camera và thông số nguồn sáng.	Dữ liệu ảnh số thô, ảnh bị suy thoái do nhiễu vật lý, mờ chuyển động, thiếu sáng hoặc nén suy hao.	Hình ảnh số 2D $I \in \mathbb{R}^{H\times W\times C}$, chuỗi video $\{I_t\}$, ảnh đa phổ, đám mây điểm 3D từ môi trường thực tế.
5. Đầu ra (Output)	Hình ảnh hoặc chuỗi video được kết xuất trực quan chân thực lên màn hình hiển thị.	Hình ảnh đã qua xử lý, tăng cường hoặc khôi phục trong cùng không gian điểm ảnh.	Dữ liệu ngữ nghĩa có cấu trúc (nhãn phân loại, hộp bao, mặt nạ phân đoạn, pose 3D, quyết định hành động).
Bản chất chuyển đổi	**Dữ liệu trừu tượng → Hình ảnh**	**Hình ảnh → Hình ảnh**	**Hình ảnh → Dữ liệu ngữ nghĩa**

1.13 Khái quát các bài toán cốt lõi trong Thị giác máy tính

Các hệ thống thị giác máy tính hiện đại được cấu thành từ 7 bài toán nền tảng. Nhằm chuẩn hóa việc đánh giá và xây dựng mô hình, mỗi bài toán được phân tích có hệ thống theo 5 tiêu chí: **Mục tiêu**

(Goal), **Phạm vi (Scope)**, **Phương pháp tiếp cận (Approach)**, **Dữ liệu đầu vào (Input)** và **Dữ liệu đầu ra (Output)**.

1. Classification (Phân loại ảnh):

- **Mục tiêu (Goal):** Xác định và gán nhãn ngữ nghĩa ở mức toàn cảnh (Image-level) cho bức ảnh từ một tập danh mục lớp định trước.
- **Phạm vi (Scope):** Cấp độ toàn ảnh, tập danh mục đóng cố định ($|\mathcal{C}|$ lớp), không định vị tọa độ hay phân tách từng đối tượng riêng lẻ.
- **Phương pháp tiếp cận (Approach):** Mạng trích xuất đặc trưng sâu (CNN / ViT) kết hợp lớp Fully-Connected + Softmax, tối ưu hóa bằng hàm mất mát Cross-Entropy Loss hoặc Focal Loss.
- **Đầu vào (Input):** Hình ảnh tĩnh $I \in \mathbb{R}^{H \times W \times C}$ (với chiều cao H, chiều rộng W và C kênh màu).
- **Đầu ra (Output):** Nhãn danh mục $c \in \mathcal{C}$ hoặc vector phân phối xác suất dự đoán $\mathbf{p} = P(Y \mid I) \in \Delta^{|\mathcal{C}|-1}$.

2. Detection (Phát hiện đối tượng):

- **Mục tiêu (Goal):** Đồng thời xác định vị trí không gian (Localization) và phân loại ngữ nghĩa (Classification) cho tất cả các thực thể quan tâm xuất hiện trong ảnh.
- **Phạm vi (Scope):** Cấp độ đối tượng cụ thể (Instance-level), phát hiện đa đối tượng và đa lớp trong cùng một khung ảnh với kích thước và tỷ lệ khung hình biến thiên.
- **Phương pháp tiếp cận (Approach):** Kiến trúc Two-stage như Faster R-CNN (Ren et al., 2015), One-stage như YOLO (Redmon et al., 2016), RetinaNet (Lin et al., 2017), SSD (Liu et al., 2016) hoặc kiến trúc dựa trên Transformer như DETR (Carion et al., 2020); tối ưu đồng thời hàm mất mát phân loại (Cross-Entropy/Focal Loss) và hồi quy tọa độ hộp bao (GIoU (Rezatofighi et al., 2019), DIoU Loss (Zheng et al., 2020)) kết hợp thuật toán lọc Non-Maximum Suppression (NMS).
- **Đầu vào (Input):** Hình ảnh tĩnh hoặc từng khung hình video $I \in \mathbb{R}^{H \times W \times C}$.
- **Đầu ra (Output):** Tập hợp các hộp bao đối tượng $\mathcal{B} = \{(x_i, y_i, w_i, h_i, c_i, s_i)\}_{i=1}^{N}$, trong đó (x_i, y_i, w_i, h_i) là tọa độ tâm và kích thước khung chữ nhật bao bọc (Bounding Box), $c_i \in \mathcal{C}$ là nhãn lớp và $s_i \in [0, 1]$ là điểm số tin cậy.

3. Recognition (Nhận dạng):

- **Mục tiêu (Goal):** Định danh cụ thể một cá thể riêng biệt (Instance-level identity) trong một tập danh tính hoặc cơ sở dữ liệu mẫu đã biết (Closed-set) sau khi vùng đối tượng đã được cô lập.
- **Phạm vi (Scope):** Cấp độ cá thể riêng biệt (Fine-grained), vùng ảnh đối tượng đã được crop hoặc căn chỉnh (ROI), không gian danh tính hữu hạn định trước ($\mathcal{Y}_{\text{known}}$).
- **Phương pháp tiếp cận (Approach):** Trích xuất đặc trưng biểu diễn mức sâu kết hợp Metric Learning với góc margin lớn (CosFace, ArcFace) trên tập nhãn cố định, thực hiện so khớp 1:N với cơ sở dữ liệu mẫu (Gallery).
- **Đầu vào (Input):** Vùng ảnh chứa đối tượng đã cô lập $I_{\text{crop}} \in \mathbb{R}^{h \times w \times C}$ và cơ sở dữ liệu mẫu đã đăng ký $\mathcal{D}_{\text{gallery}}$.
- **Đầu ra (Output):** Mã định danh cá thể cụ thể (Identity ID) $y \in \mathcal{Y}_{\text{known}}$ (hoặc nhãn Unknown) kèm điểm số so khớp tương đồng.

4. Tracking (Theo dõi đối tượng):

- **Mục tiêu (Goal):** Duy trì mã định danh duy nhất (Track ID) và ước lượng quỹ đạo chuyển động không gian theo thời gian của từng đối tượng qua chuỗi khung hình video liên tiếp. Bản chất của bài toán theo dõi là sự kết hợp liên hoàn giữa Phát hiện đối tượng và So khớp liên kết dữ liệu theo thời gian (Tracking = Detection + Matching).

- **Phạm vi (Scope):** Miền không - thời gian (Spatio-temporal), liên tục trên chuỗi video thời gian thực, phạm vi tìm kiếm cục bộ quanh vị trí khung hình liền kề ($t \to t+1$).
- **Phương pháp tiếp cận (Approach):** Mô hình Tracking-by-Detection (TBD) hoặc Joint Detection & Embedding (JDE), kết hợp mô hình chuyển động động học (Kalman Filter, Optical Flow) với giải thuật tối ưu liên kết dữ liệu như thuật toán Hungarian (Kuhn, 1955) hay ByteTrack (Zhang et al., 2022) dựa trên chỉ số IoU và độ tương đồng Cosine đặc trưng.
- **Đầu vào (Input):** Chuỗi khung hình video liên tục $\{I_t\}_{t=1}^{T}$ với $I_t \in \mathbb{R}^{H \times W \times C}$.
- **Đầu ra (Output):** Tập các vết quỹ đạo không gian - thời gian $\mathcal{T} = \{(\text{ID}_i, t, x_{i,t}, y_{i,t}, w_{i,t}, h_{i,t})\}$, đảm bảo tính nhất quán định danh của từng cá thể qua trục thời gian t.

5. Retrieval (Truy vấn hình ảnh):

- **Mục tiêu (Goal):** Tìm kiếm và xếp hạng các hình ảnh có độ tương đồng nội dung cao nhất với ảnh truy vấn từ một kho cơ sở dữ liệu lớn ở môi trường mở (Open-set).
- **Phạm vi (Scope):** Cấp độ toàn cục trên kho dữ liệu quy mô lớn $\mathcal{D}$, không gian mở đối với các thực thể chưa từng thấy khi huấn luyện, các mẫu dữ liệu ảnh tĩnh độc lập phi thời gian.
- **Phương pháp tiếp cận (Approach):** Học không gian nhúng biểu diễn sâu (Deep Metric Learning / Contrastive Learning) với Triplet Loss hoặc InfoNCE Loss; ánh xạ ảnh thành vector nhúng $\mathbf{v} = f_\theta(I) \in \mathbb{R}^d$, lập chỉ mục vector và tìm kiếm lân cận gần nhất xấp xỉ (Approximate Nearest Neighbor - ANN như FAISS, HNSW).
- **Đầu vào (Input):** Ảnh truy vấn (Query Image) $I_q \in \mathbb{R}^{H \times W \times C}$ và kho cơ sở dữ liệu ảnh mẫu $\mathcal{D} = \{I_1, I_2, \ldots, I_M\}$.
- **Đầu ra (Output):** Danh sách xếp hạng thứ tự top-K ảnh $\{I_{\pi(1)}, I_{\pi(2)}, \ldots, I_{\pi(K)}\}$ có khoảng cách biểu diễn nhỏ nhất trong không gian nhúng kèm điểm số tương đồng Cosine.

6. Counting (Đếm đối tượng):

- **Mục tiêu (Goal):** Ước lượng tổng số lượng thực thể xuất hiện trong khung cảnh, đặc biệt hiệu quả trong các tình huống có mật độ cực cao hoặc che khuất nghiêm trọng.
- **Phạm vi (Scope):** Cấp độ khung cảnh mật độ cao (đám đông, vi sinh vật, đàn gia súc), chấp nhận hiện tượng che khuất chồng chéo sâu (Severe Occlusion).
- **Phương pháp tiếp cận (Approach):** Hồi quy trường mật độ liên tục (Density Map Estimation) qua mạng nơ-ron tích chập hoàn toàn (FCNs) hoặc Transformer như CSRNet (Li et al., 2018) hay DM-Count (Wang et al., 2020) với hàm mất mát MSE hoặc Bayesian Loss dựa trên nhãn điểm tâm (Dot annotation) được lọc Gaussian; tránh hoàn toàn bước phân tách hộp bao rời rạc và lọc NMS.
- **Đầu vào (Input):** Hình ảnh tĩnh $I \in \mathbb{R}^{H \times W \times C}$ (kèm nhãn tọa độ điểm chấm (x_p, y_p) trong pha huấn luyện).
- **Đầu ra (Output):** Bản đồ mật độ liên tục $D \in \mathbb{R}^{H \times W}$, trong đó tổng tích phân $\hat{N} = \sum_{x,y} D(x,y)$ xấp xỉ tổng số đối tượng (hoặc giá trị số thực ước tính $\hat{N} \in \mathbb{R}^+$).

7. Generating (Sinh dữ liệu thị giác):

- **Mục tiêu (Goal):** Khởi tạo hình ảnh hoặc chuỗi video mới hoàn chỉnh, mô phỏng đúng phân phối dữ liệu thị giác thực tế $P(X)$ hoặc sinh có điều kiện theo yêu cầu người dùng.
- **Phạm vi (Scope):** Toàn bộ không gian phân phối xác suất dữ liệu cao chiều, từ sinh vô điều kiện đến sinh có điều kiện đa phương thức (Text-to-Image, Image-to-Image, Inpainting, ControlNet).
- **Phương pháp tiếp cận (Approach):** Mô hình xác suất tạo sinh sâu (GANs với Minimax Adversarial Loss, VAEs với ELBO, Diffusion Models / Flow Matching với Denoising Score Matching và U-Net/DiT).

- **Đầu vào (Input):** Vector nhiễu ngẫu nhiên $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ trong không gian tiềm ẩn, kèm theo điều kiện dẫn đường tùy chọn (mô tả văn bản $\mathbf{c}_{\text{text}}$, ảnh điều kiện I_{cond}, bản đồ bố cục Canny/Pose).
- **Đầu ra (Output):** Hình ảnh hoặc chuỗi video tổng hợp mới $\hat{I} \in \mathbb{R}^{H \times W \times C}$ có độ chân thực cao và thỏa mãn các ràng buộc ngữ nghĩa đầu vào.

Bảng 2 tóm tắt ma trận so sánh 7 bài toán cốt lõi của Thị giác máy tính theo đúng 5 tiêu chí chuẩn hóa.

Bảng 2: Ma trận tổng hợp so sánh 7 bài toán cốt lõi trong Thị giác máy tính theo 5 tiêu chí

Bài toán	**Mục tiêu (Goal)**	**Phạm vi (Scope)**	**Phương pháp (Approach)**	**Đầu vào (Input)**	**Đầu ra (Output)**
Classification	Gán nhãn ngữ nghĩa toàn cảnh	Toàn ảnh, tập đóng $\mathcal{C}$ lớp	CNN / ViT + Softmax (Cross-Entropy Loss)	Ảnh tĩnh $I \in \mathbb{R}^{H \times W \times C}$	Nhãn $c \in \mathcal{C}$, phân phối $\mathbf{p}$
Detection	Định vị & phân loại đối tượng	Đối tượng (Instance), đa lớp	Two/One-stage, DETR, Box Loss + NMS	Ảnh tĩnh / Khung video I	Hộp bao $\mathcal{B} = \{(x, y, w, h, c, s)\}$
Recognition	Định danh cá thể cụ thể	Cá thể (Instance), tập đóng	Deep Embedding + ArcFace / Softmax	Ảnh crop $I_{\text{crop}} + \mathcal{D}_{\text{gallery}}$	Identity ID $y \in \mathcal{Y}_{\text{known}}$
Tracking	Duy trì ID & quỹ đạo theo thời gian	Không - thời gian, video liên tục	Tracking-by-Detection + Kalman + ByteTrack	Chuỗi video $\{I_t\}_{t=1}^{T}$	Quỹ đạo $\mathcal{T} = \{(\text{ID}_i, t, \mathbf{b}_{i,t})\}$
Retrieval	Tìm ảnh tương đồng ngữ nghĩa	Toàn cục kho ảnh, tập mở (Open-set)	Metric Learning (InfoNCE, Triplet) + ANN	Ảnh truy vấn $I_q + \mathcal{D}$	Ranked List $\{I_{\pi(k)}\}$
Counting	Ước lượng số lượng thực thể	Mật độ cao, chịu che khuất nặng	Hồi quy bản đồ mật độ (CSRNet) + Density Loss	Ảnh tĩnh $I \in \mathbb{R}^{H \times W \times C}$	Bản đồ mật độ $D \to \hat{N} = \sum D$
Generating	Khởi tạo nội dung thị giác mới	Không gian phân phối dữ liệu $P(X)$	Diffusion Models, GANs, VAEs, Flow Matching	Nhiễu $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I}) +$ Condition	Ảnh/video mới $\hat{I}$

1.14 So sánh bài toán Nhận dạng (Recognition) và Truy vấn ảnh (Image Retrieval)

Mặc dù đều phục vụ mục đích xác định sự tương đồng của đối tượng thị giác, Nhận dạng (Recognition) và Truy vấn ảnh (Image Retrieval) khác biệt căn bản về mục tiêu, không gian bài toán, cơ chế tối ưu hóa hàm mất mát, dữ liệu đầu vào và dạng thức đầu ra (Bảng 3).

1.15 So sánh bài toán Truy vấn ảnh (Retrieval) và Theo dõi đối tượng (Tracking)

Sự khác biệt cốt lõi giữa Truy vấn ảnh (Retrieval) và Theo dõi đối tượng (Tracking) bắt nguồn từ yếu tố phụ thuộc thời gian, miền không gian tìm kiếm, phương pháp kết hợp mô hình động học và dạng thức đầu ra (Bảng 4):

1.16 So sánh bài toán Đếm đối tượng (Counting) và Phát hiện đối tượng (Object Detection)

Trong các khung cảnh thông thường với mật độ thưa, bài toán đếm có thể giải quyết gián tiếp thông qua việc phát hiện từng đối tượng riêng lẻ. Tuy nhiên, khi mật độ tăng cao và xuất hiện hiện tượng che khuất nghiêm trọng, hai phương pháp này thể hiện sự khác biệt rõ rệt trên cả 5 tiêu chí (Bảng 5):

1.17 Mô hình Tạo sinh (Generative) và tương quan với các tác vụ Phân tích (Discriminative)

Các bài toán trong học máy và thị giác máy tính được phân chia thành hai trường phái nền tảng dựa trên mục tiêu mô hình hóa phân phối xác suất, phạm vi biến đổi không gian dữ liệu và cách thức tối ưu hóa (Bảng 6):

Bảng 3: So sánh toàn diện giữa bài toán Recognition và Image Retrieval theo 5 tiêu chí

Tiêu chí	Nhận dạng (Recognition)	Truy vấn hình ảnh (Image Retrieval)
1. Mục tiêu (Goal)	Định danh chính xác một cá thể cụ thể (Instance Identity) thuộc về một danh tính đã đăng ký trước trong hệ thống.	Tìm kiếm và xếp hạng các hình ảnh có độ tương đồng ngữ nghĩa/nội dung cao nhất với ảnh truy vấn từ một kho dữ liệu lớn.
2. Phạm vi (Scope)	Tập đóng (Closed-set): số lượng lớp và danh tính được cố định trước trong quá trình huấn luyện; đối tượng thường là vùng ảnh đã được cô lập (Cropped/Aligned ROI).	Tập mở (Open-set): tìm kiếm các thực thể và mẫu chưa từng xuất hiện trong tập huấn luyện; phạm vi tìm kiếm toàn cục trên cơ sở dữ liệu quy mô lớn $\mathcal{D}$.
3. Phương pháp (Approach)	Học các siêu phẳng phân tách ranh giới giữa các lớp xác định bằng Softmax hoặc Margin Loss (ArcFace, CosFace); suy luận trực tiếp bằng $\arg\max$ xác suất hoặc so khớp ngưỡng với Gallery mẫu cố định.	Học không gian nhúng biểu diễn (Deep Metric Learning / Contrastive Learning) với Triplet Loss, InfoNCE Loss để tối ưu khoảng cách biểu diễn; suy luận bằng trích xuất embedding và thuật toán tìm kiếm lân cận gần nhất (k-NN, ANN, FAISS).
4. Đầu vào (Input)	Vùng ảnh chứa đối tượng đã cô lập $I_{\text{crop}} \in \mathbb{R}^{h \times w \times C}$ và cơ sở dữ liệu mẫu đã đăng ký $\mathcal{D}_{\text{gallery}}$.	Ảnh truy vấn (Query Image) $I_q \in \mathbb{R}^{H \times W \times C}$ và kho cơ sở dữ liệu ảnh mẫu quy mô lớn $\mathcal{D} = \{I_1, I_2, \ldots, I_M\}$.
5. Đầu ra (Output)	Mã định danh cá thể cụ thể (Identity ID) $y \in \mathcal{Y}_{\text{known}}$ (hoặc nhãn Unknown) kèm xác suất hoặc điểm số tin cậy.	Danh sách có thứ tự xếp hạng (Ranked List) top-K hình ảnh tương đồng nhất $\{I_{\pi(1)}, \ldots, I_{\pi(K)}\}$ kèm điểm số khoảng cách.

Bảng 4: So sánh toàn diện giữa bài toán Image Retrieval và Object Tracking theo 5 tiêu chí

Tiêu chí	Truy vấn hình ảnh (Retrieval)	Theo dõi đối tượng (Tracking)
1. Mục tiêu (Goal)	Tìm kiếm và xếp hạng các hình ảnh tương đồng nhất với ảnh mẫu trong kho dữ liệu tĩnh.	Duy trì định danh duy nhất (Track ID) và ước lượng quỹ đạo chuyển động không gian theo thời gian của đối tượng qua chuỗi video.
2. Phạm vi (Scope)	Không gian toàn cục tĩnh: các mẫu dữ liệu độc lập, phi thời gian, không có liên kết vật lý thời gian.	Không - thời gian cục bộ: chuỗi video thời gian thực liên tục; tìm kiếm trong lân cận không gian dự đoán từ khung hình liền kề ($t \to t+1$).
3. Phương pháp (Approach)	Sử dụng thuần túy mô hình đặc trưng ngoại hình tĩnh kết hợp cấu trúc dữ liệu chỉ mục vector như IVF-PQ (Jégou et al., 2011) hay HNSW.	Kết hợp mô hình ngoại hình và mô hình chuyển động động học (Kalman Filter, Optical Flow) cùng giải thuật tối ưu phân phối liên kết dữ liệu (Hungarian Matching (Kuhn, 1955), ByteTrack (Zhang et al., 2022)).
4. Đầu vào (Input)	Ảnh truy vấn I_q và kho cơ sở dữ liệu ảnh mẫu độc lập $\mathcal{D} = \{I_1, \ldots, I_M\}$.	Chuỗi khung hình video liên tục $\{I_t\}_{t=1}^{T}$ có tính tương quan thời gian cao theo tốc độ khung hình ($I_t \in \mathbb{R}^{H \times W \times C}$).
5. Đầu ra (Output)	Danh sách các hình ảnh xếp hạng theo độ tương đồng $\{I_{\pi(1)}, \ldots, I_{\pi(K)}\}$.	Tập hợp các vết quỹ đạo không gian - thời gian $\mathcal{T} = \{(\text{Track ID}_i, t, x_{i,t}, y_{i,t}, w_{i,t}, h_{i,t})\}$.

Bảng 5: So sánh toàn diện giữa bài toán Object Detection và Crowd Counting theo 5 tiêu chí

Tiêu chí	Phát hiện đối tượng (Detection)	Đếm đối tượng (Counting)
1. Mục tiêu (Goal)	Đồng thời định vị ranh giới không gian (Bounding Box) và phân loại ngữ nghĩa cho từng cá thể độc lập.	Ước lượng tổng số lượng thực thể xuất hiện trong khung cảnh, tập trung vào mật độ tổng thể thay vì ranh giới từng cá thể.
2. Phạm vi (Scope)	Thích hợp cho khung cảnh thưa đến trung bình, đối tượng có ranh giới rõ ràng; suy giảm độ chính xác nghiêm trọng khi mật độ cực cao và che khuất chồng chéo sâu.	Chuyên biệt hóa cho khung cảnh mật độ từ trung bình đến cực dày đặc (đám đông, vi sinh vật, xe cộ), bền vững trước hiện tượng che khuất nghiêm trọng.
3. Phương pháp (Approach)	Hồi quy hộp bao rời rạc và phân loại nhãn (Two-stage/One-stage/DETR) với Cross-Entropy + GIoU Loss, lọc Non-Maximum Suppression (NMS); yêu cầu nhãn hộp bao chi tiết (x, y, w, h).	Hồi quy trường mật độ liên tục (Density Map Estimation) qua FCN/Transformer với hàm mất mát MSE hoặc Bayesian Loss; chỉ yêu cầu nhãn điểm tâm (Dot annotation) Gaussian hóa; không dùng NMS.
4. Đầu vào (Input)	Hình ảnh tĩnh hoặc khung hình video $I \in \mathbb{R}^{H\times W\times C}$ (kèm nhãn hộp bao chi tiết trong pha huấn luyện).	Hình ảnh tĩnh hoặc chuỗi khung hình $I \in \mathbb{R}^{H\times W\times C}$ (kèm nhãn tọa độ điểm chấm (x_p, y_p) trong pha huấn luyện).
5. Đầu ra (Output)	Tập hợp các hộp bao rời rạc $\mathcal{B} = \{(x_i, y_i, w_i, h_i, c_i, s_i)\}_{i=1}^{N} \Longrightarrow \hat{N} = \lvert\mathcal{B}\rvert$.	Bản đồ phân phối mật độ liên tục $D \in \mathbb{R}^{H\times W} \Longrightarrow \hat{N} = \sum_{x,y} D(x, y)$ (hoặc trực tiếp giá trị vô hướng $\hat{N}$).

2 Tuần 2: Trí tuệ Nhân tạo, Học máy & Học sâu trong Khoa học Dữ liệu Thị giác

2.1 Bản chất của Trí tuệ Tự nhiên, Trí tuệ Nhân tạo và Khái niệm Thông minh

Trong khoa học nhận thức và khoa học máy tính, việc phân định ranh giới bản thể luận giữa các dạng thức trí tuệ là tiền đề lý thuyết cốt lõi nhằm định hình phương pháp thiết kế các hệ thống tính toán thông minh.

Trí tuệ Tự nhiên (Natural Intelligence, NI): Đại diện cho năng lực sinh học bẩm sinh của con người và các sinh vật bậc cao, được tôi luyện qua hàng triệu năm tiến hóa chọn lọc tự nhiên. Trí tuệ tự nhiên tích hợp hữu cơ nhiều phương thức nhận thức phức tạp: khả năng thụ cảm đa giác quan, ý thức tự thân (Self-awareness), tư duy trừu tượng hóa, suy luận quan hệ nhân quả (Causal Reasoning), năng lực sáng tạo nghệ thuật và sự thích nghi linh hoạt trước những biến đổi bất định của sinh cảnh. Toàn bộ tiến trình nhận thức này luôn gắn liền với đời sống cảm xúc nội tâm, các quy chuẩn đạo đức và bối cảnh văn hóa xã hội.

Trí tuệ Nhân tạo (Artificial Intelligence, AI): Là một phân ngành mũi nhọn của khoa học máy tính, tập trung nghiên cứu các nền tảng toán học, cấu trúc dữ liệu và giải thuật nhằm kiến tạo các hệ thống tính toán có khả năng mô phỏng các khía cạnh hành vi thông minh của con người. Phạm vi năng lực của trí tuệ nhân tạo bao gồm việc học biểu diễn từ dữ liệu (Learning), suy luận logic hình thức (Reasoning), nhận thức thị giác máy tính (Visual Perception), hiểu ngôn ngữ tự nhiên (NLP) và hoạch định chiến lược ra quyết định (Decision Making).

Nhằm làm sáng tỏ sự khác biệt cơ bản giữa hai hệ thống này, Bảng 7 tổng hợp các tiêu chí đối chiếu bản chất giữa Trí tuệ Tự nhiên và Trí tuệ Nhân tạo.

Về mặt động học vận hành, mọi thực thể thông minh (dù là sinh học hay nhân tạo) đều hiện thực hóa một vòng lặp nhận thức khép kín bao gồm ba giai đoạn kế tiếp nhau:

1. **Giai đoạn 1: Nhận biết (Perception):** Tiếp nhận các kích thích và tín hiệu vật lý từ môi trường bên ngoài. Ở sinh vật, tiến trình này diễn ra thông qua hệ giác quan sinh học với cơ chế chú ý tự nhiên nhằm chắt lọc các kích thích có ý nghĩa sinh tồn. Trong hệ thống thị giác máy tính, nhận thức khởi đầu từ việc các cảm biến quang học, máy ảnh số hoặc

Bảng 6: So sánh toàn diện giữa mô hình Tạo sinh (Generative) và các tác vụ Phân tích (Discriminative) theo 5 tiêu chí

Tiêu chí	**Tác vụ Phân tích (Discriminative Tasks)**	**Tác vụ Tạo sinh (Generative Tasks)**
1. Mục tiêu (Goal)	Trích xuất thông tin ngữ nghĩa, định vị, phân loại hoặc đưa ra quyết định từ dữ liệu thị giác đã có (nhận thức thế giới quan).	Học phân phối dữ liệu thị giác thực tế $P(X)$ để tổng hợp, khởi tạo hoặc tái tạo nội dung hình ảnh/video mới chân thực (kiến tạo nội dung).
2. Phạm vi (Scope)	Không gian phân loại hoặc hồi quy có giám sát; ánh xạ thu hẹp từ không gian ảnh cao chiều về không gian nhãn/tọa độ/vector tóm lược ($f : \mathcal{X} \to \mathcal{Y}$).	Toàn bộ không gian phân phối dữ liệu thị giác cao chiều $P(X)$ hoặc $P(X \mid Y)$; bao gồm sinh vô điều kiện, có điều kiện (Text-to-Image, ControlNet) và mô hình hóa không gian tiềm ẩn liên tục.
3. Phương pháp (Approach)	Học xác suất hậu nghiệm $P(Y \mid X)$ (ResNet (He et al., 2016), ViT (Dosovitskiy et al., 2020), YOLO (Redmon et al., 2016), DETR (Carion et al., 2020)); tối ưu hóa ranh giới phân tách lớp bằng Cross-Entropy, Focal Loss (Lin et al., 2017), GIoU Loss (Rezatofighi et al., 2019).	Học phân phối thực $P(X)$ hoặc đồng thời $P(X, Y)$ (GANs (Goodfellow et al., 2014), VAEs (Kingma & Welling, 2014), Diffusion Models (Ho et al., 2020), Flow Matching (Lipman et al., 2023), DiT (Peebles & Xie, 2023)); tối ưu hóa phân phối bằng Minimax Adversarial Loss, ELBO, Denoising Score Matching MSE.
4. Đầu vào (Input)	Hình ảnh hoặc video thô $I \in \mathbb{R}^{H \times W \times C}$ thu nhận từ môi trường thực tế.	Vector nhiễu ngẫu nhiên $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ từ không gian tiềm ẩn kèm tín hiệu dẫn đường điều kiện (văn bản $\mathbf{c}_{\text{text}}$, ảnh bố cục, pose, cạnh nét).
5. Đầu ra (Output)	Dữ liệu ngữ nghĩa có cấu trúc (nhãn phân loại, hộp bao, mặt nạ phân đoạn, vector nhúng, số đếm, vết quỹ đạo).	Hình ảnh hoặc chuỗi video tổng hợp mới hoàn chỉnh $\hat{I} \in \mathbb{R}^{H \times W \times C}$ có độ phân giải cao và chân thực.

cảm biến LiDAR thu nhận sóng ánh sáng và chuyển hóa thành mảng ma trận điểm ảnh thô $I \in \mathbb{R}^{H \times W \times C}$.

2. **Giai đoạn 2: Hiểu (Understanding / Recognition):** Chuyển hóa dữ liệu cảm giác thô thành biểu diễn ngữ nghĩa và cấu trúc tri thức. Con người hiểu sâu sắc về công năng, bối cảnh vật lý và mối liên hệ nhân quả của các thực thể trong tầm nhìn. Đối với hệ thống nhân tạo, mô hình thực hiện trích xuất đặc trưng phân cấp, ánh xạ không gian tiềm ẩn để ước lượng phân phối xác suất lớp $P(Y \mid X)$, định vị không gian và xây dựng đồ thị quan hệ cảnh (Scene Graph).
3. **Giai đoạn 3: Thích nghi và Hành động (Action / Adaptation):** Hoạch định kế hoạch và ban hành quyết định hành vi nhằm tương tác với môi trường hoặc cực đại hóa mục tiêu tối ưu. Con người hành động dựa trên ý chí, trực giác và khả năng ứng biến linh hoạt. Hệ thống AI tương tác thông qua các cơ cấu chấp hành (như góc lái của xe tự hành, tín hiệu điều khiển rơ-le an ninh) hoặc đề xuất giải pháp nghiệp vụ, đồng thời thích ứng bằng cách cập nhật tập tham số trọng số θ qua quá trình học máy.

Từ tiến trình nhận thức ba bước trên, bản chất của khái niệm **Thông minh (Intelligence)** được xác lập dựa trên hai trụ cột cốt lõi: năng lực nhận biết, mô hình hóa nhanh chóng các quy luật tiềm ẩn từ luồng dữ liệu nhiễu loạn cao chiều; và năng lực thích ứng linh hoạt (Adaptability) nhằm điều chỉnh hành vi, cập nhật trọng số tri thức để duy trì hiệu năng tối ưu khi môi trường hoặc phân phối dữ liệu đầu vào có sự biến đổi.

2.2 Tác động của Trí tuệ Nhân tạo đối với sự phát triển của Trí tuệ Tự nhiên

Mối quan hệ tương tác giữa Trí tuệ Nhân tạo và Trí tuệ Tự nhiên mang tính chất biện chứng hai chiều sâu sắc, bao gồm cả những cơ hội khuếch đại nhận thức lẫn các thách thức suy giảm tư duy tiềm ẩn:

Bảng 7: Đối chiếu nền tảng giữa Trí tuệ Tự nhiên (NI) và Trí tuệ Nhân tạo (AI)

Tiêu chí đối chiếu	Trí tuệ Tự nhiên (NI)	Trí tuệ Nhân tạo (AI)
1. Cơ chất vật lý	Mạng nơ-ron sinh học hữu cơ với khoảng 86 tỷ tế bào thần kinh và 10^{14} khớp thần kinh synapse, vận hành qua tín hiệu điện hóa sinh học.	Nền tảng bán dẫn vô cơ trên các vi xử lý chuyên dụng (Silicon, GPU, TPU), thực thi tính toán dựa trên dòng chuyển dịch bit nhị phân.
2. Nguồn gốc & Mục đích	Sản phẩm của tiến hóa sinh học qua chọn lọc tự nhiên; mục đích cốt lõi là duy trì sự sống, thích nghi môi trường và bảo tồn nòi giống.	Do con người thiết kế và lập trình; đóng vai trò là công cụ tự động hóa tính toán nhằm giải quyết các bài toán kinh tế, kỹ thuật và xã hội.
3. Cơ chế học tập	Học hiệu quả từ số ít mẫu (Few-shot Learning), đúc kết tri thức qua trải nghiệm thực chứng và duy trì khả năng học liên tục suốt đời mà không bị xóa bỏ ký ức nền tảng.	Phụ thuộc vào quá trình tối ưu hóa thống kê trên tập dữ liệu quy mô lớn; dễ gặp hiện tượng quên tri thức cũ (Catastrophic Forgetting) khi chuyển sang miền dữ liệu mới.
4. Bản chất tư duy	Tư duy trừu tượng, suy luận quan hệ nhân quả và thấu hiểu bản chất nguồn gốc của sự vật hiện tượng nhằm trả lời câu hỏi vì sao (**"Tại sao"**).	So khớp mẫu quy mô lớn, tính toán tương quan xác suất thống kê $P(Y \mid X)$ và xấp xỉ hàm số phức tạp để trả lời câu hỏi dự đoán biến cố (**"Cái gì"**).
5. Ý thức & Cảm xúc	Sở hữu ý thức tự thân, trải nghiệm cảm xúc thực thể, gắn liền với lòng trắc ẩn và tinh thần trách nhiệm đạo đức trước cộng đồng.	Vận hành dựa trên các phép biến đổi đại số tuyến tính; phản hồi ngôn ngữ hoặc biểu cảm thị giác chỉ là kết quả mô phỏng thống kê trên bề mặt dữ liệu.
6. Hiệu suất năng lượng	Tối ưu hóa chuyển hóa năng lượng vượt bậc: toàn bộ não bộ con người chỉ tiêu thụ công suất sinh học ước tính khoảng 20 W.	Tiêu hao năng lượng lớn: các cụm máy chủ huấn luyện mô hình nền tảng đòi hỏi công suất từ hàng trăm kilowatt đến hàng megawatt điện.
7. Phạm vi năng lực	Trí tuệ tổng quát (General Intelligence): con người có thể tiếp thu đồng thời nhiều kỹ năng dị biệt và chuyển giao tri thức linh hoạt giữa các lĩnh vực.	Trí tuệ chuyên biệt (Narrow AI): thực thi vượt trội trong một tác vụ giới hạn được huấn luyện cụ thể nhưng gặp nhiều rào cản khi tổng quát hóa sang miền nghiệp vụ mới.

Xét về mặt tích cực, AI đóng vai trò như một đòn bẩy khuếch đại nhận thức (Cognitive Augmentation). Bằng cách tự động hóa các tác vụ tính toán lặp lại có tính quy chuẩn cao như thống kê dữ liệu quy mô lớn, rà soát văn bản pháp lý hay phân loại hàng loạt hình ảnh, AI giải phóng nguồn lực nhận thức của con người để tập trung vào các hoạt động sáng tạo chiến lược, tư duy phản biện và hoạch định tầm nhìn. Hơn thế nữa, AI mở rộng ranh giới khám phá khoa học khi giải quyết những bài toán đa chiều vượt ngoài năng lực trực giác sinh học. Minh chứng tiêu biểu là việc hệ thống AlphaFold (Jumper et al., 2021; Abramson et al., 2024) dự đoán chính xác cấu trúc 3D của hơn 200 triệu protein và các phức hợp phân tử sinh học, tạo bước ngoặt đột phá cho y học và sinh học phân tử.

Ngược lại, việc ứng dụng thiếu kiểm soát có thể dẫn đến nguy cơ suy giảm nhận thức do phụ thuộc công nghệ (Cognitive Offloading). Xu hướng ỷ lại vào các thuật toán tự động (Automation Bias) làm suy yếu phản xạ tư duy độc lập, suy giảm khả năng phân tích logic nền tảng và làm thui chột dung lượng trí nhớ làm việc (Working Memory). Bên cạnh đó, các thuật toán đề xuất thông tin cá nhân hóa dễ tạo ra hiện tượng bong bóng thông tin (Echo Chambers), cô lập người dùng trong những góc nhìn định kiến và làm suy giảm khả năng thấu cảm xã hội.

Như vậy, trí tuệ nhân tạo không tất yếu dẫn đến sự phát triển hay thoái hóa của trí tuệ con người. Kết quả thực tiễn hoàn toàn phụ thuộc vào phương thức con người định vị công nghệ: xem AI là công cụ đồng hành khuếch đại năng lực tư duy (Co-pilot) hay một sự thay thế thụ động toàn diện.

2.3 Phân định bản chất giữa Thông minh (Intelligence) và Thông tuệ (Wisdom)

Trong khoa học nhận thức và triết học, "Thông minh" và "Thông tuệ" (hay Trí huệ) đại diện cho hai nấc thang nhận thức hoàn toàn khác biệt về mặt mục đích và giá trị, được phân tích chi tiết trong Bảng 8.

Bảng 8: Đối chiếu toàn diện giữa khái niệm Thông minh (Intelligence) và Thông tuệ (Wisdom)

Tiêu chí	Thông minh (Intelligence)	Thông tuệ / Trí huệ (Wisdom)
Định nghĩa cốt lõi	Năng lực giải quyết bài toán và tối ưu hóa quy trình: thực thi tác vụ một cách chính xác, nhanh chóng và hiệu quả nhất (*Doing things right*).	Năng lực định hướng giá trị và mục đích: lựa chọn đúng mục tiêu có ý nghĩa nhân văn và đạo đức để cống hiến (*Doing the right things*).
Đối tượng xử lý	Dữ liệu số, cấu trúc thông tin, quy tắc logic và các không gian trạng thái toán học.	Ý nghĩa xã hội, chuẩn mực đạo đức, hạnh phúc con người và tính bền vững lâu dài của hệ sinh thái.
Động lực vận hành	Tối ưu hóa hàm mục tiêu kỹ thuật (cực tiểu hóa hàm tổn thất hoặc cực đại hóa phần thưởng kỳ vọng) dựa trên dữ liệu thống kê quá khứ.	Cân bằng hài hòa giữa lợi ích cá nhân và cộng đồng, gắn liền với lòng trắc ẩn (Compassion) và sự thấu cảm chiều sâu.
Khả năng hiện thực	Hệ thống máy tính và mô hình AI có thể đạt tới trình độ vượt bậc (Super-human) trong các tác vụ tính toán chuyên biệt.	Là đặc tính độc bản của nhận thức sinh học và trải nghiệm sống nội tâm; AI hiện đại chưa thể đạt được năng lực thông tuệ tự thân.

2.4 Giải thích Luận điểm: "Thông minh là Khả năng Thích ứng với sự Thay đổi"

Nhà vật lý lý thuyết Stephen Hawking từng đúc kết bản chất của trí tuệ thông qua phát biểu:

> *"Intelligence is the ability to adapt to change."* (*Thông minh là khả năng thích ứng với sự thay đổi*.)

Luận điểm này phản ánh nguyên lý vận hành cốt lõi của tính thông minh khi xét dưới cả hai lăng kính: tâm lý học nhận thức sinh học và khoa học máy tính hiện đại:

Dưới góc độ tâm lý học nhận thức, nhà nghiên cứu Jean Piaget (Piaget, 1950) khẳng định rằng sự phát triển trí tuệ thực chất là tiến trình cân bằng nhận thức động (Equilibration) thông qua hai cơ chế tương hỗ: **Đồng hóa (Assimilation)**, tức quá trình tiếp nhận thông tin mới từ môi trường vào các lược đồ tư duy (Schemas) sẵn có; và **Điều ứng (Accommodation)**, tức việc biến đổi và tái cấu trúc các lược đồ nhận thức nội tại để tương thích với những dữ kiện thực tế mới phát sinh. Sinh vật không thể tồn tại nếu chỉ vận hành bằng các phản xạ cứng nhắc; khả năng điều ứng linh hoạt trước biến động sinh thái chính là biểu hiện cao nhất của trí thông minh.

Dưới góc độ học máy và thị giác máy tính, một mô hình chỉ ghi nhớ máy móc các điểm dữ liệu trong tập huấn luyện (hiện tượng Overfitting) hoàn toàn không có tính thông minh thực sự. Thước đo thông minh của hệ thống AI được xác lập thông qua năng lực thích ứng với các biến số chưa từng xuất hiện:

- *Khả năng tổng quát hóa ngoài phân phối (Out-of-Distribution, OOD):* Mô hình duy trì độ chính xác cao khi phân phối ảnh đầu vào bị biến đổi do nhiễu thời tiết, góc chụp hay độ chiếu sáng bất thường.
- *Thích ứng miền (Domain Adaptation) và Học chuyển giao (Transfer Learning):* Tận dụng tri thức đã học từ miền nguồn (như dữ liệu ảnh mô phỏng 3D) để nhanh chóng thích ứng với miền đích thực tế chỉ với số lượng mẫu gán nhãn tối thiểu.
- *Học tăng cường thích ứng (Reinforcement Learning):* Tác tử tự động điều chỉnh chính sách hành động $\pi(a \mid s)$ khi ma trận chuyển trạng thái của môi trường biến động.
- *Học trong ngữ cảnh (In-Context Learning):* Khả năng của các mô hình thị giác ngôn ngữ lớn trong việc tiếp nhận nhiệm vụ mới thông qua một vài chỉ dẫn (Prompts) hoặc mẫu thị giác minh họa mà không cần tái huấn luyện trọng số mô hình.

2.5 Bản chất của Học máy: Phân tích tư tưởng Arthur Samuel (1959)

Năm 1959, Arthur Samuel đã thiết lập nền móng cho học máy hiện đại qua định nghĩa kinh điển:

> *"Machine Learning: Field of study that gives computers the ability to learn without being explicitly programmed."* (*Học máy: Phân ngành nghiên cứu mang lại cho máy tính khả năng tự học mà không cần phải được lập trình tường minh.*)

Tư tưởng mang tính bước ngoặt của Samuel đã phân định rõ ràng hai hệ hình tính toán căn bản trong khoa học máy tính:

Trong hệ hình lập trình truyền thống (Rule-based Systems), lập trình viên phải thấu suốt toàn bộ bản chất quy luật và tự tay viết tường minh từng câu lệnh điều kiện (`if-else`) cùng các công thức giải tích. Mô hình này vận hành theo sơ đồ tuyến tính:

$$\text{Dữ liệu đầu vào (Data)} + \text{Quy tắc lập trình (Rules)} \longrightarrow \text{Kết quả đầu ra (Output)} \tag{1}$$

Phương pháp tiếp cận dựa trên luật gặp phải giới hạn cố hữu khi đối mặt với các bài toán thị giác máy tính phi cấu trúc. Con người không thể lập trình thủ công hàng triệu nhánh điều kiện để bao quát toàn bộ sự biến thiên phức tạp của các ma trận điểm ảnh tự nhiên.

Ngược lại, hệ hình học máy (Learning-based Systems) đảo ngược hoàn toàn quy trình xây dựng phần mềm. Kỹ sư chỉ cần thiết lập một kiến trúc tham số hóa $f_\theta(x)$ cùng một hàm mục tiêu (Loss Function) $\mathcal{L}$. Thông qua giải thuật tối ưu hóa, máy tính sẽ tự động điều chỉnh tập tham số θ để mô hình hóa quy luật ẩn sâu trong dữ liệu:

$$\text{Dữ liệu đầu vào (Data)} + \text{Mục tiêu mong muốn (Output)} \longrightarrow \text{Mô hình tri thức } (\theta^*) \tag{2}$$

2.6 Định nghĩa Hình thức về Học máy theo Tom Mitchell (1997)

Nhằm chuẩn hóa học máy thành một ngành khoa học toán học có khả năng đo lường chính xác, Tom Mitchell (1997) đã đưa ra định nghĩa hình thức:

> *"A computer program is said to learn from experience $\mathcal{E}$ with respect to some class of tasks $\mathcal{T}$ and performance measure $\mathcal{P}$, if its performance at tasks in $\mathcal{T}$, as measured by $\mathcal{P}$, improves with experience $\mathcal{E}$."*

Định nghĩa này cấu thành từ bộ ba toán học $(\mathcal{T}, \mathcal{E}, \mathcal{P})$ chặt chẽ:

- **Nhiệm vụ $\mathcal{T}$ (Task):** Mục tiêu nghiệp vụ xác định mà thuật toán cần giải quyết. Ví dụ trong thị giác máy tính: Phát hiện và định vị các vết nứt trên bề mặt sản phẩm công nghiệp từ ảnh $I \in \mathbb{R}^{H \times W \times C}$.
- **Trải nghiệm $\mathcal{E}$ (Experience):** Tập hợp dữ liệu lịch sử hoặc chuỗi tương tác thực tế mà mô hình được tiếp cận. Ví dụ: Tập dữ liệu huấn luyện $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ gồm $100,000$ hình ảnh đã được chuyên gia gán nhãn hộp bao lỗi chuẩn xác.
- **Độ đo hiệu năng $\mathcal{P}$ (Performance Measure):** Thước đo định lượng dùng để đánh giá chất lượng thực thi tác vụ. Ví dụ: Chỉ số độ chính xác trung bình mAP@0.5 trên tập kiểm thử độc lập $\mathcal{D}_{\text{test}}$.

Về mặt giải tích, thuật toán được chứng minh là đang thực hiện tiến trình học tập khi chất lượng thực thi tăng đơn điệu theo lượng trải nghiệm tích lũy:

$$\frac{\partial \mathcal{P}(\mathcal{T})}{\partial |\mathcal{E}|} > 0 \quad \text{tương đương với} \quad \lim_{|\mathcal{E}| \to \infty} \mathcal{L}(\theta; \mathcal{E}) \to \min \tag{3}$$

2.7 Tiến trình Chuyển tiếp từ Học máy sang Học sâu: Giới hạn và Động lực Đột phá

Lịch sử phát triển của thị giác máy tính ghi nhận bước chuyển dịch mang tính nền tảng từ các thuật toán học máy cổ điển sang các kiến trúc học sâu:

Trong kỷ nguyên học máy truyền thống (trước năm 2012), quy trình xử lý hình ảnh bị phân tách thành hai công đoạn độc lập: (1) Trích xuất đặc trưng thủ công (Hand-crafted Feature Extraction) dựa trên các bộ mô tả hình học cục bộ do con người thiết kế như SIFT (Lowe, 2004), HOG (Dalal & Triggs, 2005) hay LBP; (2) Đưa các vector đặc trưng này vào các mô hình phân loại nông như Support Vector Machines (SVM (Cortes & Vapnik, 1995)) hoặc Random Forest (Breiman, 2001). Ngược lại, học sâu tích hợp toàn bộ tiến trình thành một mạng nơ-ron thống nhất theo cơ chế học biểu diễn đầu-cuối (End-to-End Representation Learning), ánh xạ trực tiếp từ cường độ pixel thô đến dự đoán ngữ nghĩa cuối cùng.

Sự chuyển dịch này xuất phát từ ba hạn chế căn bản của phương pháp học máy truyền thống:

- *Nút thắt cổ chai từ đặc trưng thủ công:* Các bộ trích xuất đặc trưng truyền thống chỉ nắm bắt được các biến đổi hình học cục bộ đơn giản (đường biên, góc, kết cấu xám), dễ bị phá vỡ tính bất biến trước các biến dạng phối cảnh phức tạp, hiện tượng che khuất hoặc thay đổi điều kiện ánh sáng mạnh.
- *Lời nguyền số chiều (Curse of Dimensionality):* Khi độ phân giải ảnh tăng cao, không gian đặc trưng bùng nổ theo cấp số nhân khiến các thuật toán dựa trên ranh giới phẳng hoặc khoảng cách hình học bị suy thoái hiệu năng nghiêm trọng.
- *Sự bão hòa dung lượng biểu diễn:* Khi quy mô dữ liệu mở rộng từ hàng vạn lên hàng triệu mẫu, hiệu năng của các mô hình nông nhanh chóng chạm ngưỡng bão hòa do dung lượng tham số bị hạn chế.

Ngược lại, học sâu tạo nên bước đột phá nhờ ba động lực cốt lõi: khả năng tự động học biểu diễn phân cấp (Hierarchical Feature Abstraction) từ đường nét cục bộ ở tầng thấp đến ngữ nghĩa hoàn chỉnh ở tầng cao; tuân theo định luật mở rộng quy mô (Scaling Laws) khi hiệu năng liên tục tăng trưởng tương ứng với dung lượng dữ liệu và kích thước mô hình dưới sự hỗ trợ của phần cứng GPU; và khả năng biểu diễn đa phương thức thống nhất giữa hình ảnh, video, âm thanh và ngôn ngữ tự nhiên.

2.8 Nền tảng của Học sâu: Vì sao Nguyên lý Đơn giản tạo nên Khả năng Biểu diễn Vượt trội?

Về mặt toán học, mọi kiến trúc mạng nơ-ron sâu phức tạp đều được cấu thành từ việc xếp chồng ba phép toán nền tảng:

1. Phép biến đổi affine tuyến tính: $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$.
2. Phép kích hoạt phi tuyến cục bộ từng phần tử: $\mathbf{a} = \sigma(\mathbf{z})$ (với σ là các hàm ReLU, GELU, hoặc Sigmoid).
3. Quá trình tối ưu hóa tham số thông qua giải thuật lan truyền ngược (Backpropagation) dựa trên quy tắc đạo hàm chuỗi (Chain Rule) và phương pháp hạ dốc độ dốc ngẫu nhiên (SGD (Robbins & Monro, 1951)).

Sự kết hợp của những nguyên lý giải tích đơn giản này tạo nên năng lực biểu diễn vượt trội nhờ ba cơ chế toán học sâu sắc:

1. Định lý Xấp xỉ Phổ quát (Universal Approximation Theorem): Mạng nơ-ron chứa các tầng ẩn phi tuyến có khả năng xấp xỉ bất kỳ hàm số liên tục nào với sai số nhỏ tùy ý. Khi mở rộng chiều sâu kiến trúc, cấu trúc phân cấp cho phép mô hình biểu diễn các hàm logic cực kỳ phức tạp với số lượng tham số tiết kiệm hơn theo hàm mũ so với một mạng nông tương đương.

2. Cấu trúc thiên lệch cảm ứng (Inductive Bias): Các kiến trúc hiện đại được thiết kế để khai thác tối đa cấu trúc vật lý của dữ liệu thị giác. Phép toán tích chập (Convolution) tận dụng tính bất biến tịnh tiến và tính tương quan không gian cục bộ của ảnh tự nhiên. Tương tự, cơ chế tự chú ý (Self-Attention) trong Vision Transformer mô hình hóa quan hệ phụ thuộc toàn cảnh linh hoạt giữa các vùng ảnh.

3. Hình học tối ưu trong không gian tham số cao chiều: Trong không gian tham số hàng triệu đến hàng tỷ chiều, các điểm cực tiểu cục bộ xấu (Spurious Local Minima) hiếm khi tồn tại; thay vào đó, các điểm dừng chủ yếu là các điểm yên ngựa (Saddle Points) mà giải thuật tối ưu hóa có thể vượt qua, giúp mô hình hội tụ về các thung lũng nghiệm phẳng có khả năng tổng quát hóa cao.

2.9 Mô hình 5 Cấp độ Nhận thức và Dữ liệu: Ứng dụng Giám sát An ninh Trường học

Quá trình chuyển hóa dữ liệu từ dạng tín hiệu vật lý thô sang hành động có giá trị nhân văn được hệ thống hóa qua **5 Cấp độ Nhận thức**:

1. **Level 1: Unfiltered Data (Dữ liệu thô chưa lọc):** Tín hiệu điểm ảnh quang học, sóng âm thanh, dữ liệu cảm biến vật lý thô.
2. **Level 2: Pattern Information (Thông tin mẫu):** Các mẫu hình học, đặc trưng thị giác, vector nhúng ngữ nghĩa được trích xuất có cấu trúc.
3. **Level 3: Predictability Knowledge (Tri thức có tính dự đoán):** Định danh đối tượng, xu hướng chuyển động, mối liên hệ nhân quả và quy luật hành vi.
4. **Level 4: Choice Intelligence (Trí thông minh ra quyết định):** Khả năng lựa chọn phương án can thiệp tối ưu theo quy tắc nghiệp vụ.
5. **Level 5: Compassion Wisdom (Thông tuệ thấu cảm):** Đánh giá hoàn cảnh nhân văn, thấu hiểu nguyên nhân gốc rễ và đưa ra giải pháp nhân bản.

Bảng 9 phân tích ma trận ứng dụng 5 cấp độ này vào bài toán **Hệ thống Giám sát An ninh & Điểm danh Khuôn mặt tại Trường học**.

Bảng 9: Ma trận phân tích 5 cấp độ nhận thức ứng dụng trong Hệ thống Giám sát An ninh Trường học

Task hệ thống	**Cấp độ nhận thức**	**Dữ liệu đầu vào (Input)**	**Dữ liệu đầu ra (Output)**	**Nội dung chi tiết & Ví dụ minh họa trong ô**
Task 1: Thu nhận luồng hình ảnh	**Level 1: Unfiltered Data**	Luồng video RTSP từ camera giám sát tại cổng trường	Mảng ma trận byte điểm ảnh thô $I_t \in \mathbb{R}^{H \times W \times 3}$	**Mức 1: Dữ liệu thô.** Thu nhận liên tục 30 khung hình mỗi giây chứa toàn bộ thông tin nền (bóng râm, cây cối, xe cộ, biển thiên ánh sáng) mà hệ thống chưa phân định được sự hiện diện của học sinh.
Task 2: Phát hiện & Trích xuất khuôn mặt	**Level 2: Pattern Information**	Khung ảnh thô I_t	Hộp bao khuôn mặt $\mathcal{B}$ và Vector nhúng đặc trưng $\mathbf{e} \in \mathbb{R}^{512}$	**Mức 2: Thông tin mẫu.** Thuật toán RetinaFace định vị vùng mặt và trích xuất vector 512 chiều biểu diễn cấu trúc khoảng cách hình học giữa mắt, mũi và miệng.
Task 3: Định danh & Đối chiếu lịch sử	**Level 3: Predictability Knowledge**	Vector nhúng $\mathbf{e}$ kèm cơ sở dữ liệu học sinh $\mathcal{D}_{\text{student}}$	Danh tính học sinh kèm độ tin cậy và lịch sử chuyên cần	**Mức 3: Tri thức dự đoán.** Xác định khuôn mặt thuộc về học sinh với độ tin cậy 99.4%; nhận diện học sinh đi trễ 3 ngày liên tiếp, dự báo xu hướng vi phạm nội quy.
Task 4: Điều khiển cổng & Cảnh báo tự động	**Level 4: Choice Intelligence**	Kết quả định danh và quy tắc nghiệp vụ an ninh	Tín hiệu relay mở chốt cổng, thông báo phụ huynh hoặc chuông báo	**Mức 4: Trí thông minh lựa chọn.** Nếu là học sinh hợp lệ: tự động kích hoạt relay mở cổng và gửi tin nhắn điểm danh cho phụ huynh; nếu phát hiện đối tượng lạ: tự động khóa cổng và báo bảo vệ.
Task 5: Đánh giá hoàn cảnh & Hỗ trợ nhân văn	**Level 5: Compassion Wisdom**	Nhật ký chuyên cần, biểu cảm mệt mỏi và dữ liệu hoàn cảnh gia đình	Đề xuất tư vấn tâm lý, hỗ trợ hoàn cảnh khó khăn	**Mức 5: Thông tuệ thấu cảm.** Không xử lý trừ điểm thi đua máy móc; hệ thống ghi nhận dấu hiệu kiệt sức kèm thông tin gia cảnh khó khăn để đề xuất giáo viên chủ nhiệm thăm hỏi và hỗ trợ tâm lý kịp thời.

2.10 Mô hình Giải pháp 3 Tầng trong Khoa học Dữ liệu Thị giác

Để hiện thực hóa một giải pháp thị giác máy tính hoàn chỉnh phục vụ thực tế, kiến trúc kỹ thuật được chuẩn hóa thành **Mô hình Giải pháp 3 Tầng** (Bảng 10):

1. **Tầng 1: Kỹ thuật Cốt lõi / Nền tảng Thuật toán (Foundational Technologies):** Các mô hình giải tích toán học, kiến trúc mạng nơ-ron nền tảng (CNN, Vision Transformer), hàm mất mát chuyên biệt, thuật toán tối ưu (AdamW (Loshchilov & Hutter, 2019), SGD (Robbins & Monro, 1951)) và hình học đa góc nhìn (Epipolar, Homography).

2. **Tầng 2: Tác vụ Thị giác Cơ sở (Core Vision Tasks):** Các khối chức năng thị giác cơ bản giải quyết bài toán kỹ thuật chuyên biệt như Phân loại ảnh, Phát hiện đối tượng, Nhận dạng khuôn mặt, Ước lượng chiều sâu hay Theo dõi vết chuyển động.

3. **Tầng 3: Hệ thống Ứng dụng Nghiệp vụ Hoàn chỉnh (End-to-End Business Applications):** Tích hợp các khối chức năng Tầng 2 cùng hệ thống cơ sở dữ liệu, phân quyền bảo mật, giao diện người dùng và cơ chế điều khiển phần cứng nhằm tạo thành sản phẩm hoàn chỉnh phục vụ nhu cầu xã hội.

Bảng 10: Cấu trúc Mô hình Giải pháp 3 Tầng trong Thị giác máy tính

Phân tầng	**Thành phần cốt lõi**	**Ví dụ minh họa cụ thể**
Tầng 1: Kỹ thuật cốt lõi	Kiến trúc mạng (ResNet (He et al., 2016), Swin-ViT (Liu et al., 2021), YOLO-backbone (Redmon et al., 2016)), Hàm mất mát (ArcFace, InfoNCE, GIoU (Rezatofighi et al., 2019)), Thư viện tối ưu (PyTorch, TensorRT).	Mạng ResNet-50 (He et al., 2016) kết hợp hàm mất mát ArcFace tối ưu hóa bằng thuật toán AdamW (Loshchilov & Hutter, 2019) trên tập dữ liệu chuẩn.
Tầng 2: Tác vụ thị giác	Các bài toán cơ sở: Face Detection, Facial Landmark Alignment, Re-Identification, Crowd Density Estimation.	Thuật toán phát hiện 68 điểm mốc khuôn mặt và trích xuất vector đặc trưng khuôn mặt 512 chiều.
Tầng 3: Ứng dụng nghiệp vụ	Hệ thống hoàn chỉnh tích hợp cơ sở dữ liệu, phân quyền bảo mật, cảnh báo thời gian thực và tự động hóa quy trình nghiệp vụ.	Hệ thống Kiosk tự động Check-in hành khách tại sân bay tích hợp cổng kiểm soát xuất nhập cảnh sinh trắc học.

2.11 Khái quát các Nhóm Ứng dụng Cơ bản của Thị giác Máy tính

Thị giác máy tính hiện đại đóng vai trò công nghệ nền tảng cho 5 nhóm lĩnh vực công nghiệp trọng yếu, được tổng hợp trong Bảng 11.

Bảng 11: Tổng hợp 5 nhóm lĩnh vực ứng dụng cơ bản của Thị giác máy tính

Lĩnh vực ứng dụng	**Các tác vụ Tầng 2 tích hợp**	**Hệ thống sản phẩm Tầng 3 tiêu biểu**
1. Giám sát an ninh & Đô thị thông minh	Detection, Tracking, License Plate Recognition, Crowd Counting.	Hệ thống camera giám sát và phạt nguội vi phạm giao thông; Trung tâm điều hành an ninh trật tự đô thị.
2. Y tế số & Chẩn đoán hình ảnh	Semantic Segmentation, 3D Voxel Reconstruction, Anomaly Detection.	Phần mềm tự động khoanh vùng tổn thương khối u trên ảnh chụp CT/MRI; Hệ thống phát hiện sớm polyp nội soi tiêu hóa.
3. Tự hành & Robot công nghiệp	3D Object Detection, Depth Estimation, Visual SLAM, Optical Flow.	Hệ thống xe tự lái; Cánh tay robot công nghiệp tự động định vị gắp thả linh kiện và kiểm tra lỗi hàn vi mạch.
4. Bán lẻ & Thương mại điện tử	Image Retrieval, Fine-grained Classification, Pose Estimation.	Công cụ tìm kiếm sản phẩm bằng hình ảnh trên sàn thương mại điện tử; Chuỗi cửa hàng bán lẻ tự động không người thu ngân.
5. Điện ảnh, CGI & Thực tế ảo (AR/VR)	3D Face Reconstruction, Motion Capture, Generative Video Inpainting.	Công nghệ kỹ xảo điện ảnh kỹ thuật số (VFX); Tạo nhân vật số Digital Avatar thời gian thực; Bộ lọc hiệu ứng khuôn mặt AR trên thiết bị di động.

2.12 Phân tích Chuyên sâu: Ứng dụng Trí tuệ Nhân tạo trong Ngành Điện ảnh

Ứng dụng AI trong phân tích trải nghiệm khán giả điện ảnh (AI for Cinema Audience Analysis) là minh chứng sinh động cho sự liên kết chặt chẽ giữa cả ba tầng kỹ thuật nhằm tối ưu hóa giá trị nghệ thuật và hiệu quả thương mại (Bảng 12).

Bảng 12: Quy trình phân tích trải nghiệm khán giả điện ảnh ứng dụng AI

Giai đoạn phân tích	**Công nghệ & Tác vụ thị giác (Tầng 1 & 2)**	**Giá trị ứng dụng nghiệp vụ điện ảnh (Tầng 3)**
1. Nhận diện khuôn mặt & Cảm xúc	Camera hồng ngoại trong rạp tối kết hợp thuật toán Facial Landmark Detection và mô hình phân loại biểu cảm theo hệ thống FACS.	Định lượng chính xác mức độ kích thích cảm xúc của khán giả tại từng phân cảnh; kiểm chứng thực nghiệm xem cảnh phim có đạt hiệu ứng nghệ thuật kỳ vọng.
2. Phân tích hành vi & Hướng nhìn	Mô hình ước lượng tư thế đầu (Head Pose), theo dõi ánh mắt (Eye Gaze Tracking) và nhận diện hành vi cơ thể (ngả người, che mắt, xem điện thoại).	Đánh giá mức độ tập trung và gắn kết (Engagement Level) của người xem; phát hiện chính xác các đoạn phim gây nhàm chán hoặc phân tâm.
3. Đồng bộ Timeline & Tối ưu kịch bản	Tích hợp chuỗi dữ liệu cảm xúc khán giả đồng bộ thời gian với biểu đồ dựng phim (Cut-by-cut synchronization).	Hỗ trợ đạo diễn và nhà sản xuất trong các buổi chiếu thử nghiệm (Test Screening): cắt gọt các phân đoạn dư thừa, điều chỉnh nhịp độ phim (Pacing) và tối ưu hóa chiến dịch quảng bá phòng vé.

2.13 Ý nghĩa Thực tiễn của Bài toán Đếm Đám đông ở Tầng Ứng dụng Nghiệp vụ

Ở cấp độ kỹ thuật Tầng 2, bài toán Đếm đám đông (Crowd Counting) thực hiện ánh xạ ảnh thô $I \in \mathbb{R}^{H\times W\times C}$ sang trường mật độ liên tục $D(x, y)$ để ước lượng tổng số thực thể $\hat{N} = \sum_{x,y} D(x, y)$. Khi được tích hợp lên **Tầng 3 (Hệ thống Ứng dụng Nghiệp vụ Thực tiễn)**, công nghệ này mang lại ba giá trị quản trị và an toàn xã hội thiết thực:

Trước hết, hệ thống đóng vai trò quyết định trong việc phòng ngừa thảm họa giẫm đạp (Stampede Prevention). Tại các sự kiện lễ hội, nhà ga tàu điện ngầm hoặc sân vận động đông người, hệ thống liên tục giám sát mật độ cục bộ. Khi mật độ vượt ngưỡng an toàn (> 4 người$/\text{m}^2$), hệ thống tự động phát tín hiệu cảnh báo nguy cơ tắc nghẽn để lực lượng an ninh kịp thời phân luồng di chuyển trước khi sự cố xảy ra.

Thứ hai, dữ liệu đếm đám đông hỗ trợ tối ưu hóa phân bổ nguồn lực công cộng. Các cơ quan quản lý đô thị có thể dựa vào biểu đồ mật độ thời gian thực để linh hoạt điều phối tần suất xe buýt, tàu điện, bố trí lực lượng cứu hộ và mở thêm các làn thoát hiểm cục bộ.

Cuối cùng, trong lĩnh vực bán lẻ và quy hoạch không gian thương mại, việc phân tích lưu lượng người đi bộ qua từng gian hàng giúp các nhà quản lý xác định vị trí thu hút khách hàng để tối ưu hóa bố cục mặt bằng kinh doanh và định giá vị trí đặt biển quảng cáo một cách khách quan.

2.14 Phân tích Hệ thống Tái tạo Khuôn mặt 3D theo Mô hình Giải pháp 3 Tầng

Hệ thống tái tạo khuôn mặt 3D (3D Face Reconstruction) từ ảnh 2D đơn góc nhìn là ví dụ mẫu mực cho một kiến trúc phân tầng toàn diện từ cơ sở toán học hình học đến phần mềm nghiệp vụ hoàn chỉnh (Bảng 13).

Bảng 13: Phân tích hệ thống Tái tạo Khuôn mặt 3D theo Mô hình Giải pháp 3 Tầng

Phân tầng	Nhiệm vụ & Thành phần kỹ thuật	Chi tiết giải pháp công nghệ
Tầng 1: Kỹ thuật cốt lõi	Cơ sở hình học 3D, kiến trúc mạng nơ-ron và hàm tối ưu khả vi (Differentiable Rendering).	Mô hình tham số hình học 3D Morphable Model (3DMM như BFM, FLAME); Mạng Transformer trích xuất hệ số hình học $\boldsymbol{\alpha}$, biểu cảm $\boldsymbol{\beta}$ và tư thế $\mathbf{p}$; Hàm mất mát Photometric Consistency Loss, Landmark Reprojection Loss và Perceptual Identity Loss.
Tầng 2: Tác vụ thị giác cơ sở	Các khối trích xuất thông tin trung gian từ ảnh 2D sang không gian 3D.	Phát hiện 68 hoặc 468 điểm mốc không gian 3D; Ước lượng trường độ sâu đơn sắc (Monocular Depth Estimation); Tách bản đồ phản xạ ánh sáng bề mặt (Albedo/Texture UV Map) và góc chiếu sáng hình cầu (Spherical Harmonics).
Tầng 3: Ứng dụng nghiệp vụ hoàn chỉnh	Tích hợp vào các phần mềm ứng dụng phục vụ người dùng cuối.	• **Y tế & Thẩm mỹ:** Mô phỏng trực quan kết quả phẫu thuật chỉnh hình hàm mặt trước khi can thiệp y khoa. • **Giải trí & Game:** Tự động khởi tạo Digital Avatar 3D cá nhân hóa có khả năng biểu cảm thời gian thực trong game và thế giới thực tế ảo. • **Bảo mật sinh trắc học:** Xác thực khuôn mặt 3D chống giả mạo (Face Anti-spoofing) ngăn chặn các hành vi vượt rào bằng ảnh in 2D hoặc video kỹ thuật số.

3 Tuần 3: Ứng dụng Đồ họa Máy tính và Đề tài Nghiên cứu Sinh Đồ họa Vector (Text-to-Vector)

3.1 Bản chất và Mục tiêu của Đồ họa Máy tính

Đồ họa Máy tính (Computer Graphics) là một phân ngành nền tảng của khoa học máy tính, nghiên cứu các giải pháp phần cứng và phần mềm nhằm mô phỏng, biểu diễn và tổng hợp thế giới thực hoặc các thế giới trừu tượng dưới dạng thông tin thị giác và âm thanh kỹ thuật số (Hearn & Baker, 2004; Foley et al., 1996).

Về mặt lịch sử và lý thuyết mô hình hóa, quá trình phát triển của đồ họa máy tính phản ánh tiến trình nâng bậc không gian biểu diễn toán học từ thấp lên cao:

$$\text{2D } (x, y) \longrightarrow \text{3D } (x, y, z) \longrightarrow \text{4D } (x, y, z, t) \longrightarrow \text{5D } (x, y, z, t, s) \tag{4}$$

Trong chuỗi tiến trình này:

- **Không gian 2 chiều (2D):** Mô hình hóa các thực thể phẳng thông qua tọa độ (x, y), đặt nền móng cho việc xử lý văn bản, sơ đồ kỹ thuật số, giao diện người dùng và nghệ thuật đồ họa phẳng.
- **Không gian 3 chiều (3D):** Bổ sung trục độ sâu z, cho phép định nghĩa các khối hình học lập thể, bề mặt vật thể thực trong không gian Euclide thông qua lưới đa giác (Polygon Mesh) hoặc bề mặt tham số (Parametric Surfaces).
- **Không gian 4 chiều (4D):** Tích hợp thêm chiều thời gian t, chuyển đổi từ mô hình hình học tĩnh sang hoạt họa chuyển động (Computer Animation) và mô phỏng các hiện tượng động học vật lý thời gian thực.
- **Không gian 5 chiều (5D):** Bổ sung chiều ngữ cảnh, thang đo kích thước hoặc trạng thái cảm biến s (State / Scale / Semantics), phục vụ các hệ thống mô phỏng đa tỉ lệ, đa phương thức trong thực tế ảo tương tác (VR/AR) và các bản sao số kỹ thuật số (Digital Twins).

3.2 Phân tích Ứng dụng của Đồ họa Máy tính trong 7 Lĩnh vực Trọng yếu

Đồ họa máy tính đóng vai trò công nghệ cốt lõi trong toàn bộ nền kinh tế số. Bảng 14 phân tích chi tiết ứng dụng của đồ họa máy tính trên 7 lĩnh vực trọng yếu được giảng dạy trong học phần.

Bảng 14: Tổng hợp và đối chiếu ứng dụng của Đồ họa Máy tính trong 7 lĩnh vực công nghiệp cốt lõi

Lĩnh vực ứng dụng	**Đặc trưng kỹ thuật đồ họa cốt lõi**	**Hệ thống sản phẩm thực tế tiêu biểu**
1. Tự động hóa văn phòng (Office Automation)	Trực quan hóa dữ liệu thống kê (Bar, Pie, Line charts); giao diện người dùng đồ họa (GUI); biểu diễn biểu đồ luồng nghiệp vụ.	Bộ ứng dụng Microsoft Office, Google Workspace; phần mềm xây dựng Dashboard thông minh như Tableau, Microsoft PowerBI.
2. Xuất bản để bàn (Desktop Publishing - DTP)	Xử lý typography vector tham số hóa (TrueType, OpenType); phân tách màu CMYK; quản lý dàn trang độ phân giải cao; chuẩn PostScript/PDF.	Adobe InDesign, QuarkXPress, Scribus; hệ thống biên soạn tài liệu khoa học chất lượng cao LaTeX.
3. Thiết kế & Chế tạo trợ giúp máy tính (CAD-CAM)	Mô hình hóa hình học chính xác (NURBS, CSG, B-rep); phân tích ứng suất phần tử hữu hạn (FEA); xuất mã G-code cho máy tiện/phay CNC.	AutoCAD, Dassault Systèmes SolidWorks, CATIA, PTC Creo; phần mềm thiết kế vi mạch Altium Designer.
4. Nghệ thuật & Quảng cáo (Art + Advertising)	Chỉnh sửa ảnh raster đa lớp; thiết kế nhận diện thương hiệu vector; nghệ thuật minh họa số (Digital Painting); Typography 3D.	Adobe Photoshop, Adobe Illustrator, CorelDRAW; phần mềm vẽ kỹ thuật số Procreate, Clip Studio Paint.
5. Điều khiển quy trình (Process Control / SCADA)	Giao diện tương tác người máy (HMI) thời gian thực; sơ đồ động học hệ thống (Mimic Diagrams); hiển thị cảnh báo trực quan đa cấp độ.	Hệ thống SCADA giám sát nhà máy nhiệt điện, trạm biến áp, dây chuyền lắp ráp ô tô tự động, lọc dầu của Siemens, Schneider Electric.
6. Giải trí (Entertainment: Phim & Games)	Kết xuất quang học (Ray Tracing, Global Illumination); mô phỏng vật lý hạt (khói, lửa, nước); tạo hình nhân vật và diễn hoạt 3D thời gian thực.	Unreal Engine 5, Unity; phần mềm 3D chuyên nghiệp Autodesk Maya, Blender; phim hoạt hình của Pixar, Walt Disney.
7. Giáo dục & Đào tạo (Education)	Mô phỏng hình học không gian tương tác; phòng thí nghiệm ảo (Virtual Labs); buồng mô phỏng lái máy bay/tàu thủy (Flight Simulators).	Hệ thống mô phỏng bay CAE Flight Simulator; phần mềm toán học tương tác GeoGebra; phòng học ảo 3D mô phỏng giải phẫu học cơ thể người.

3.3 Đề tài Nghiên cứu Chuyên sâu: Sinh Đồ họa Vector từ Ngôn ngữ Tự nhiên (Text-to-Vector Graphics Generation)

Trong khuôn khổ môn học Đồ họa Máy tính, nhóm nghiên cứu lựa chọn đề tài **"Sinh Đồ họa Vector từ Ngôn ngữ Tự nhiên" (Text-to-Vector Graphics Generation, đặc biệt hướng tới định dạng chuẩn Scalable Vector Graphics - SVG)**. Dưới đây là phân tích toàn diện về bản chất học thuật và giá trị ứng dụng thực tiễn của đề tài.

3.3.1 Bản chất Học thuật và Thách thức Nghiên cứu

Trong khi lĩnh vực AI tạo sinh đã đạt được những bước tiến vượt bậc với việc sinh ảnh điểm ảnh (Raster Image Generation) nhờ các mạng khuếch tán (Diffusion Models) như Stable Diffusion hay Midjourney, việc sinh đồ họa vector lại đặt ra một thách thức hoàn toàn khác biệt về mặt bản thể luận:

1. Sự lệch pha giữa Trí tuệ Thị giác Nhân tạo và Tư duy Thiết kế Con người: Các mô hình học sâu truyền thống tư duy trên không gian lưới điểm ảnh cố định (Fixed Pixel Grids), nơi bức ảnh được biểu diễn bằng một ma trận tensor có kích thước xác định $I \in \mathbb{R}^{H \times W \times C}$. Ngược lại, đồ họa vector và tư duy của các nhà thiết kế đồ họa vận hành dựa trên các thực thể hình học tham số (Geometric Primitives) như đường cong Bézier bậc ba, hình tròn, đa giác và các lớp đối tượng phân cấp ngữ nghĩa (Hierarchical Layers). Sự chuyển dịch từ miền điểm ảnh rời rạc sang miền đường cong tham

số liên tục là một bài toán khó do tính chất phi khả vi (Non-differentiable) của các trình raster hóa truyền thống.

2. Toàn cảnh 5 hướng tiếp cận nghiên cứu học thuật hiện đại: Để giải quyết bài toán Text-to-Vector, cộng đồng khoa học quốc tế trong giai đoạn 2020 đến nay đã phát triển 5 nhánh tiếp cận chủ đạo:

1. **Tối ưu hóa lặp qua Trình kết xuất khả vi (Optimization-based Synthesis via Differentiable Rendering):** Khởi đầu từ việc Li et al. công bố DiffVG (Li et al., 2020), cho phép truyền ngược đạo hàm từ pixel về tọa độ điểm điều khiển Bézier. CLIPDraw (Frans et al., 2022) sử dụng không gian nhúng CLIP để tối ưu nét vẽ. Tiếp đó, VectorFusion (Jain et al., 2023) áp dụng Score Distillation Sampling (SDS) từ Stable Diffusion, và SVG-Dreamer (Xing et al., 2024) nâng cấp lên Variational Particle Score Distillation (VPSD) kết hợp phân rã chú ý chéo (Cross-Attention) để cải thiện độ sắc nét của hình vẽ vector.
2. **Biểu diễn Ẩn Thần kinh và Không gian Tiềm ẩn Đường cong (Neural Implicit & Latent Path Representations):** Nhằm khắc phục tình trạng đường cong bị răng cưa, tự cắt nhau và biến dạng khi tối ưu hóa điểm tự do, các phương pháp như T2V-NPR sử dụng VAE để học không gian tiềm ẩn của các đường cong chuẩn mực, trong khi NIVeL và NeuralSVG sử dụng mạng nơ-ron truyền thẳng (MLP) làm hàm biểu diễn ẩn, kết hợp kỹ thuật Nested Dropout để ép mạng học thứ tự lớp hình học từ lớn đến nhỏ.
3. **Mô hình Ngôn ngữ Lớn & Đa phương thức Sinh mã Trực tiếp (Autoregressive Code Generation with LLMs & MLLMs):** Coi tệp SVG như một ngôn ngữ hình thức dạng XML, các nghiên cứu mới nhất như StarVector (Rodriguez & et al., 2025) (CVPR 2025) và LLM4SVG (Xing & et al., 2025) (CVPR 2025) huấn luyện các mô hình đa phương thức trên hàng triệu tệp SVG thực tế, thiết kế các token ngữ nghĩa SVG chuyên biệt để sinh trực tiếp mã SVG sạch, giữ đúng các thẻ cơ sở như `<circle>`, `<rect>`, `<polygon>` chỉ trong vài chục giây suy luận.
4. **Lai ghép Ngôn ngữ và Thị giác (Hybrid LLM-Template + Image Diffusion Refinement):** Điển hình là Chat2SVG (CVPR 2025), giao nhiệm vụ kiến trúc sư phân rã bố cục ngữ nghĩa cho LLM (như Claude 3.5 Sonnet) để tạo khung mẫu, sau đó dùng mô hình khuếch tán ảnh và SAM để điêu khắc chi tiết vi mô cho từng thành phần.
5. **Mô phỏng Quy trình Tư duy Thiết kế Phân lớp bằng Diffusion Transformer (Process-Centric Layered Generation via DiT):** Nghiên cứu LayerTracer (ICCV 2025) học trực tiếp quy trình vẽ từng lớp của họa sĩ thông qua kiến trúc DiT (FLUX.1) trên bố cục hình rắn (Serpentine Layout), tạo ra bản thiết kế phân lớp hoàn toàn độc lập và khớp với tư duy con người.

3.3.2 Ý nghĩa và Giá trị Ứng dụng Thực tiễn

Định dạng đồ họa Scalable Vector Graphics (SVG) sở hữu những ưu thế kỹ thuật vượt trội không thể thay thế trong ngành công nghiệp đồ họa hiện đại:

- **Độ phân giải độc lập (Resolution Independence):** Hình ảnh vector có thể phóng to đến kích thước vô hạn mà hoàn toàn không bị hiện tượng vỡ hạt (Pixelation), đảm bảo chất lượng hiển thị sắc nét tuyệt đối từ màn hình đồng hồ thông minh, điện thoại, máy tính màn hình Retina/4K đến các biển quảng cáo tấm lớn ngoài trời.
- **Dung lượng siêu nhẹ và tối ưu hóa Web:** Tập tin SVG thực chất là văn bản mã nguồn XML có cấu trúc toán học gọn nhẹ (thường chỉ vài Kilobytes), nhẹ hơn hàng chục đến hàng trăm lần so với ảnh raster (PNG, JPEG), giúp giảm tải băng thông và tăng tốc độ tải trang web.
- **Tính khả biên tập và tái cấu trúc (High Editability):** Khác với ảnh raster là một khối pixel hợp nhất không thể tách rời, đồ họa vector lưu trữ từng đối tượng độc lập. Người dùng có thể dễ dàng nhấp chọn từng hình khối, thay đổi màu sắc, dịch chuyển vị trí, chỉnh sửa độ cong hoặc gán chuyển động hoạt hình bằng CSS/JavaScript mà không cần vẽ lại từ đầu.
- **Tự động hóa chu trình sản xuất công nghiệp và thiết kế UI/UX:** Ứng dụng mô hình Text-to-Vector cho phép các nhà thiết kế và lập trình viên tự động tạo lập hàng loạt biểu

tượng (Icons), hình minh họa đồ họa (Illustrations), sơ đồ giao diện người dùng và tệp cắt kỹ thuật số (Laser cutting, CNC) chỉ từ các câu lệnh mô tả ngắn, rút ngắn thời gian thiết kế từ nhiều giờ xuống còn vài giây.

4 Tuần 4: Hệ Tọa độ, Cơ chế Chiếu và Giải thuật Vẽ Đoạn thẳng, Đường tròn

4.1 Bản chất và Phân định Ba Hệ Tọa độ trong Đồ họa Máy tính (WCS, UCS, DCS)

Trong quy trình kết xuất và xử lý đồ họa máy tính, việc quản lý không gian hình học được phân định thông qua ba hệ tọa độ nền tảng, phản ánh các giai đoạn chuyển hóa dữ liệu từ mô hình trừu tượng sang màn hình hiển thị vật lý:

1. Hệ Tọa độ Thế giới Thực (World Coordinate System, WCS): WCS là hệ tọa độ Decartes tuyệt đối toàn cục được dùng làm chuẩn tham chiếu chung cho toàn bộ không gian cảnh đồ họa. Trong hệ tọa độ này, mọi đối tượng, nguồn sáng, góc nhìn camera và các điều kiện biên môi trường đều được xác định vị trí tương đối với nhau. Đơn vị đo trong WCS mang ý nghĩa vật lý thực tế (như mét, milimét, kilômét hoặc inch), cho phép mô hình hóa các vật thể theo đúng tỉ lệ kích thước tự nhiên.

2. Hệ Tọa độ Người dùng / Mô hình hóa (User Coordinate System, UCS / Modeling Coordinate System, MCS): UCS là hệ tọa độ cục bộ độc lập do người thiết kế hoặc lập trình viên tự định nghĩa cho từng đối tượng hình học riêng lẻ. Việc sử dụng UCS giúp quá trình tạo dựng mô hình trở nên thuận tiện và trực quan, bởi gốc tọa độ thường được đặt tại tâm đối xứng hoặc một đỉnh đặc trưng của vật thể. Người dùng có thể tự do tịnh tiến, xoay hoặc thay đổi hướng của các trục tọa độ cục bộ này trong quá trình thiết kế. Để đưa một vật thể từ không gian mô hình hóa UCS vào không gian chung WCS, hệ thống đồ họa áp dụng phép biến đổi mô hình (Modeling Transformation) thông qua ma trận biến đổi affine:

$$\mathbf{P}_{\text{WCS}} = \mathbf{M}_{\text{model}} \cdot \mathbf{P}_{\text{UCS}} \tag{5}$$

3. Hệ Tọa độ Thiết bị / Màn hình Hiển thị (Device Coordinate System, DCS / Screen Coordinate System): DCS là hệ tọa độ 2D rời rạc tương ứng trực tiếp với lưới điểm ảnh vật lý của thiết bị xuất (màn hình hiển thị hoặc máy in). Trục tọa độ trong DCS được định nghĩa bằng các giá trị số nguyên:

$$x_{\text{DCS}} \in \{0, 1, \ldots, X_{\max}\}, \quad y_{\text{DCS}} \in \{0, 1, \ldots, Y_{\max}\} \tag{6}$$

Gốc tọa độ $(0, 0)$ của DCS thường được quy ước đặt tại góc trên bên trái (phổ biến trong hệ thống cửa sổ đồ họa máy tính) hoặc góc dưới bên trái của màn hình.

Để chuyển đổi một vùng quan sát trong thế giới thực sang một khung nhìn trên màn hình, đồ họa máy tính thực thi phép biến đổi Cửa sổ sang Khung nhìn (Window-to-Viewport Transformation, WinToView). Nếu cửa sổ quan sát trong thế giới thực được giới hạn bởi $[x_{w\,\min}, x_{w\,\max}] \times [y_{w\,\min}, y_{w\,\max}]$ và khung nhìn màn hình được giới hạn bởi $[x_{v\,\min}, x_{v\,\max}] \times [y_{v\,\min}, y_{v\,\max}]$, tọa độ hiển thị (x_v, y_v) được ánh xạ tuyến tính theo công thức:

$$x_v = x_{v\,\min} + (x_w - x_{w\,\min}) \frac{x_{v\,\max} - x_{v\,\min}}{x_{w\,\max} - x_{w\,\min}}, \quad y_v = y_{v\,\min} + (y_w - y_{w\,\min}) \frac{y_{v\,\max} - y_{v\,\min}}{y_{w\,\max} - y_{w\,\min}} \tag{7}$$

4.2 Cơ sở Lý thuyết: Mô phỏng Thế giới Thực thông qua Biểu diễn Vector Kích thước Nhỏ

Trong đồ họa máy tính, một nghịch lý thú vị thường được đặt ra: *Tại sao máy tính có thể mô phỏng chính xác thế giới thực rộng lớn khi kích thước lưu trữ của các vector dữ liệu lại vô cùng nhỏ gọn?*

Bản chất của vấn đề nằm ở hai nguyên lý khoa học nền tảng:

1. Trừu tượng hóa Toán học và Biểu diễn Tham số: Đồ họa vector không lưu trữ hàng tỷ hạt vật chất hay hàng triệu điểm ảnh rời rạc của thế giới thực. Thay vào đó, nó trừu tượng hóa thế giới thành các quy luật toán học hình học tham số (Parametric Geometric Models). Ví dụ, một mặt cầu trong không gian thực có kích thước bán kính hàng nghìn kilômét chỉ cần được biểu diễn bằng một bộ 4 số thực: tọa độ tâm (x_c, y_c, z_c) và bán kính r. Tương tự, một bề mặt khí động học phức tạp của cánh máy bay được mô tả chính xác tuyệt đối bằng một lưới tập hợp vài chục điểm điều khiển của mặt cong NURBS (Non-Uniform Rational B-Splines). Dung lượng của các tham số số học này chỉ chiếm vài chục đến vài trăm byte nhớ, nhưng mang đầy đủ thông tin hình học giải tích chuẩn xác.

2. Tính Bất biến về Thang đo và Ánh xạ Tỉ lệ Hình học (Scale Invariance): Mô hình toán học hoàn toàn độc lập với kích thước vật lý của màn hình hiển thị. Nhờ phép biến đổi tỉ lệ (Scaling

Transformation) thông qua ma trận đại số:

$$\mathbf{S}(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{8}$$

hệ thống đồ họa có thể co giãn thế giới mô phỏng về bất kỳ thang đo kích thước nào. Cùng một công thức vector, hệ thống có thể phóng to hàng triệu lần để quan sát cấu trúc tinh thể vi mô của phân tử nano, hoặc thu nhỏ hàng tỷ lần để mô phỏng quỹ đạo chuyển động của các thiên thể trong hệ ngân hà trên màn hình máy tính có kích thước chỉ vài chục inch.

4.3 Cơ chế Mô phỏng Đối tượng Thực 3 Chiều trên Màn hình Hiển thị 2 Chiều

Màn hình máy tính về bản chất vật lý là một bề mặt phẳng hai chiều. Để tái tạo chân thực cảm giác không gian ba chiều có chiều sâu cho mắt người, đồ họa máy tính triển khai một chuỗi kỹ thuật kết xuất liên hoàn (Viewing Pipeline) (Hearn & Baker, 2004):

1. Phép Chiếu Hình học (Geometric Projections): Ánh xạ các điểm từ không gian 3 chiều (x, y, z) xuống mặt phẳng quan sát 2 chiều (x_p, y_p):

- *Phép chiếu Song song (Parallel / Orthographic Projection):* Các tia chiếu song song với nhau. Phép chiếu này bảo toàn tính song song của các đường thẳng và giữ nguyên tỉ lệ kích thước thực của đối tượng, không phụ thuộc vào khoảng cách tới người quan sát. Phương pháp này là tiêu chuẩn bắt buộc trong các bản vẽ kỹ thuật cơ khí và kiến trúc xây dựng (CAD).
- *Phép chiếu Phối cảnh (Perspective Projection):* Các tia chiếu cùng hội tụ về một điểm gốc gọi là tâm chiếu (Center of Projection, tương đương vị trí mắt người quan sát hoặc ống kính máy ảnh). Phép chiếu phối cảnh tạo ra hiện tượng biến dạng xa gần tự nhiên (Foreshortening): vật thể càng ở xa tâm chiếu thì kích thước hình chiếu trên mặt phẳng chiếu càng nhỏ, và các đường thẳng song song trong không gian sẽ hội tụ tại một điểm tụ (Vanishing Point). Ma trận chiếu phối cảnh chuẩn hóa với khoảng cách tiêu cự d có dạng:

$$x_p = \frac{x}{z/d}, \quad y_p = \frac{y}{z/d} \tag{9}$$

2. Thuật toán Khử Mặt Khuất (Hidden Surface Removal, HSR): Trong không gian thực, các vật thể phía trước sẽ che khuất các vật thể phía sau. Đồ họa máy tính sử dụng thuật toán Đệm độ sâu (Z-Buffer / Depth-Buffer Algorithm). Tại mỗi pixel (x, y) trên màn hình, hệ thống duy trì giá trị độ sâu z nhỏ nhất (gần người quan sát nhất). Khi một đa giác mới được raster hóa, giá trị z_{new} của nó sẽ được so sánh với $z_{\text{buffer}}(x, y)$; nếu $z_{\text{new}} < z_{\text{buffer}}(x, y)$, điểm ảnh mới sẽ được cập nhật màu sắc và ghi đè giá trị độ sâu mới vào bộ đệm.

3. Mô hình Chiếu sáng và Tô bóng (Illumination and Shading): Để bề mặt vật thể thể hiện được độ cong và chất liệu thực tế, hệ thống áp dụng các mô hình quang học mô phỏng tương tác giữa ánh sáng và bề mặt vật liệu, tiêu biểu là Mô hình Chiếu sáng Phong (Phong Illumination Model):

$$I = I_a k_a + I_d k_d (\mathbf{L} \cdot \mathbf{N}) + I_s k_s (\mathbf{R} \cdot \mathbf{V})^n \tag{10}$$

trong đó $I_a k_a$ là thành phần ánh sáng môi trường (Ambient), $I_d k_d (\mathbf{L} \cdot \mathbf{N})$ là ánh sáng khuếch tán Lambertian (Diffuse), và $I_s k_s (\mathbf{R} \cdot \mathbf{V})^n$ là ánh sáng phản xạ gương tạo vệt sáng bóng (Specular Highlight). Sau đó, kỹ thuật tô bóng Gouraud Shading (nội suy màu sắc qua các đỉnh) hoặc Phong Shading (nội suy vector pháp tuyến $\mathbf{N}$ qua từng điểm ảnh) được áp dụng để loại bỏ cảm giác đa giác sắc cạnh thô ráp.

4. Ánh xạ Vân Bề mặt (Texture Mapping): Dán các bức ảnh 2D (Textures) lên bề mặt các đa giác 3D kết hợp với kỹ thuật Bump Mapping hoặc Normal Mapping để mô phỏng độ nhám, nếp nhăn và kết cấu chi tiết của vật liệu (gỗ, đá, da, kim loại) mà không làm tăng số lượng đa giác hình học cần xử lý.

4.4 Phân tích Toàn diện Giải thuật Vẽ Đoạn thẳng và Khảo sát Độ dốc

Bài toán vẽ đoạn thẳng trên lưới điểm ảnh số yêu cầu xác định tập hợp các pixel nguyên (x_k, y_k) sao cho đường gấp khúc nối các pixel này xấp xỉ tốt nhất đoạn thẳng giải tích nối hai điểm mút $A(x_a, y_a)$ và $B(x_b, y_b)$, đồng thời đảm bảo tính liên tục không gian trong lân cận 8 pixel.

Phương trình tường minh của đoạn thẳng trong mặt phẳng 2D có dạng:

$$y = m \cdot x + b \tag{11}$$

trong đó hệ số góc (độ dốc) m và hằng số dịch chuyển b được xác định bởi:

$$m = \frac{\Delta y}{\Delta x} = \frac{y_b - y_a}{x_b - x_a}, \quad b = y_a - m \cdot x_a \tag{12}$$

Từ định nghĩa vi sai $\Delta y = m \cdot \Delta x$, ta có mối quan hệ độ lớn biến thiên:

$$|\Delta y| = |m| \cdot |\Delta x| \tag{13}$$

Biểu thức này dẫn đến hai trường hợp cơ bản quyết định giải thuật tạo điểm ảnh:

Trường hợp 1: Độ dốc tuyệt đối nhỏ hơn hoặc bằng 1 ($|m| \leq 1 \iff |\Delta y| \leq |\Delta x|$): Trong trường hợp này, chiều dài biến thiên theo trục hoành x lớn hơn hoặc bằng chiều dài biến thiên theo trục tung y. Nếu ta chọn bước nhảy đơn vị theo trục hoành là $\Delta x = \pm 1$, khi đó bước nhảy tương ứng của tung độ thỏa mãn:

$$|\Delta y| = |m| \cdot 1 \leq 1 \tag{14}$$

Điều này có nghĩa là mỗi khi x tăng đúng 1 pixel, giá trị thực của y biến thiên một khoảng không vượt quá 1 pixel. Nhờ đó, tại mỗi bước, pixel kế tiếp chỉ dịch chuyển sang cột bên cạnh và tung độ chỉ thay đổi tối đa 1 đơn vị, đảm bảo đoạn thẳng vẽ ra liên tục, không bao giờ xuất hiện hiện tượng gián đoạn hay thủng lỗ theo phương ngang. Do đó, x được chọn làm biến độc lập tăng đều, và tung độ tại bước kế tiếp được tính theo công thức:

$$x_{k+1} = x_k + 1, \quad y_{k+1} = y_k + m \tag{15}$$

Trường hợp 2: Độ dốc tuyệt đối lớn hơn 1 ($|m| > 1 \iff |\Delta y| > |\Delta x|$): Nếu trong trường hợp này ta vẫn tiếp tục chọn $\Delta x = \pm 1$, bước nhảy tung độ sẽ là $|\Delta y| = |m| > 1$. Khi đó, giá trị y sẽ nhảy cóc qua nhiều pixel, tạo thành các khoảng trống gián đoạn nghiêm trọng trên màn hình theo phương thẳng đứng. Để khắc phục triệt để, ta phải hoán đổi vai trò của hai trục: chọn trục tung y làm biến độc lập tăng đều với $\Delta y = \pm 1$. Khi đó, bước nhảy tương ứng theo trục hoành thỏa mãn:

$$|\Delta x| = \frac{1}{|m|} \cdot |\Delta y| = \frac{1}{|m|} < 1 \tag{16}$$

Lúc này, mỗi khi y tăng đúng 1 pixel, x chỉ biến thiên một khoảng nhỏ hơn 1 pixel, đảm bảo tính liên tục tuyệt đối trong lân cận 8 pixel theo phương thẳng đứng:

$$y_{k+1} = y_k + 1, \quad x_{k+1} = x_k + \frac{1}{m} \tag{17}$$

Phương pháp này chính là nền tảng của giải thuật DDA (Digital Differential Analyzer). Tuy nhiên, DDA đòi hỏi các phép tính số thực dấu phẩy động và hàm làm tròn `round()` tại mọi bước lặp, gây suy giảm tốc độ kết xuất đồ họa.

4.5 Cơ sở Giới hạn Bài toán $0 < m < 1, x_a < x_b$ và Cơ chế Suy diễn Toàn diện

Trong nghiên cứu thuật toán vẽ đoạn thẳng, các nhà khoa học luôn giới hạn bài toán khảo sát ban đầu về trường hợp chuẩn tắc:

$$0 < m < 1 \quad \text{và} \quad x_a < x_b \tag{18}$$

tương ứng với góc nghiêng $\theta \in (0^\circ, 45^\circ)$ trong Bát phân thứ nhất (Octant 1).

Lý do Giới hạn Bài toán: Việc giới hạn vào khoảng này giúp tối giản hóa công thức toán học và cấu trúc rẽ nhánh của giải thuật. Khi $x_a < x_b$, quá trình quét luôn tiến từ trái sang phải ($\Delta x =$

$x_b - x_a > 0$). Khi $0 < m < 1$, tại mỗi bước hoành độ luôn tăng đúng 1 đơn vị ($x_{k+1} = x_k + 1$), và hệ thống chỉ cần đưa ra quyết định nhị phân duy nhất giữa hai pixel lân cận: pixel nằm ngang $(x_k + 1, y_k)$ hoặc pixel nằm chéo lên $(x_k + 1, y_k + 1)$.

Cơ chế Suy diễn cho Toàn bộ các Trường hợp còn lại: Mọi đoạn thẳng bất kỳ trong không gian 2D đều có thể quy về trường hợp chuẩn tắc thông qua các phép biến đổi đối xứng hình học và xử lý điều kiện biên:

1. **Đảm bảo chiều quét từ trái sang phải ($x_a < x_b$):** Nếu $x_a > x_b$, ta chỉ việc hoán đổi hai điểm mút: gán $A \leftrightarrow B$. Khi đó đoạn thẳng được vẽ từ B sang A với cùng tập hợp các pixel nhưng đảm bảo $\Delta x > 0$.
2. **Các trường hợp suy biến đặc biệt:**
   - Đoạn thẳng nằm ngang ($m = 0 \iff y_a = y_b$): Giữ nguyên $y = y_a$, tăng dần x từ x_a đến x_b và vẽ pixel (x, y_a).
   - Đoạn thẳng đứng ($m \to \infty \iff x_a = x_b$): Giữ nguyên $x = x_a$, tăng dần y từ $\min(y_a, y_b)$ đến $\max(y_a, y_b)$ và vẽ pixel (x_a, y).
   - Đoạn thẳng nghiêng đúng $45°$ ($|m| = 1$): Cả x và y cùng tăng hoặc giảm đồng thời 1 đơn vị ở mỗi bước lặp ($x_{k+1} = x_k + 1, y_{k+1} = y_k \pm 1$).
3. **Suy diễn cho 8 Bát phân tổng quát (Octants):** Dựa vào dấu của $\Delta x, \Delta y$ và so sánh độ lớn giữa $|\Delta x|$ và $|\Delta y|$:
   - Nếu $|\Delta y| \leq |\Delta x|$: Trục x tăng đều từng bước 1 ($s_x = \text{sign}(\Delta x) = 1$). Nếu $m > 0$, y tăng về phía dương ($s_y = 1$); nếu $m < 0$, y biến thiên về phía âm ($s_y = -1$).
   - Nếu $|\Delta y| > |\Delta x|$: Hoán đổi vai trò hai trục tọa độ ($x \leftrightarrow y$). Trục y tăng đều từng bước 1 ($s_y = \text{sign}(\Delta y) = 1$), và trục x được cập nhật dựa trên hàm quyết định tương ứng.

4.6 Cơ chế Khử Phép tính Số thực của Thuật toán Bresenham Đoạn thẳng

Năm 1965, Jack E. Bresenham đã đề xuất giải thuật kinh điển mang tên ông (Bresenham, 1965), loại bỏ hoàn toàn các phép toán số thực và hàm làm tròn của thuật toán DDA, chuyển toàn bộ quá trình tính toán về các phép cộng và trừ số nguyên thuần túy.

Xét đoạn thẳng trong bát phân thứ nhất với $0 < m < 1$ và $x_a < x_b$. Giả sử tại bước thứ k, thuật toán đã xác định pixel cần vẽ là (x_k, y_k). Ở bước kế tiếp $k + 1$, hoành độ chắc chắn là $x_{k+1} = x_k + 1$. Ta cần lựa chọn giữa hai pixel ứng viên:

$$\text{Pixel dưới: } (x_k + 1, y_k) \quad \text{hoặc} \quad \text{Pixel trên: } (x_k + 1, y_k + 1) \tag{19}$$

Tung độ thực tế của điểm nằm trên đoạn thẳng tại hoành độ $x_k + 1$ là:

$$y = m(x_k + 1) + b \tag{20}$$

Khoảng cách thẳng đứng từ tung độ thực tế y đến pixel dưới và pixel trên lần lượt là:

$$d_1 = y - y_k = m(x_k + 1) + b - y_k \tag{21}$$
$$d_2 = (y_k + 1) - y = y_k + 1 - [m(x_k + 1) + b] \tag{22}$$

Hiệu số giữa hai khoảng cách này phản ánh pixel nào nằm gần đoạn thẳng thực tế hơn:

$$\begin{aligned} d_1 - d_2 &= [m(x_k + 1) + b - y_k] - [y_k + 1 - m(x_k + 1) - b] \\ &= 2m(x_k + 1) - 2y_k + 2b - 1 \end{aligned} \tag{23}$$

Thay $m = \frac{\Delta y}{\Delta x}$ vào biểu thức:

$$d_1 - d_2 = 2\frac{\Delta y}{\Delta x}(x_k + 1) - 2y_k + 2b - 1 \tag{24}$$

Đại lượng này vẫn chứa phép chia số thực $\frac{\Delta y}{\Delta x}$. Nhận thấy rằng $\Delta x > 0$, Bresenham đã nhân toàn bộ biểu thức trên với Δx để định nghĩa hàm tham số quyết định (Decision Parameter) p_k:

$$p_k = \Delta x(d_1 - d_2) = 2\Delta y \cdot x_k - 2\Delta x \cdot y_k + C \tag{25}$$

trong đó hằng số $C = 2\Delta y + \Delta x(2b - 1)$.

Quy tắc chọn pixel dựa trên dấu của p_k:

- Nếu $p_k < 0 \iff d_1 < d_2$: Đoạn thẳng nằm gần pixel dưới hơn, do đó ta chọn pixel $(x_k + 1, y_k)$.
- Nếu $p_k \geq 0 \iff d_1 \geq d_2$: Đoạn thẳng nằm gần pixel trên hơn (hoặc cách đều), ta chọn pixel $(x_k + 1, y_k + 1)$.

Để thiết lập công thức tính truy hồi, ta xét giá trị p_{k+1} tại bước kế tiếp:

$$p_{k+1} = 2\Delta y \cdot x_{k+1} - 2\Delta x \cdot y_{k+1} + C \tag{26}$$

Lấy hiệu số giữa p_{k+1} và p_k:

$$\begin{aligned} p_{k+1} - p_k &= 2\Delta y(x_{k+1} - x_k) - 2\Delta x(y_{k+1} - y_k) \\ &= 2\Delta y(1) - 2\Delta x(y_{k+1} - y_k) \end{aligned} \tag{27}$$

Do đó, công thức truy hồi cập nhật tham số quyết định có dạng:

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x(y_{k+1} - y_k) \tag{28}$$

Từ đây, ta có hai trường hợp cập nhật cụ thể:

- Nếu $p_k < 0$: Chọn $y_{k+1} = y_k \implies y_{k+1} - y_k = 0$, ta cập nhật:
$$p_{k+1} = p_k + 2\Delta y \tag{29}$$
- Nếu $p_k \geq 0$: Chọn $y_{k+1} = y_k + 1 \implies y_{k+1} - y_k = 1$, ta cập nhật:
$$p_{k+1} = p_k + 2\Delta y - 2\Delta x \tag{30}$$

Vì các điểm mút ban đầu có tọa độ nguyên, nên $\Delta x, \Delta y, 2\Delta y$ và $2\Delta y - 2\Delta x$ đều là các hằng số nguyên. Toàn bộ thuật toán hoàn toàn không sử dụng bất kỳ phép tính số thực hay phép chia nào, mà chỉ gồm phép cộng, trừ số nguyên và phép nhân 2 (có thể tối ưu bằng phép dịch bit sang trái « `1`), mang lại hiệu năng tính toán cực cao trên phần cứng.

4.7 Chứng minh Công thức Khởi tạo Giá trị p_0

Để hoàn thiện giải thuật truy hồi Bresenham, ta cần tính giá trị ban đầu p_0 tại bước $k = 0$:

Điểm mút đầu tiên $(x_0, y_0) = (x_a, y_a)$ nằm chính xác trên đường thẳng giải tích, do đó thỏa mãn:

$$y_0 = m \cdot x_0 + b \implies b = y_0 - m \cdot x_0 = y_0 - \frac{\Delta y}{\Delta x}x_0 \tag{31}$$

Tại bước đầu tiên, ta cần quyết định pixel cho cột $x_1 = x_0 + 1$. Áp dụng công thức tổng quát của tham số quyết định:

$$p_0 = \Delta x[2m(x_0 + 1) - 2y_0 + 2b - 1] \tag{32}$$

Thay $m = \frac{\Delta y}{\Delta x}$ và $b = y_0 - \frac{\Delta y}{\Delta x}x_0$ vào biểu thức:

$$\begin{aligned} p_0 &= \Delta x \left[2\frac{\Delta y}{\Delta x}(x_0 + 1) - 2y_0 + 2\left(y_0 - \frac{\Delta y}{\Delta x}x_0\right) - 1\right] \\ &= \Delta x \left[2\frac{\Delta y}{\Delta x}x_0 + 2\frac{\Delta y}{\Delta x} - 2y_0 + 2y_0 - 2\frac{\Delta y}{\Delta x}x_0 - 1\right] \\ &= \Delta x \left[2\frac{\Delta y}{\Delta x} - 1\right] \end{aligned} \tag{33}$$

Nhân phân phối Δx vào bên trong dấu ngoặc vuông:

$$p_0 = 2\Delta y - \Delta x \tag{34}$$

Đây là công thức khởi tạo chuẩn xác của thuật toán vẽ đoạn thẳng Bresenham, chứng minh rằng p_0 luôn luôn là một giá trị nguyên.

4.8 Cơ sở Vẽ Hình tròn trên Lưới Tọa độ Nguyên và Tính Đối xứng 8 Phần

Phương trình đại số của đường tròn tâm tại gốc tọa độ $O(0,0)$ với bán kính r có dạng:

$$x^2 + y^2 = r^2 \tag{35}$$

Hàm đường tròn được định nghĩa tổng quát bằng:

$$f_{\text{circle}}(x,y) = x^2 + y^2 - r^2 \tag{36}$$

Vị trí tương đối của một điểm tọa độ bất kỳ (x,y) so với đường biên đường tròn được phân loại chặt chẽ qua dấu của hàm:

$$f_{\text{circle}}(x,y) = \begin{cases} < 0, & \text{nếu } (x,y) \text{ nằm phía trong đường tròn} \\ = 0, & \text{nếu } (x,y) \text{ nằm chính xác trên đường biên đường tròn} \\ > 0, & \text{nếu } (x,y) \text{ nằm phía ngoài đường tròn} \end{cases} \tag{37}$$

Nguyên lý Đối xứng 8 Phần (8-Way Symmetry): Đường tròn tâm $O(0,0)$ sở hữu tính đối xứng hoàn hảo qua trục hoành $y = 0$, trục tung $x = 0$ và hai đường phân giác $y = x$, $y = -x$. Do đó, toàn bộ chu vi đường tròn được chia thành 8 cung bát phân bằng nhau.

Nếu thuật toán tính toán được tập hợp các pixel trên cung bát phân thứ hai (từ điểm đỉnh $(0,r)$ ở góc 90° xuống giao điểm với đường phân giác tại góc 45°, nơi $x = y$), ta có thể tự động suy ra 7 điểm còn lại trên các cung khác chỉ bằng phép đổi dấu và hoán vị tọa độ:

$$(x,y), \quad (y,x), \quad (-x,y), \quad (-y,x), \quad (-x,-y), \quad (-y,-x), \quad (x,-y), \quad (y,-x) \tag{38}$$

Sau đó, nếu đường tròn có tâm thực tế tại $C(x_c, y_c)$, ta chỉ cần thực hiện phép tịnh tiến bằng cách cộng thêm (x_c, y_c) vào từng tọa độ đã tính.

4.9 Phân tích Tính Liên tục 8-Lân cận: Vì sao x phải Tăng đều trong Cung $[90^\circ, 45^\circ]$

Trong cung bát phân khảo sát (từ điểm đỉnh $(0,r)$ đến điểm giao $x = y$), câu hỏi đặt ra là: *Để đảm bảo đường cong liên tục không bị thủng lỗ trong lân cận 8 pixel, ta phải chọn biến x hay y tăng đều?*

Lấy đạo hàm hai vế phương trình đường tròn $x^2 + y^2 = r^2$ theo biến x:

$$2x + 2y\frac{dy}{dx} = 0 \implies \frac{dy}{dx} = -\frac{x}{y} \tag{39}$$

Độ dốc tiếp tuyến của cung tròn có độ lớn là:

$$|m| = \left|\frac{dy}{dx}\right| = \frac{x}{y} \tag{40}$$

Trên cung bát phân khảo sát, hoành độ x bắt đầu từ 0 và tăng dần đến khi $x = y$, do đó ta luôn có bất đẳng thức:

$$0 \le x \le y \implies 0 \le \frac{x}{y} \le 1 \implies |m| \le 1 \tag{41}$$

Vì độ lớn độ dốc $|m| \le 1$, tốc độ biến thiên của tung độ y luôn chậm hơn hoặc bằng tốc độ biến thiên của hoành độ x. Nếu ta chọn y làm biến tăng đều, bước nhảy của x sẽ lớn hơn 1 pixel ($\Delta x > 1$), khiến các pixel trên đường tròn bị ngắt quãng, tạo ra các lỗ hổng thị giác.

Do đó, **bắt buộc x phải là biến tăng đều từng bước nguyên đơn vị**:

$$x_{k+1} = x_k + 1 \tag{42}$$

Khi x tăng đúng 1 pixel, tung độ thực tế y giảm một khoảng nhỏ hơn hoặc bằng 1 pixel ($0 \le -\Delta y \le 1$). Vì vậy, pixel kế tiếp chỉ có đúng hai khả năng lựa chọn: giữ nguyên tung độ ($y_{k+1} = y_k$) hoặc giảm đúng 1 đơn vị ($y_{k+1} = y_k - 1$). Điều này đảm bảo đường cong tròn hoàn toàn liên tục và kết dính trong lân cận 8 pixel.

4.10 Chứng minh Hình học và Đại số của Thuật toán Điểm Giữa (Midpoint Circle Algorithm)

Thuật toán Điểm Giữa (Midpoint Circle Algorithm) do Jack E. Bresenham phát triển (Bresenham, 1977) giải quyết bài toán chọn pixel tối ưu bằng cách đánh giá vị trí hình học của điểm chính giữa hai pixel ứng viên:

Giả sử tại bước k, thuật toán đã chọn pixel (x_k, y_k). Tại bước $k+1$, hoành độ là $x_{k+1} = x_k + 1$. Hai pixel ứng viên là:

$$\text{Pixel trên (hướng Đông, East):} \quad E(x_k + 1, y_k) \tag{43}$$

$$\text{Pixel dưới (hướng Đông Nam, SouthEast):} \quad SE(x_k + 1, y_k - 1) \tag{44}$$

Điểm chính giữa (Midpoint) M nằm giữa hai pixel ứng viên E và SE có tọa độ:

$$M = \left(x_k + 1, y_k - \frac{1}{2}\right) \tag{45}$$

Tham số quyết định tại bước k chính là giá trị của hàm đường tròn tính tại điểm giữa M:

$$p_k = f_{\text{circle}}(M) = f_{\text{circle}}\left(x_k + 1, y_k - \frac{1}{2}\right) = (x_k + 1)^2 + \left(y_k - \frac{1}{2}\right)^2 - r^2 \tag{46}$$

Khoảng cách từ gốc tọa độ $O(0,0)$ đến điểm giữa M là $d_M = \sqrt{(x_k+1)^2 + (y_k - 1/2)^2}$, do đó:

$$p_k = d_M^2 - r^2 \tag{47}$$

Chứng minh hai trường hợp quyết định:

- **Trường hợp 1:** $p_k < 0 \iff d_M^2 < r^2 \iff d_M < r$**:** Điểm giữa M nằm ở phía trong đường tròn thực tế. Điều này đồng nghĩa với việc đường tròn thực tế đi qua phía trên điểm giữa M. Khoảng cách từ tâm đến pixel trên E là $d_E > d_M$, trong khi khoảng cách đến pixel dưới là $d_{SE} < d_M$. Vì bán kính đường tròn r thỏa mãn $d_M < r < d_E$, nên cung tròn nằm kẹp giữa M và E. Do đó, khoảng cách từ cung tròn đến pixel trên E nhỏ hơn khoảng cách đến pixel dưới SE:
$$|d_E - r| < |d_{SE} - r| \tag{48}$$
Như vậy, pixel trên $E(x_k + 1, y_k)$ nằm gần đường tròn hơn, và ta chọn $y_{k+1} = y_k$.
- **Trường hợp 2:** $p_k \geq 0 \iff d_M^2 \geq r^2 \iff d_M \geq r$**:** Điểm giữa M nằm ở phía ngoài hoặc nằm chính xác trên đường biên đường tròn. Điều này đồng nghĩa với việc đường tròn thực tế đi qua phía dưới điểm giữa M. Khi đó, bán kính đường tròn r thỏa mãn $d_{SE} < r \leq d_M$, tức là cung tròn nằm kẹp giữa SE và M. Do đó, khoảng cách từ cung tròn đến pixel dưới SE nhỏ hơn hoặc bằng khoảng cách đến pixel trên E:
$$|d_{SE} - r| \leq |d_E - r| \tag{49}$$
Như vậy, pixel dưới $SE(x_k + 1, y_k - 1)$ nằm gần đường tròn hơn (hoặc cách đều), và ta chọn $y_{k+1} = y_k - 1$.

Công thức truy hồi cập nhật p_{k+1}: Tại bước kế tiếp $k+1$, tham số quyết định được tính tại $x_{k+1}+1$:

$$p_{k+1} = f_{\text{circle}}\left(x_{k+1} + 1, y_{k+1} - \frac{1}{2}\right) = (x_{k+1} + 1)^2 + \left(y_{k+1} - \frac{1}{2}\right)^2 - r^2 \tag{50}$$

Lấy hiệu số $p_{k+1} - p_k$:

$$\begin{aligned} p_{k+1} - p_k &= \left[(x_{k+1}+1)^2 - (x_k+1)^2\right] + \left[\left(y_{k+1} - \frac{1}{2}\right)^2 - \left(y_k - \frac{1}{2}\right)^2\right] \\ &= (x_{k+1}+1)^2 - x_{k+1}^2 + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k) \\ &= 2x_{k+1} + 1 + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k) \end{aligned} \tag{51}$$

Từ đây, ta thu được hai công thức cập nhật tương ứng:

- Nếu $p_k < 0$: Chọn $y_{k+1} = y_k$, biểu thức trở thành:

$$p_{k+1} = p_k + 2x_{k+1} + 1 \tag{52}$$

- Nếu $p_k \geq 0$: Chọn $y_{k+1} = y_k - 1$, biểu thức trở thành:

$$p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1} \tag{53}$$

4.11 Chứng minh Thay thế Khử Số thực: $p_0 = \frac{5}{4} - r$ bằng $p'_0 = 1 - r$ khi r Nguyên

Tại bước khởi tạo ban đầu, thuật toán bắt đầu từ đỉnh trên cùng của đường tròn:

$$(x_0, y_0) = (0, r) \tag{54}$$

Giá trị tham số quyết định ban đầu p_0 được tính tại điểm giữa đầu tiên $M_0 = (1, r - 1/2)$:

$$p_0 = f_{\text{circle}}\left(1, r - \frac{1}{2}\right) = 1^2 + \left(r - \frac{1}{2}\right)^2 - r^2 = 1 + r^2 - r + \frac{1}{4} - r^2 = \frac{5}{4} - r \tag{55}$$

Biểu thức này chứa phân số $\frac{1}{4}$, khiến phép tính khởi tạo ban đầu đòi hỏi số thực:

$$p_0 = 1 - r + \frac{1}{4} \tag{56}$$

Để chuyển toàn bộ thuật toán về số học nguyên, ta đặt biến phụ:

$$p'_k = p_k - \frac{1}{4} \tag{57}$$

Khi đó, tại bước khởi tạo $k = 0$:

$$p'_0 = p_0 - \frac{1}{4} = \left(\frac{5}{4} - r\right) - \frac{1}{4} = 1 - r \tag{58}$$

Vì bán kính r là một số nguyên dương ($r \in \mathbb{Z}^+$), nên $p'_0 = 1 - r$ chắc chắn là một số nguyên thuần túy.

Hơn thế nữa, xét độ biến thiên giữa hai bước lặp liên tiếp $\Delta p = p_{k+1} - p_k$:

$$p'_{k+1} - p'_k = \left(p_{k+1} - \frac{1}{4}\right) - \left(p_k - \frac{1}{4}\right) = p_{k+1} - p_k \tag{59}$$

Độ biến thiên này tại mọi bước đều là số nguyên ($2x_{k+1} + 1$ hoặc $2x_{k+1} + 1 - 2y_{k+1}$). Do đó, p'_k luôn luôn là số nguyên với mọi bước lặp $k \geq 0$.

Bây giờ ta tiến hành chứng minh rằng việc thay thế p_k bằng p'_k không làm thay đổi bất kỳ quyết định rẽ nhánh nào:

Ta có quan hệ:

$$p_k = p'_k + \frac{1}{4} \quad \text{với } p'_k \in \mathbb{Z} \tag{60}$$

1. Chứng minh chiều điều kiện $p_k \geq 0 \iff p'_k \geq 0$:

- Giả sử $p'_k \geq 0$. Vì p'_k là số nguyên, các giá trị khả dĩ của p'_k là $\{0, 1, 2, 3, \dots\}$. Khi đó:

$$p_k = p'_k + \frac{1}{4} \geq 0 + \frac{1}{4} = \frac{1}{4} > 0 \implies p_k \geq 0 \tag{61}$$

- Ngược lại, giả sử $p'_k < 0$. Vì p'_k là số nguyên, giá trị lớn nhất mà p'_k có thể đạt được là -1. Khi đó:

$$p_k = p'_k + \frac{1}{4} \leq -1 + \frac{1}{4} = -\frac{3}{4} < 0 \implies p_k < 0 \tag{62}$$

  Theo nguyên lý phản chứng, nếu $p_k \geq 0$ thì bắt buộc p'_k không thể nhỏ hơn 0, tức là $p'_k \geq 0$.

2. Chứng minh chiều điều kiện $p_k < 0 \iff p'_k < 0$**:** Tương tự, từ chứng minh trên ta suy ra ngay lập tức:

$$p_k < 0 \iff p'_k < 0 \tag{63}$$

Kết luận: Dấu của p_k và dấu của p'_k hoàn toàn đồng nhất tại mọi bước lặp của thuật toán:

$$p_k \geq 0 \iff p'_k \geq 0 \quad \text{và} \quad p_k < 0 \iff p'_k < 0 \tag{64}$$

Do đó, ta hoàn toàn có thể thay thế khởi tạo $p_0 = \frac{5}{4} - r$ bằng $p'_0 = 1 - r$. Toàn bộ thuật toán Midpoint Circle được thực thi 100% bằng các phép toán số nguyên, khử sạch hoàn toàn sự hiện diện của số thực trên phần cứng.

5 TUẦN 5: THUẬT TOÁN VẼ ĐƯỜNG CONG ELLIPSE, ĐƯỜNG CONG BÉZIER VÀ PHÂN VÙNG ĐA GIÁC

5.1 THUẬT TOÁN VẼ CUNG TRÒN VÀ CUNG ELLIPSE TRÊN LƯỚI TỌA ĐỘ NGUYÊN

Trong đồ họa máy tính, việc tạo dựng các đường cong bậc hai (Conic Sections) như cung tròn và ellipse đòi hỏi các giải pháp rời rạc hóa tối ưu nhằm xác định chính xác các điểm ảnh nguyên trên màn hình hiển thị (Hearn & Baker, 2004; Foley et al., 1996).

1. Kỹ thuật Vẽ Cung tròn (Circular Arc Generation): Khác với đường tròn khép kín hoàn chỉnh vốn tận dụng triệt để tính đối xứng 8 phần để sinh toàn bộ chu vi từ một cung bát phân đơn lẻ, việc vẽ một cung tròn giới hạn bởi góc xuất phát θ_{start} và góc kết thúc θ_{end} (với $\theta \in [0, 2\pi]$) đòi hỏi cơ chế kiểm tra điều kiện góc hoặc tọa độ mút:

- *Phương pháp góc cực:* Thuật toán Midpoint Circle duyệt qua các điểm ứng viên, sau đó tính góc cực $\theta = \text{atan2}(y - y_c, x - x_c)$ (hoặc dùng phép so sánh tích vô hướng và tích có hướng giữa vector hiện tại với hai vector giới hạn biên mút). Điểm ảnh chỉ được xuất ra bộ đệm khung hình (Frame Buffer) nếu góc nằm trong khoảng $[\theta_{\text{start}}, \theta_{\text{end}}]$.
- *Phương pháp điểm mút:* Xác định trước tọa độ nguyên của hai điểm đầu mút $P_{\text{start}}(x_s, y_s)$ và $P_{\text{end}}(x_e, y_e)$ trên đường tròn giải tích, sau đó chạy giải thuật gia số dọc theo chiều quay xác định và dừng vòng lặp khi chạm tới điểm mút kết thúc.

2. Thuật toán Vẽ Đường cong Ellipse Điểm Giữa (Midpoint Ellipse Algorithm): Phương trình đại số của một hình ellipse chuẩn có tâm tại gốc tọa độ $O(0, 0)$ với bán trục lớn r_x dọc theo trục hoành và bán trục nhỏ r_y dọc theo trục tung có dạng:

$$\frac{x^2}{r_x^2} + \frac{y^2}{r_y^2} = 1 \tag{65}$$

Khử mẫu số, ta định nghĩa hàm ellipse giải tích:

$$f_{\text{ellipse}}(x, y) = r_y^2 x^2 + r_x^2 y^2 - r_x^2 r_y^2 \tag{66}$$

với dấu của hàm phân định vị trí tương đối: $f_{\text{ellipse}}(x, y) < 0$ khi điểm nằm trong ellipse, $f_{\text{ellipse}}(x, y) = 0$ khi điểm nằm trên biên, và $f_{\text{ellipse}}(x, y) > 0$ khi điểm nằm ngoài ellipse.

Hình ellipse đối xứng qua hai trục tọa độ $x = 0$ và $y = 0$, do đó chỉ cần khảo sát trong góc phần tư thứ nhất ($x \geq 0, y \geq 0$), sau đó áp dụng tính đối xứng 4 phần (4-Way Symmetry) để suy ra các điểm trên 3 góc phần tư còn lại:

$$(x, y), \quad (-x, y), \quad (x, -y), \quad (-x, -y) \tag{67}$$

Khác với đường tròn có độ cong đồng đều, độ dốc tiếp tuyến của ellipse thay đổi liên tục:

$$\frac{dy}{dx} = -\frac{r_y^2 x}{r_x^2 y} \tag{68}$$

Do đó, góc phần tư thứ nhất được phân tách thành hai miền rõ rệt thông qua điểm phân giới có độ lớn độ dốc bằng 1 ($|\frac{dy}{dx}| = 1 \iff 2r_y^2 x = 2r_x^2 y$):

Miền 1 (Region 1: $|\frac{dy}{dx}| < 1 \iff 2r_y^2 x < 2r_x^2 y$): Độ dốc tiếp tuyến thoải ($|m| < 1$). Hoành độ x tăng đều từng bước nguyên ($x_{k+1} = x_k + 1$), trong khi tung độ y giữ nguyên hoặc giảm 1 đơn vị. Tham số quyết định tại điểm giữa $M(x_k + 1, y_k - 1/2)$ có dạng:

$$p_{1k} = f_{\text{ellipse}}\left(x_k + 1, y_k - \frac{1}{2}\right) = r_y^2(x_k + 1)^2 + r_x^2\left(y_k - \frac{1}{2}\right)^2 - r_x^2 r_y^2 \tag{69}$$

Giá trị khởi tạo tại đỉnh trên $(0, r_y)$ là:

$$p_{10} = f_{\text{ellipse}}\left(1, r_y - \frac{1}{2}\right) = r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2 \tag{70}$$

Công thức truy hồi cập nhật cho bước kế tiếp:

- Nếu $p_{1k} < 0$: Điểm giữa nằm trong ellipse, chọn pixel trên $(x_k + 1, y_k)$, cập nhật:
$$p_{1,k+1} = p_{1k} + 2r_y^2 x_{k+1} + r_y^2 \tag{71}$$
- Nếu $p_{1k} \geq 0$: Điểm giữa nằm ngoài ellipse, chọn pixel dưới $(x_k + 1, y_k - 1)$, cập nhật:
$$p_{1,k+1} = p_{1k} + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2 \tag{72}$$

Miền 1 kết thúc ngay khi $2r_y^2 x \geq 2r_x^2 y$.

Miền 2 (Region 2: $|\frac{dy}{dx}| \geq 1 \iff 2r_y^2 x \geq 2r_x^2 y$): Độ dốc tiếp tuyến dốc đứng ($|m| \geq 1$). Tung độ y giảm đều từng bước nguyên ($y_{k+1} = y_k - 1$), trong khi hoành độ x giữ nguyên hoặc tăng 1 đơn vị. Tham số quyết định tại điểm giữa $M(x_k + 1/2, y_k - 1)$ có dạng:
$$p_{2k} = f_{\text{ellipse}}\left(x_k + \frac{1}{2}, y_k - 1\right) = r_y^2 \left(x_k + \frac{1}{2}\right)^2 + r_x^2 (y_k - 1)^2 - r_x^2 r_y^2 \tag{73}$$

Giá trị khởi tạo p_{20} được tính tại điểm cuối cùng $(x_{\text{last}}, y_{\text{last}})$ của Miền 1:
$$p_{20} = f_{\text{ellipse}}\left(x_{\text{last}} + \frac{1}{2}, y_{\text{last}} - 1\right) = r_y^2 \left(x_{\text{last}} + \frac{1}{2}\right)^2 + r_x^2 (y_{\text{last}} - 1)^2 - r_x^2 r_y^2 \tag{74}$$

Công thức truy hồi cập nhật cho bước kế tiếp:

- Nếu $p_{2k} > 0$: Điểm giữa nằm ngoài ellipse, chọn pixel bên trái $(x_k, y_k - 1)$, cập nhật:
$$p_{2,k+1} = p_{2k} - 2r_x^2 y_{k+1} + r_x^2 \tag{75}$$
- Nếu $p_{2k} \leq 0$: Điểm giữa nằm trong ellipse, chọn pixel chéo $(x_k + 1, y_k - 1)$, cập nhật:
$$p_{2,k+1} = p_{2k} + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2 \tag{76}$$

Miền 2 tiếp diễn cho đến khi tung độ chạm gốc $y = 0$.

5.2 Hạn chế của Phương trình Tường minh $y = f(x)$ trong Biểu diễn Đường cong

Phương trình tường minh (Explicit Form) $y = f(x)$ bộc lộ các khiếm khuyết hình học và giải thuật nghiêm trọng khi mô hình hóa các đường cong trong đồ họa máy tính:

1. Không thể biểu diễn các đường cong đa trị (Multi-valued Curves): Theo định nghĩa ánh xạ giải tích, một hàm số tường minh $y = f(x)$ chỉ gán duy nhất một giá trị y cho mỗi giá trị x. Tuy nhiên, hầu hết các đường cong tự nhiên và hình học thực tế (như hình tròn, ellipse, hình quả táo, đường xoắn ốc hay đường cong tự uốn lượn) đều có nhiều giá trị tung độ tương ứng với một hoành độ. Ví dụ với đường tròn tâm O bán kính r, phương trình tường minh buộc phải phân tách thành hai nhánh hàm độc lập $y = \sqrt{r^2 - x^2}$ và $y = -\sqrt{r^2 - x^2}$, gây phức tạp cho cấu trúc dữ liệu và giải thuật quản lý.

2. Điểm kỳ dị và Độ dốc vô hạn (Infinite Slopes / Vertical Tangents): Khi tiếp tuyến của đường cong tiến dần tới phương thẳng đứng ($\frac{dy}{dx} \to \pm\infty$), bước nhảy tung độ Δy bùng nổ vượt khỏi giới hạn hiển thị của lưới điểm ảnh. Nếu tiếp tục tăng đều x, các điểm ảnh vẽ ra sẽ bị ngắt quãng nghiêm trọng, tạo thành các khoảng trống lỗ hổng trên màn hình.

3. Mất tính Bất biến Hình học trước Phép quay (Lack of Rotational Invariance): Hình dạng của một thực thể vật lý không đổi khi quay trong không gian. Tuy nhiên, dạng hàm tường minh $y = f(x)$ phụ thuộc tuyệt đối vào hệ trục tọa độ tham chiếu. Một hàm số đơn trị sau khi quay một góc θ có thể biến thành hàm đa trị hoặc không thể biểu diễn giải tích dưới dạng $y = f(x)$.

Ưu thế của Dạng Tham số (Parametric Form): Để giải quyết triệt để các rào cản trên, đồ họa máy tính hiện đại chuẩn hóa việc biểu diễn đường cong dưới dạng tham số vector:
$$\mathbf{P}(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix}, \quad t \in [0, 1] \tag{77}$$

Mỗi thành phần tọa độ là một hàm đa thức đơn trị trơn tru theo biến tham số chuẩn hóa t. Dạng tham số cho phép biểu diễn tự nhiên các đường cong khép kín, đường uốn lượn tự cắt, bảo toàn tính bất biến đối với mọi phép biến đổi affine (quay, tịnh tiến, co giãn), đồng thời kiểm soát chính xác điểm đầu và điểm cuối của đoạn cong.

5.3 Cơ sở Toán học và Kỹ thuật Vẽ Đường cong Bézier trên Lưới Tọa độ Nguyên

Đường cong Bézier, được phát minh độc lập bởi Pierre Bézier và Paul de Casteljau (Bézier, 1972; de Casteljau, 1959), là chuẩn mực toán học trong mô hình hóa hình học tham số và thiết kế đồ họa vector (định dạng SVG, PostScript, OpenType).

1. Phương trình Đường cong Bézier Tổng quát: Cho $n+1$ điểm điều khiển (Control Points) $\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_n \in \mathbb{R}^2$. Đường cong Bézier bậc n được xác định bởi tổ hợp lồi của các đa thức cơ sở Bernstein (Bernstein Polynomials):

$$\mathbf{P}(u) = \sum_{k=0}^{n} \mathbf{p}_k B_{k,n}(u), \quad u \in [0,1] \tag{78}$$

trong đó đa thức Bernstein bậc n được định nghĩa bằng:

$$B_{k,n}(u) = C_n^k u^k (1-u)^{n-k} = \frac{n!}{k!(n-k)!} u^k (1-u)^{n-k} \tag{79}$$

2. Kỹ thuật Vẽ trên Lưới Tọa độ Nguyên: Do màn hình hiển thị hoạt động trên lưới điểm ảnh nguyên rời rạc, có hai phương pháp tiêu chuẩn để đưa đường cong Bézier lên màn hình:

- *Phương pháp lấy mẫu rời rạc hóa (Uniform Parameter Sampling):* Chia đoạn tham số $u \in [0,1]$ thành N bước rời rạc $\Delta u = 1/N$, tính tọa độ thực $(x(u_i), y(u_i))$ rồi áp dụng thuật toán vẽ đoạn thẳng Bresenham nối các điểm nguyên kế tiếp $([x(u_i)], [y(u_i)])$ với nhau.
- *Phương pháp chia nhỏ đệ quy De Casteljau (Recursive Subdivision):* Chia đôi đường cong tại điểm giữa $u = 0.5$ thành hai đoạn cong con. Đệ quy tiếp tục cho đến khi đa giác điều khiển của từng đoạn con đạt tiêu chuẩn độ phẳng (Flatness Tolerance, tức khoảng cách từ các điểm điều khiển bên trong đến đoạn thẳng nối hai điểm mút nhỏ hơn 0.5 pixel). Khi đó, đoạn cong con được xấp xỉ bằng một đoạn thẳng và vẽ trực tiếp bằng giải thuật Bresenham nguyên.

5.4 Kiểm soát Bậc Thấp cho Tập Điểm Điều khiển Lớn: Đường cong Ghép mảnh (Piecewise Curves)

Khi một thiết kế kỹ thuật chứa hàng trăm hoặc hàng nghìn điểm điều khiển ($N = 1000$), việc xây dựng một đường cong Bézier đơn lẻ với bậc $n = N - 1 = 999$ dẫn đến những thảm họa tính toán và thẩm mỹ:

- **Mất ổn định số học (Numerical Instability):** Bậc đa thức u^{999} dẫn đến các hệ số tổ hợp nhị thức khổng lồ C_{999}^k, vượt quá độ chính xác dấu phẩy động của phần cứng và gây tràn số nghiêm trọng.
- **Mất tính Kiểm soát Cục bộ (Loss of Local Control):** Do tính chất toàn cục của đa thức Bernstein, việc dịch chuyển một điểm điều khiển bất kỳ sẽ làm biến dạng toàn bộ hình dáng của toàn bộ đường cong từ đầu đến cuối.
- **Hiện tượng Dao động Runge (Runge Phenomenon):** Đa thức bậc quá cao dễ sinh ra các gợn sóng uốn éo không mong muốn ở các vùng biên mút.

Giải pháp: Đường cong Bézier Ghép mảnh (Piecewise Cubic Bézier Curves / Splines): Thay vì dùng một đường cong bậc cao duy nhất, hệ thống chia tập 1000 điểm điều khiển thành chuỗi các phân đoạn đường cong Bézier bậc thấp (chuẩn mực là bậc 3, Cubic Bézier, gồm 4 điểm điều khiển mỗi đoạn). Mỗi đoạn cong nhỏ độc lập đảm bảo việc tính toán đại số ma trận cực kỳ gọn nhẹ và ổn định. Đồng thời, chỉnh sửa một điểm điều khiển chỉ tác động cục bộ lên đoạn cong chứa nó mà không làm xáo trộn các phân đoạn đồ họa ở xa.

5.5 Ý nghĩa Hình học của Đặc tính Nội suy Điểm Mút $P(0) = p_0, P(1) = p_n$

Đặc tính nội suy điểm mút của đường cong Bézier:

$$\mathbf{P}(0) = \mathbf{p}_0 \quad \text{và} \quad \mathbf{P}(1) = \mathbf{p}_n \tag{80}$$

giữ vai trò nền tảng quyết định trong việc ghép nối các phân đoạn đường cong phức tạp (Composite Curves / Splines) mà không làm gãy nét:

1. Đảm bảo Tính Liên tục Vị trí (C^0 / G^0 Continuity): Đặc tính này bảo đảm đường cong luôn đi qua chính xác điểm điều khiển đầu tiên và điểm điều khiển cuối cùng. Nhờ đó, để nối hai phân đoạn cong $\mathbf{P}^{(1)}(u)$ và $\mathbf{P}^{(2)}(u)$ lại với nhau tại mối nối, ta chỉ cần ràng buộc điểm cuối của đoạn thứ nhất trùng với điểm đầu của đoạn thứ hai:

$$\mathbf{P}^{(1)}(1) = \mathbf{P}^{(2)}(0) \iff \mathbf{p}_n^{(1)} = \mathbf{p}_0^{(2)} \tag{81}$$

2. Đảm bảo Tính Liên tục Tiếp tuyến và Pháp tuyến (C^1 / G^1 Continuity): Vector tiếp tuyến tại điểm mút xuất phát và kết thúc của đường cong Bézier luôn có phương trùng với đoạn nối hai điểm điều khiển kề mút:

$$\mathbf{P}'(0) = n(\mathbf{p}_1 - \mathbf{p}_0), \quad \mathbf{P}'(1) = n(\mathbf{p}_n - \mathbf{p}_{n-1}) \tag{82}$$

Do đó, để hai phân đoạn cong nối tiếp nhau một cách trơn tru, mềm mại mà không bị gãy góc (kink), ta chỉ cần thiết lập điều kiện ba điểm điều khiển kề mối nối thẳng hàng:

$$\mathbf{p}_{n-1}^{(1)}, \quad \mathbf{p}_n^{(1)} = \mathbf{p}_0^{(2)}, \quad \mathbf{p}_1^{(2)} \quad \text{thẳng hàng} \tag{83}$$

Khi đó, vector tiếp tuyến của hai đoạn cong tại mối nối cùng phương (ngược hướng), kéo theo vector pháp tuyến (vuông góc với tiếp tuyến) cũng hoàn toàn thẳng hàng và trùng khớp. Đây chính là tiền đề cốt lõi giúp các phần mềm thiết kế đồ họa vector (như Adobe Illustrator hay Inkscape) cho phép người dùng kéo các tay đòn điều khiển (Tangent Handles) qua điểm neo để tạo ra các đường cong trơn tru hoàn hảo.

5.6 Phương trình Tham số của Đường cong Bézier Bậc Ba (Cubic Bézier Curve)

Đường cong Bézier bậc ba được định nghĩa bởi 4 điểm điều khiển $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$. Với $n = 3$, các đa thức cơ sở Bernstein là:

$$B_{0,3}(u) = (1-u)^3 \tag{84}$$

$$B_{1,3}(u) = 3u(1-u)^2 \tag{85}$$

$$B_{2,3}(u) = 3u^2(1-u) \tag{86}$$

$$B_{3,3}(u) = u^3 \tag{87}$$

Phương trình tham số của đường cong Bézier bậc ba có dạng khai triển:

$$\mathbf{P}(u) = (1-u)^3\mathbf{p}_0 + 3u(1-u)^2\mathbf{p}_1 + 3u^2(1-u)\mathbf{p}_2 + u^3\mathbf{p}_3, \quad u \in [0,1] \tag{88}$$

Dưới dạng biểu diễn ma trận thuận tiện cho các phép nhân GPU:

$$\mathbf{P}(u) = \begin{bmatrix} u^3 & u^2 & u & 1 \end{bmatrix} \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{p}_0 \\ \mathbf{p}_1 \\ \mathbf{p}_2 \\ \mathbf{p}_3 \end{bmatrix} \tag{89}$$

5.7 Kiểm tra Đạo hàm Bậc Hai $P''(0)$ và $P''(1)$: Độ cong tại Hai Điểm Mút

Để kiểm tra hành vi độ cong của đường cong Bézier bậc ba tại hai đầu mút, ta tiến hành đạo hàm hai cấp theo tham số u:

Lấy đạo hàm cấp một của $\mathbf{P}(u)$:

$$\begin{aligned} \mathbf{P}'(u) &= -3(1-u)^2\mathbf{p}_0 + [3(1-u)^2 - 6u(1-u)]\mathbf{p}_1 + [6u(1-u) - 3u^2]\mathbf{p}_2 + 3u^2\mathbf{p}_3 \\ &= 3(1-u)^2(\mathbf{p}_1 - \mathbf{p}_0) + 6u(1-u)(\mathbf{p}_2 - \mathbf{p}_1) + 3u^2(\mathbf{p}_3 - \mathbf{p}_2) \end{aligned} \tag{90}$$

Thay $u = 0$ và $u = 1$, ta thu được các vector tiếp tuyến đầu mút:

$$\mathbf{P}'(0) = 3(\mathbf{p}_1 - \mathbf{p}_0), \quad \mathbf{P}'(1) = 3(\mathbf{p}_3 - \mathbf{p}_2) \tag{91}$$

Tiếp tục lấy đạo hàm cấp hai:

$$\begin{aligned}\mathbf{P}''(u) &= -6(1-u)(\mathbf{p}_1-\mathbf{p}_0)+[6(1-u)-6u](\mathbf{p}_2-\mathbf{p}_1)+6u(\mathbf{p}_3-\mathbf{p}_2)\\ &= 6(1-u)[(\mathbf{p}_2-\mathbf{p}_1)-(\mathbf{p}_1-\mathbf{p}_0)]+6u[(\mathbf{p}_3-\mathbf{p}_2)-(\mathbf{p}_2-\mathbf{p}_1)]\\ &= 6(1-u)(\mathbf{p}_0-2\mathbf{p}_1+\mathbf{p}_2)+6u(\mathbf{p}_1-2\mathbf{p}_2+\mathbf{p}_3)\end{aligned} \tag{92}$$

Tại điểm bắt đầu $u=0$:

$$\mathbf{P}''(0) = 6(\mathbf{p}_0-2\mathbf{p}_1+\mathbf{p}_2) = 6[(\mathbf{p}_2-\mathbf{p}_1)-(\mathbf{p}_1-\mathbf{p}_0)] \tag{93}$$

Tại điểm kết thúc $u=1$:

$$\mathbf{P}''(1) = 6(\mathbf{p}_1-2\mathbf{p}_2+\mathbf{p}_3) = 6[(\mathbf{p}_3-\mathbf{p}_2)-(\mathbf{p}_2-\mathbf{p}_1)] \tag{94}$$

Ý nghĩa Hình học và Kỹ thuật: Đạo hàm cấp hai phản ánh gia tốc chuyển động và bán kính độ cong (Curvature) của đường cong. Kết quả trên chứng minh rằng độ cong tại điểm bắt đầu chỉ phụ thuộc duy nhất vào ba điểm điều khiển đầu tiên $(\mathbf{p}_0,\mathbf{p}_1,\mathbf{p}_2)$, trong khi độ cong tại điểm kết thúc chỉ phụ thuộc vào ba điểm điều khiển cuối cùng $(\mathbf{p}_1,\mathbf{p}_2,\mathbf{p}_3)$. Đây là cơ sở toán học để thiết lập điều kiện liên tục độ cong C^2 khi nối ghép hai phân đoạn Bézier, đòi hỏi vector sai phân cấp hai tại mối nối phải bằng nhau.

5.8 Phương trình Đoạn thẳng theo Tham số Đi qua p_0, p_1

Phương trình tham số của đoạn thẳng nối hai điểm mút $\mathbf{p}_0$ và $\mathbf{p}_1$ được định nghĩa bằng phép nội suy tuyến tính (Linear Interpolation, Lerp):

$$\mathbf{P}(t) = (1-t)\mathbf{p}_0 + t\mathbf{p}_1 = \mathbf{p}_0 + t(\mathbf{p}_1-\mathbf{p}_0), \quad t\in[0,1] \tag{95}$$

Nhận xét rằng biểu thức này chính là dạng suy biến của đường cong Bézier bậc nhất ($n=1$) với hai đa thức Bernstein cơ sở $B_{0,1}(t)=1-t$ và $B_{1,1}(t)=t$. Khi $t=0$, điểm trùng với $\mathbf{p}_0$; khi $t=1$, điểm trùng với $\mathbf{p}_1$; và khi t quét tuyến tính từ 0 đến 1, điểm $\mathbf{P}(t)$ vạch nên đoạn thẳng nối hai mút.

5.9 Chứng minh Thuật toán De Casteljau cho Đường cong Bézier Bậc 2 và Bậc 3

Thuật toán De Casteljau là phương pháp hình học giải tích thực hiện phép nội suy tuyến tính lặp (Recursive Linear Interpolation) để tính tọa độ điểm trên đường cong Bézier:

1. Chứng minh với 3 Điểm Điều khiển $\mathbf{p}_0,\mathbf{p}_1,\mathbf{p}_2$ (Bézier Bậc Hai):

- Bước chia thứ nhất ($r=1$): Nội suy trên các đoạn nối ban đầu với tỉ lệ t:

$$\mathbf{p}_0^1(t) = (1-t)\mathbf{p}_0 + t\mathbf{p}_1 \tag{96}$$

$$\mathbf{p}_1^1(t) = (1-t)\mathbf{p}_1 + t\mathbf{p}_2 \tag{97}$$

- Bước chia thứ hai ($r=2$): Nội suy giữa hai điểm trung gian $\mathbf{p}_0^1(t)$ và $\mathbf{p}_1^1(t)$:

$$\mathbf{p}_0^2(t) = (1-t)\mathbf{p}_0^1(t) + t\mathbf{p}_1^1(t) \tag{98}$$

Thay thế các biểu thức ở bước một vào:

$$\begin{aligned}\mathbf{p}_0^2(t) &= (1-t)[(1-t)\mathbf{p}_0+t\mathbf{p}_1]+t[(1-t)\mathbf{p}_1+t\mathbf{p}_2]\\ &= (1-t)^2\mathbf{p}_0+t(1-t)\mathbf{p}_1+t(1-t)\mathbf{p}_1+t^2\mathbf{p}_2\\ &= (1-t)^2\mathbf{p}_0+2t(1-t)\mathbf{p}_1+t^2\mathbf{p}_2\end{aligned} \tag{99}$$

Biểu thức thu được hoàn toàn trùng khớp với định nghĩa đường cong Bézier bậc hai:

$$\mathbf{P}(t) = \sum_{k=0}^{2} C_2^k t^k (1-t)^{2-k}\mathbf{p}_k \tag{100}$$

2. Chứng minh với 4 Điểm Điều khiển $\mathbf{p}_0,\mathbf{p}_1,\mathbf{p}_2,\mathbf{p}_3$ (Bézier Bậc Ba):

- Bước chia thứ nhất ($r = 1$):

$$\mathbf{p}_0^1(t) = (1-t)\mathbf{p}_0 + t\mathbf{p}_1 \tag{101}$$
$$\mathbf{p}_1^1(t) = (1-t)\mathbf{p}_1 + t\mathbf{p}_2 \tag{102}$$
$$\mathbf{p}_2^1(t) = (1-t)\mathbf{p}_2 + t\mathbf{p}_3 \tag{103}$$

- Bước chia thứ hai ($r = 2$):

$$\mathbf{p}_0^2(t) = (1-t)\mathbf{p}_0^1(t) + t\mathbf{p}_1^1(t) = (1-t)^2\mathbf{p}_0 + 2t(1-t)\mathbf{p}_1 + t^2\mathbf{p}_2 \tag{104}$$
$$\mathbf{p}_1^2(t) = (1-t)\mathbf{p}_1^1(t) + t\mathbf{p}_2^1(t) = (1-t)^2\mathbf{p}_1 + 2t(1-t)\mathbf{p}_2 + t^2\mathbf{p}_3 \tag{105}$$

- Bước chia thứ ba ($r = 3$):

$$\mathbf{p}_0^3(t) = (1-t)\mathbf{p}_0^2(t) + t\mathbf{p}_1^2(t) \tag{106}$$

$$\begin{aligned}\mathbf{p}_0^3(t) &= (1-t)\left[(1-t)^2\mathbf{p}_0 + 2t(1-t)\mathbf{p}_1 + t^2\mathbf{p}_2\right] \\ &\quad + t\left[(1-t)^2\mathbf{p}_1 + 2t(1-t)\mathbf{p}_2 + t^2\mathbf{p}_3\right] \\ &= (1-t)^3\mathbf{p}_0 + 2t(1-t)^2\mathbf{p}_1 + t^2(1-t)\mathbf{p}_2 \\ &\quad + t(1-t)^2\mathbf{p}_1 + 2t^2(1-t)\mathbf{p}_2 + t^3\mathbf{p}_3 \\ &= (1-t)^3\mathbf{p}_0 + 3t(1-t)^2\mathbf{p}_1 + 3t^2(1-t)\mathbf{p}_2 + t^3\mathbf{p}_3 \end{aligned} \tag{107}$$

Biểu thức thu được hoàn toàn trùng khớp với định nghĩa đường cong Bézier bậc ba:

$$\mathbf{P}(t) = \sum_{k=0}^{3} C_3^k t^k (1-t)^{3-k}\mathbf{p}_k \tag{108}$$

Chứng minh hoàn tất. Thuật toán De Casteljau không chỉ cung cấp cơ sở hình học vững chắc cho đường cong Bézier mà còn là giải thuật có độ ổn định số học cao nhất để tính toán tọa độ điểm trên thực tế.

5.10 So sánh Bản chất giữa Spline Hermite và Đường cong Bézier Bậc Ba

Cả đường cong Hermite bậc ba (Cubic Hermite Spline) và đường cong Bézier bậc ba đều sử dụng đa thức bậc 3 theo tham số $u \in [0, 1]$, nhưng khác biệt căn bản về triết lý giao tiếp dữ liệu:

1. Mô hình Toán học Hermite: Đường cong Hermite được xác định bởi hai điểm mút $(\mathbf{P}_0, \mathbf{P}_1)$ và hai vector tiếp tuyến (đạo hàm bậc nhất) tại hai đầu mút $(\mathbf{D}_0, \mathbf{D}_1)$:

$$\mathbf{P}_H(u) = (2u^3 - 3u^2 + 1)\mathbf{P}_0 + (-2u^3 + 3u^2)\mathbf{P}_1 + (u^3 - 2u^2 + u)\mathbf{D}_0 + (u^3 - u^2)\mathbf{D}_1 \tag{109}$$

2. Tương quan Đại số và Điểm khác biệt Cốt lõi: Hai dạng đường cong này hoàn toàn tương đương về mặt toán học thông qua phép ánh xạ:

$$\mathbf{P}_0 = \mathbf{p}_0, \quad \mathbf{P}_1 = \mathbf{p}_3, \quad \mathbf{D}_0 = 3(\mathbf{p}_1 - \mathbf{p}_0), \quad \mathbf{D}_1 = 3(\mathbf{p}_3 - \mathbf{p}_2) \tag{110}$$

Tức là hai điểm điều khiển giữa của Bézier được suy ra trực tiếp từ vector tiếp tuyến của Hermite: $\mathbf{p}_1 = \mathbf{p}_0 + \frac{1}{3}\mathbf{D}_0$ và $\mathbf{p}_2 = \mathbf{p}_3 - \frac{1}{3}\mathbf{D}_1$.

Ưu thế Vượt trội của Đường cong Bézier trong Đồ họa:

- *Tính trực quan đối với người thiết kế:* Dạng Hermite đòi hỏi người dùng phải chỉ định các vector đạo hàm vô hình $(\mathbf{D}_0, \mathbf{D}_1)$ với cả hướng và độ lớn đại số trừu tượng. Ngược lại, Bézier thay thế các vector toán học bằng các điểm điều khiển trực quan $(\mathbf{p}_1, \mathbf{p}_2)$. Người thiết kế chỉ cần tương tác kéo thả điểm trên màn hình để uốn nắn đường cong.
- *Tính chất Bao lồi (Convex Hull Property):* Toàn bộ đường cong Bézier luôn nằm hoàn toàn bên trong đa giác bao lồi tạo bởi 4 điểm điều khiển. Tính chất hình học cực kỳ quan trọng này không tồn tại ở dạng Hermite thông thường, hỗ trợ đắc lực cho các thuật toán kiểm tra va chạm, cắt xén (Clipping) và tối ưu hóa vùng hiển thị.

5.11 Phương pháp Xác định Điểm Nằm Trong và Ngoài Hình Đa giác

Để xác định một điểm kiểm tra $P(x, y)$ nằm bên trong (Interior) hay bên ngoài (Exterior) một đa giác phẳng hai chiều, đồ họa máy tính ứng dụng hai quy tắc chuẩn mực:

1. Quy tắc Đếm Giao điểm Lẻ/Chẵn (Odd-Even Rule / Parity Test): Từ điểm kiểm tra P, phóng một tia bán vô hạn (Ray) theo một hướng bất kỳ (thông thường là tia nằm ngang hướng về phía $+x$) ra vô cực. Đếm tổng số lần tia cắt qua các cạnh của đa giác:

- Nếu số lượng giao điểm là số lẻ: Điểm P nằm bên trong đa giác.
- Nếu số lượng giao điểm là số chẵn: Điểm P nằm bên ngoài đa giác.

2. Quy tắc Số Vòng quay (Non-Zero Winding Number Rule): Đếm số vòng mà chu vi đa giác quay quanh điểm P. Phóng một tia từ P ra vô cực, mỗi khi có một cạnh của đa giác cắt ngang qua tia từ phải sang trái (theo chiều kim đồng hồ) thì cộng thêm 1; nếu cắt từ trái sang phải thì trừ đi 1. Điểm P nằm trong đa giác nếu tổng số vòng quay khác 0, và nằm ngoài nếu số vòng quay bằng 0.

5.12 Chứng minh Cơ sở Toán học Dựa trên Định lý Đường cong Jordan

Định lý Đường cong Jordan (Jordan Curve Theorem) là định lý topo kinh điển chứng minh tính đúng đắn của việc phân loại điểm:

Phát biểu Định lý Jordan: Mọi đường cong phẳng đơn, liên tục và khép kín C chia mặt phẳng $\mathbb{R}^2$ thành đúng hai miền liên thông rời rạc: một miền trong bị chặn (Interior, S_{int}) và một miền ngoài không bị chặn (Exterior, S_{ext}), với đường cong C là biên giới chung duy nhất của cả hai miền. Bất kỳ đường đi liên tục nào nối một điểm thuộc S_{int} với một điểm thuộc S_{ext} bắt buộc phải cắt qua đường cong C ít nhất một lần.

Chứng minh Xác định Vị trí Điểm $P(x, y)$: Xét một điểm mốc P_∞ nằm ở vô cực, chắc chắn thuộc miền ngoài ($P_\infty \in S_{\text{ext}}$). Nối một đoạn thẳng hoặc tia L từ điểm kiểm tra P đến P_∞.

- Chu vi đa giác là một đường cong Jordan khép kín phân tách mặt phẳng thành S_{int} và S_{ext}.
- Xét một điểm chuyển động liên tục dọc theo tia L từ P ra P_∞. Mỗi khi tia cắt xuyên qua một cạnh của đa giác, điểm chuyển động bắt buộc phải thay đổi trạng thái miền: từ trong ra ngoài ($S_{\text{int}} \to S_{\text{ext}}$) hoặc từ ngoài vào trong ($S_{\text{ext}} \to S_{\text{int}}$).
- Gọi k là tổng số giao điểm mà tia L cắt qua biên đa giác. Điểm kết thúc P_∞ đã biết chắc chắn nằm ở miền ngoài (S_{ext}). Do đó:
  - Nếu k là số lẻ: Để kết thúc tại miền ngoài, trạng thái ban đầu của P bắt buộc phải khác với S_{ext}, tức là $P \in S_{\text{int}}$ (nằm bên trong).
  - Nếu k là số chẵn: Số lần đi vào và đi ra triệt tiêu lẫn nhau, do đó trạng thái ban đầu của P phải cùng trạng thái với P_∞, tức là $P \in S_{\text{ext}}$ (nằm bên ngoài).

Chứng minh hoàn tất.

5.13 Nguồn gốc và Bản chất Topo của Quy tắc Chẵn Lẻ (Odd-Even Rule)

Quy tắc Chẵn Lẻ (Odd-Even Rule) thực chất là sự số hóa trực tiếp định lý đường cong Jordan trên hệ tọa độ rời rạc của máy tính:

Bản chất của quy tắc xuất phát từ tính chất đối xứng nhị phân của việc phân hoạch không gian. Một đường biên đóng đóng vai trò như một bức tường ngăn cách. Mỗi lần một thực thể vượt qua bức tường, trạng thái nhị phân của nó bị đảo ngược:

$$\text{Trạng thái}(P) = \left(\sum_{i=1}^{m} \text{Giao điểm}(L, E_i) \right) \bmod 2 \tag{111}$$

Giá trị bằng 1 tương ứng với miền trong (Interior) và bằng 0 tương ứng với miền ngoài (Exterior).

Sở dĩ quy tắc này luôn đúng và liên quan chặt chẽ đến định lý Jordan là vì định lý Jordan khẳng định tính bất biến topo: không tồn tại bất kỳ con đường nào đi từ miền trong ra miền ngoài mà không

vượt qua biên. Do đó, tính chẵn lẻ của số giao điểm không phụ thuộc vào hướng của tia kiểm tra (dù phóng tia sang phải, sang trái hay theo góc chéo bất kỳ, tính chẵn lẻ của số lần xuyên qua biên luôn luôn là một đại lượng bất biến).

5.14 Kỹ thuật Rút ngắn Cạnh tại các Đỉnh Không Cực trị trong Thuật toán Tô màu theo Dòng quét

Thuật toán tô màu đa giác theo dòng quét (Scan-line Polygon Fill Algorithm) hoạt động bằng cách cho dòng quét $y = y_{\text{scan}}$ quét tuần tự từ đáy lên đỉnh đa giác. Tại mỗi dòng quét, thuật toán tìm các giao điểm với các cạnh trong Bảng cạnh kích hoạt (Active Edge Table, AET), sắp xếp giao điểm theo hoành độ x tăng dần, rồi tô màu các đoạn pixel nằm kẹp giữa từng cặp giao điểm lẻ và chẵn $(x_1, x_2), (x_3, x_4), \ldots$ theo quy tắc chẵn lẻ.

Vấn đề Xảy ra khi Dòng quét Đi qua Đỉnh Đa giác: Một đỉnh là giao điểm chung của hai cạnh kề nhau. Nếu dòng quét đi qua đỉnh, phép toán tìm giao điểm thông thường sẽ tìm thấy đỉnh đó hai lần (một lần trên mỗi cạnh), sinh ra hai giao điểm có tọa độ trùng nhau:

$$x^{(1)}_{\text{intersect}} = x^{(2)}_{\text{intersect}} = x_{\text{vertex}} \tag{112}$$

Sự xuất hiện của hai giao điểm trùng nhau này có thể phá vỡ quy tắc Parity của toàn bộ các pixel phía sau trên dòng quét. Để xử lý chính xác, thuật toán phân loại đỉnh thành hai trường hợp bản chất:

1. Đỉnh Cực trị Địa phương (Local Extremum gồm cực tiểu địa phương Local Minimum hoặc cực đại địa phương Local Maximum): Hai cạnh kề nhau của đỉnh cùng nằm về một phía của dòng quét (cùng nằm phía trên hoặc cùng nằm phía dưới, tức tung độ của hai đỉnh lân cận cùng lớn hơn hoặc cùng nhỏ hơn tung độ của đỉnh đang xét).

- Khi dòng quét đi qua đỉnh cực trị, nó chỉ tiếp xúc với mép biên của đa giác mà không đi xuyên qua từ trong ra ngoài (hoặc ngược lại).
- Để bảo toàn quy tắc chẵn lẻ, đỉnh cực trị phải được tính là **hai giao điểm** (hoặc bỏ qua không tính). Khi được tính hai lần, tính chẵn lẻ không bị đảo lộn, các đoạn tô màu phía sau vẫn vận hành chính xác. Do đó, **giữ nguyên độ dài hai cạnh, không rút ngắn**.

2. Đỉnh Không Cực trị (Non-extremum Vertex / Monotonic Vertex): Hai cạnh kề nhau của đỉnh nằm về hai phía đối lập của dòng quét: một cạnh đi lên phía trên và một cạnh đi xuống phía dưới.

- Khi dòng quét đi qua đỉnh này, nó thực sự đâm xuyên từ miền ngoài vào miền trong đa giác (hoặc từ miền trong ra miền ngoài).
- Nếu tính cả hai cạnh, ta có 2 giao điểm (số chẵn), khiến thuật toán lầm tưởng dòng quét không đi xuyên qua biên, dẫn đến việc đảo ngược toàn bộ trạng thái tô màu của các pixel phía sau trên dòng quét đó (gây hiện tượng tràn màu ra ngoài hoặc mất màu bên trong).
- Vì vậy, đỉnh không cực trị bắt buộc chỉ được tính là **đúng một giao điểm duy nhất**.

Giải pháp Kỹ thuật: Rút ngắn Cạnh (Edge Shortening): Để đảm bảo đỉnh không cực trị chỉ tạo ra đúng 1 giao điểm khi dòng quét đi qua, thuật toán chuẩn hóa quy ước rút ngắn một trong hai cạnh kề đi đúng 1 đơn vị tung độ:

$$y_{\max} = y_{\max} - 1 \quad \text{(đối với cạnh nằm phía dưới đỉnh)} \tag{113}$$

hoặc:

$$y_{\min} = y_{\min} + 1 \quad \text{(đối với cạnh nằm phía trên đỉnh)} \tag{114}$$

Nhờ việc rút ngắn cạnh này, tại dòng quét đi qua đỉnh, chỉ có duy nhất một cạnh chứa đỉnh đó còn nằm trong Bảng cạnh kích hoạt AET, đảm bảo số giao điểm luôn được ghi nhận là một số lẻ khi xuyên qua biên, giúp thuật toán Scan-line Fill tô màu chính xác 100% không gian đa giác mà không gặp lỗi tràn biên.

6 TUẦN 6: KỸ THUẬT TÔ MIỀN NÂNG CAO VÀ PHÉP BIẾN ĐỔI HÌNH HỌC TRONG KHÔNG GIAN 2D VÀ 3D

6.1 THUẬT TOÁN DÒNG QUÉT VỚI GÓC NGHIÊNG α BẤT KỲ VÀ BƯỚC NHẢY A PIXEL

Trong thuật toán tô màu đa giác theo dòng quét chuẩn tắc, các dòng quét luôn được giả định nằm ngang, song song với trục hoành Ox ($\alpha = 0$), và khoảng cách giữa hai dòng quét liên tiếp được cố định bằng đúng 1 pixel ($\Delta y = 1$). Tuy nhiên, trong nhiều bài toán đồ họa kỹ thuật thực tế, tiêu biểu như kỹ thuật đánh bóng dạng sọc (Cross-hatching), quét ảnh raster chéo trong máy in laser, hoặc quét hiển thị theo hướng quét tùy biến, hệ thống cần triển khai các dòng quét nghiêng một góc $\alpha \in [0, \pi)$ so với trục hoành, với khoảng cách trực giao giữa hai dòng quét kề nhau là A pixel ($A \geq 1$).

Xét về mặt toán học, bài toán tổng quát này có thể được giải quyết triệt để thông qua hai hướng tiếp cận chính:

1. Hướng tiếp cận Biến đổi Tọa độ Affine: Thay vì sửa đổi trực tiếp cấu trúc của thuật toán dòng quét kinh điển, ta thực hiện một phép đổi hệ quy chiếu. Giả sử đa giác ban đầu S được định nghĩa bởi tập đỉnh $\{P_i(x_i, y_i)\}_{i=1}^{n}$ trong hệ tọa độ hiển thị chuẩn. Ta áp dụng phép quay không gian phẳng quanh gốc tọa độ một góc $-\alpha$:

$$\begin{bmatrix} x_i' \\ y_i' \\ 1 \end{bmatrix} = \mathbf{R}(-\alpha) \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\alpha & \sin\alpha & 0 \\ -\sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} \tag{115}$$

Sau phép biến đổi này, họ các đường quét nghiêng ban đầu trở thành họ các đường quét nằm ngang hoàn toàn song song với trục Ox' trong hệ tọa độ mới (x', y'). Khoảng cách trực giao giữa hai dòng quét liên tiếp lúc này chính là bước nhảy tung độ $\Delta y' = A$.

Thuật toán khi đó được tiến hành theo các bước tuần tự:

a) Xác định giới hạn tung độ cực tiểu và cực đại của đa giác đã quay: $y'_{\min} = \min_i y_i'$ và $y'_{\max} = \max_i y_i'$.

b) Khởi tạo các dòng quét tại các mức tung độ rời rạc $y'_k = y'_{\min} + k \cdot A$ với $k \in \mathbb{N}$ thỏa mãn $y'_k \leq y'_{\max}$.

c) Sử dụng cấu trúc Bảng cạnh kích hoạt (Active Edge Table) để tìm tập các giao điểm hoành độ $x'_{k,1} < x'_{k,2} < \cdots < x'_{k,2m}$ trên từng dòng quét y'_k.

d) Áp dụng quy tắc Chẵn Lẻ (Odd-Even Rule) để xác định các đoạn thẳng bên trong đa giác nối từ $(x'_{k,2j-1}, y'_k)$ đến $(x'_{k,2j}, y'_k)$ với $j = 1, \ldots, m$.

e) Áp dụng phép quay ngược $\mathbf{R}(\alpha)$ để chuyển tọa độ các điểm mút của từng đoạn quét trở lại hệ tọa độ ban đầu:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \mathbf{R}(\alpha) \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \tag{116}$$

Sau đó, sử dụng giải thuật vẽ đoạn thẳng nguyên Bresenham để raster hóa trực tiếp từng đoạn quét lên bộ đệm khung hình.

2. Hướng tiếp cận Giải tích Trực tiếp: Trong trường hợp không muốn biến đổi toàn bộ tọa độ đỉnh, ta mô hình hóa họ dòng quét song song dưới dạng phương trình đường thẳng tổng quát:

$$-x\sin\alpha + y\cos\alpha = C_k \tag{117}$$

trong đó vector pháp tuyến đơn vị của dòng quét là $\mathbf{n} = (-\sin\alpha, \cos\alpha)^T$, và khoảng cách giữa hai đường liên tiếp là $\Delta C = C_{k+1} - C_k = A$. Với mỗi cạnh P_iP_{i+1} của đa giác có phương trình tham số $P(t) = P_i + t(P_{i+1} - P_i)$ với $t \in [0, 1]$, giao điểm với dòng quét thứ k được xác định khi tham số t thỏa mãn:

$$t = \frac{C_k - (-x_i\sin\alpha + y_i\cos\alpha)}{-(x_{i+1} - x_i)\sin\alpha + (y_{i+1} - y_i)\cos\alpha} \tag{118}$$

Nếu $t \in [0, 1]$, giao điểm tồn tại và được đưa vào danh sách giao điểm của dòng quét C_k. Các giao điểm sau đó được sắp xếp theo tọa độ chiếu dọc theo vector chỉ phương $\mathbf{u} = (\cos\alpha, \sin\alpha)^T$ và tiến hành tô màu từng cặp điểm tương ứng.

6.2 Thuật toán Tô màu Dòng quét cho Biên Dạng Đường tròn và Ellipse

Việc tô màu miền trong của các đường cong bậc hai như đường tròn và ellipse sở hữu những đặc thù hình học vượt trội so với đa giác tổng quát. Nhờ tính chất lồi (Convexity) và tính đối xứng trục hoàn hảo, mỗi dòng quét ngang $y = y_{\text{scan}}$ khi cắt qua hình tròn hoặc hình ellipse sẽ luôn tạo ra tối đa hai giao điểm thực duy nhất, tương ứng với biên trái và biên phải của miền cần tô màu.

1. Tô Dòng quét cho Hình tròn: Xét đường tròn tâm tại $I(x_c, y_c)$ và bán kính R. Dòng quét ngang tại tung độ nguyên y cắt đường tròn khi và chỉ khi $|y - y_c| \leq R$. Bằng phương pháp giải tích giải trực tiếp phương trình $(x - x_c)^2 + (y - y_c)^2 = R^2$, hoành độ của hai điểm biên trái và phải tại dòng quét y được xác định:

$$x_{\text{left}}(y) = x_c - \sqrt{R^2 - (y - y_c)^2}, \quad x_{\text{right}}(y) = x_c + \sqrt{R^2 - (y - y_c)^2} \tag{119}$$

Quá trình tô màu cho dòng quét y quy về việc kích hoạt toàn bộ các pixel nguyên (x, y) thỏa mãn:

$$\lceil x_{\text{left}}(y) \rceil \leq x \leq \lfloor x_{\text{right}}(y) \rfloor \tag{120}$$

Khác với việc tính căn bậc hai gây tốn kém thời gian CPU tại mỗi dòng, giải thuật tối ưu trong đồ họa tích hợp trực tiếp quá trình tô dòng quét vào thuật toán vẽ đường tròn Điểm Giữa (Midpoint Circle Algorithm). Khi biến nguyên x tăng dần từ 0 lên đến y, tại mỗi bước duyệt pixel biên (x_k, y_k), tính đối xứng 8 phần cho phép ta đồng thời xuất ra 4 đoạn quét ngang hoàn chỉnh:

$$\text{Dòng } y = y_c + y_k : \quad x \in [x_c - x_k, x_c + x_k] \tag{121}$$
$$\text{Dòng } y = y_c - y_k : \quad x \in [x_c - x_k, x_c + x_k] \tag{122}$$
$$\text{Dòng } y = y_c + x_k : \quad x \in [x_c - y_k, x_c + y_k] \tag{123}$$
$$\text{Dòng } y = y_c - x_k : \quad x \in [x_c - y_k, x_c + y_k] \tag{124}$$

Nhờ đó, toàn bộ hình tròn được tô kín đặc bằng các phép toán cộng trừ số nguyên thuần túy với độ phức tạp thời gian đạt mức tối ưu $O(R)$.

2. Tô Dòng quét cho Hình Ellipse: Đối với hình ellipse chuẩn có tâm tại $I(x_c, y_c)$, bán trục ngang r_x và bán trục đứng r_y, phương trình biên có dạng:

$$\frac{(x - x_c)^2}{r_x^2} + \frac{(y - y_c)^2}{r_y^2} = 1 \tag{125}$$

Tại mỗi dòng quét ngang $y \in [y_c - r_y, y_c + r_y]$, bề rộng nửa khoảng quét được xác định chính xác theo công thức:

$$\Delta x(y) = r_x \sqrt{1 - \frac{(y - y_c)^2}{r_y^2}} \tag{126}$$

và đoạn pixel cần tô là $[x_c - \lfloor \Delta x(y) \rfloor, x_c + \lfloor \Delta x(y) \rfloor]$. Tương tự như đường tròn, thuật toán Midpoint Ellipse phân tách góc phần tư thứ nhất thành hai miền dựa trên độ dốc tiếp tuyến. Trong quá trình sinh các điểm biên (x_k, y_k), thuật toán áp dụng tính đối xứng 4 phần để đồng thời tô hai dải pixel đối xứng qua trục hoành tâm ellipse: dòng $y = y_c + y_k$ và dòng $y = y_c - y_k$, nối từ $x_c - x_k$ đến $x_c + x_k$. Cơ chế này loại bỏ hoàn toàn sự cần thiết của danh sách cạnh tích cực (AEL) và triệt tiêu mọi chi phí sắp xếp tọa độ giao điểm.

6.3 Bản chất và Vai trò Nền tảng của các Phép Biến đổi Hình học trong Đồ họa Máy tính

Phép biến đổi hình học (Geometric Transformation) là nền tảng toán học cốt lõi chi phối toàn bộ quy trình xây dựng, thao tác và kết xuất đồ họa máy tính hiện đại (Hearn & Baker, 2004; Foley et al., 1996). Về bản chất giải tích, phép biến đổi hình học là một ánh xạ $T : \mathbb{R}^n \to \mathbb{R}^n$ làm thay đổi vị trí, hướng, kích thước hoặc hình dạng của các đối tượng hình học trong không gian.

Vai trò quyết định của các phép biến đổi hình học được thể hiện rõ nét qua năm khía cạnh nền tảng sau:

- *Mô hình hóa Đối tượng Phức hợp (Hierarchical Modeling):* Trong thực tế, các thực thể 3D phức tạp (như cơ thể người, robot công nghiệp hay xe hơi) được cấu thành từ hàng trăm bộ phận cơ sở đơn giản. Mỗi bộ phận được thiết kế độc lập trong hệ tọa độ cục bộ (Local Object Coordinates). Nhờ các phép tịnh tiến, quay và co giãn, các bộ phận này được lắp ráp chính xác vào hệ tọa độ thế giới (World Coordinate System) theo quan hệ phân cấp cây động học (Kinematic Tree).
- *Điều hướng Camera và Tạo Khung nhìn (Viewing Pipeline):* Để hiển thị khung cảnh 3D lên màn hình 2D, đồ họa máy tính phải mô phỏng lại vị trí mắt nhìn và hướng quan sát. Phép biến đổi góc nhìn (Viewing Transformation) dịch chuyển toàn bộ thế giới sao cho camera đặt tại gốc tọa độ và trục ngắm trùng với trục tọa độ chuẩn, tạo tiền đề cho các phép chiếu phối cảnh (Perspective Projection) và chiếu trực giao (Orthographic Projection).
- *Diễn hoạt Chuyển động (Animation & Kinematics):* Mọi chuyển động vật lý, từ chuyển động quay của bánh xe đến biến dạng mềm mại của cơ mặt nhân vật, đều được tính toán bằng cách biến thiên liên tục các tham số biến đổi hình học theo thời gian t.
- *Tương tác Người - Máy (Interactive Manipulation):* Các thao tác quen thuộc như phóng to (Zoom), thu nhỏ, xoay vật thể (Orbit), kéo trượt (Pan) trên các phần mềm đồ họa tương tác CAD/CAM hay engine đồ họa game thực chất là việc cập nhật ma trận biến đổi hình học tương ứng với dữ liệu đầu vào từ chuột và bàn phím.
- *Tối ưu hóa Tính toán thông qua Tọa độ Đồng nhất (Homogeneous Coordinates):* Bằng cách nhúng không gian Affine n chiều vào không gian xạ ảnh $n+1$ chiều, mọi phép biến đổi hình học phi tuyến như tịnh tiến đều được tuyến tính hóa thành phép nhân ma trận. Điều này cho phép ghép nối (Concatenate) hàng loạt phép biến đổi liên hoàn thành một ma trận tổng hợp duy nhất kích thước 4×4, tối ưu hóa triệt để phần cứng tính toán song song trên GPU.

6.4 Thiết lập Phép Quay và Co giãn 2D quanh Tâm Bất kỳ $C(x_c, y_c)$

Trong không gian hai chiều, các phép biến đổi quay chuẩn tắc $R(\theta)$ và co giãn chuẩn tắc $S(s_x, s_y)$ luôn lấy gốc tọa độ $O(0,0)$ làm tâm điểm tham chiếu. Khi cần áp dụng phép quay hoặc co giãn đối tượng quanh một điểm tâm tùy ý $C(x_c, y_c) \neq O$, việc áp dụng trực tiếp ma trận chuẩn sẽ làm sai lệch hoàn toàn vị trí tương quan của vật thể trong không gian.

Để giải quyết vấn đề này, nguyên lý tổng hợp phép biến đổi (Composite Transformation) được áp dụng thông qua một chuỗi ba bước liên hoàn:

1. Tịnh tiến toàn bộ không gian sao cho điểm tâm $C(x_c, y_c)$ dịch chuyển về trùng khít với gốc tọa độ $O(0,0)$, sử dụng ma trận tịnh tiến $\mathbf{T}(-x_c, -y_c)$.
2. Thực hiện phép biến đổi cơ sở (quay một góc θ hoặc co giãn với tỉ lệ s_x, s_y) lấy gốc tọa độ O làm tâm.
3. Tịnh tiến ngược toàn bộ không gian để đưa điểm tâm trở lại vị trí ban đầu $C(x_c, y_c)$, sử dụng ma trận tịnh tiến ngược $\mathbf{T}(x_c, y_c)$.

1. Phép Quay quanh Tâm $C(x_c, y_c)$: Ma trận biến đổi tổng hợp $\mathbf{M}_{\text{rot}}$ trong hệ tọa độ đồng nhất được biểu diễn bằng tích ma trận:

$$\begin{aligned}\mathbf{M}_{\text{rot}} &= \mathbf{T}(x_c, y_c) \cdot \mathbf{R}(\theta) \cdot \mathbf{T}(-x_c, -y_c) \\ &= \begin{bmatrix} 1 & 0 & x_c \\ 0 & 1 & y_c \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_c \\ 0 & 1 & -y_c \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} \cos\theta & -\sin\theta & x_c(1-\cos\theta) + y_c\sin\theta \\ \sin\theta & \cos\theta & y_c(1-\cos\theta) - x_c\sin\theta \\ 0 & 0 & 1 \end{bmatrix} \end{aligned} \tag{127}$$

Khi nhân ma trận này với vector tọa độ điểm $P(x, y, 1)^T$, ta thu được tọa độ điểm sau khi quay $P'(x', y', 1)^T$:

$$\begin{cases} x' = x_c + (x - x_c)\cos\theta - (y - y_c)\sin\theta \\ y' = y_c + (x - x_c)\sin\theta + (y - y_c)\cos\theta \end{cases} \tag{128}$$

2. Phép Co giãn quanh Tâm $C(x_c, y_c)$: Hoàn toàn tương tự, ma trận biến đổi co giãn tổng hợp $\mathbf{M}_{\text{scale}}$ có dạng:

$$\begin{aligned}\mathbf{M}_{\text{scale}} &= \mathbf{T}(x_c, y_c) \cdot \mathbf{S}(s_x, s_y) \cdot \mathbf{T}(-x_c, -y_c) \\ &= \begin{bmatrix}1 & 0 & x_c \\ 0 & 1 & y_c \\ 0 & 0 & 1\end{bmatrix} \begin{bmatrix}s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1\end{bmatrix} \begin{bmatrix}1 & 0 & -x_c \\ 0 & 1 & -y_c \\ 0 & 0 & 1\end{bmatrix} \\ &= \begin{bmatrix}s_x & 0 & x_c(1-s_x) \\ 0 & s_y & y_c(1-s_y) \\ 0 & 0 & 1\end{bmatrix}\end{aligned} \tag{129}$$

Hệ phương trình tọa độ tường minh tương ứng là:

$$\begin{cases}x' = x_c + (x - x_c)s_x \\ y' = y_c + (y - y_c)s_y\end{cases} \tag{130}$$

Công thức trên chứng minh rằng khi $s_x = s_y = 1$, điểm P giữ nguyên vị trí, và khi P trùng với tâm C thì tọa độ hoàn toàn bất biến trước phép co giãn.

6.5 Xác định Ma trận Biến đổi Đối xứng qua một Trục Tuyến tính Bất kỳ trong Mặt phẳng 2D

Phép đối xứng qua trục (Reflection) là một phép biến đổi đẳng cự (Isometry) làm đảo ngược tính định hướng (Handedness) của không gian. Phép đối xứng cơ sở qua trục hoành Ox được biểu diễn bởi ma trận:

$$\mathbf{R}_x = \begin{bmatrix}1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1\end{bmatrix} \tag{131}$$

Xét bài toán tổng quát: Tìm ma trận đối xứng qua một trục thẳng bất kỳ L trong mặt phẳng 2D. Đường thẳng L được cho bởi phương trình tường minh $y = mx + b$ hoặc đi qua một điểm $P_0(x_0, y_0)$ với góc nghiêng θ so với trục hoành ($m = \tan\theta, \theta = \arctan(m)$).

Để xây dựng phép biến đổi, ta phân rã bài toán thành chuỗi năm phép biến đổi hình học cơ sở:

1. Tịnh tiến đường thẳng L về gốc tọa độ: Điểm giao của L với trục tung là $(0, b)$, ta tịnh tiến không gian bởi ma trận $\mathbf{T}(0, -b)$ sao cho đường thẳng đi qua gốc tọa độ O.
2. Xoay đường thẳng L trùng với trục hoành Ox: Áp dụng phép quay một góc $-\theta$ quanh gốc tọa độ bởi ma trận $\mathbf{R}(-\theta)$.
3. Thực hiện phép đối xứng cơ sở qua trục Ox: Áp dụng ma trận $\mathbf{R}_x$.
4. Xoay ngược đường thẳng về góc nghiêng ban đầu: Áp dụng ma trận quay $\mathbf{R}(\theta)$.
5. Tịnh tiến ngược đường thẳng về vị trí ban đầu: Áp dụng ma trận tịnh tiến $\mathbf{T}(0, b)$.

Ma trận biến đổi đối xứng tổng hợp $\mathbf{M}_{\text{ref}}$ được xác lập thông qua tích chuỗi ma trận:

$$\mathbf{M}_{\text{ref}} = \mathbf{T}(0, b) \cdot \mathbf{R}(\theta) \cdot \mathbf{R}_x \cdot \mathbf{R}(-\theta) \cdot \mathbf{T}(0, -b) \tag{132}$$

Nhận xét rằng tích ba ma trận ở giữa chính là phép đối xứng qua đường thẳng đi qua gốc tọa độ nghiêng góc θ:

$$\begin{aligned}\mathbf{R}(\theta) \cdot \mathbf{R}_x \cdot \mathbf{R}(-\theta) &= \begin{bmatrix}\cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1\end{bmatrix} \begin{bmatrix}1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1\end{bmatrix} \begin{bmatrix}\cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1\end{bmatrix} \\ &= \begin{bmatrix}\cos^2\theta - \sin^2\theta & 2\sin\theta\cos\theta & 0 \\ 2\sin\theta\cos\theta & \sin^2\theta - \cos^2\theta & 0 \\ 0 & 0 & 1\end{bmatrix} \\ &= \begin{bmatrix}\cos(2\theta) & \sin(2\theta) & 0 \\ \sin(2\theta) & -\cos(2\theta) & 0 \\ 0 & 0 & 1\end{bmatrix}\end{aligned} \tag{133}$$

Kết hợp với các ma trận tịnh tiến $\mathbf{T}(0, b)$ và $\mathbf{T}(0, -b)$, ta thu được ma trận đối xứng tổng quát hoàn chỉnh:

$$\mathbf{M}_{\text{ref}} = \begin{bmatrix} \cos(2\theta) & \sin(2\theta) & -b\sin(2\theta) \\ \sin(2\theta) & -\cos(2\theta) & b(1+\cos(2\theta)) \\ 0 & 0 & 1 \end{bmatrix} \tag{134}$$

Sử dụng các hệ thức lượng giác liên hệ với hệ số góc $m = \tan\theta$:

$$\cos(2\theta) = \frac{1-m^2}{1+m^2}, \quad \sin(2\theta) = \frac{2m}{1+m^2} \tag{135}$$

ta có thể tính toán toàn bộ các hệ số của ma trận $\mathbf{M}_{\text{ref}}$ trực tiếp từ các tham số đại số (m, b) của đường thẳng mà không cần gọi các hàm lượng giác tiêu tốn tài nguyên.

6.6 Vai trò của Phép Biến đổi từ Hệ Tọa độ Thế giới Thực sang Hệ Tọa độ Thiết bị Hiển thị

Trong quy trình kết xuất đồ họa (Graphics Pipeline), sự phân tách giữa không gian thế giới thực và không gian thiết bị hiển thị là nguyên lý kiến trúc mang tính sống còn. Đối tượng đồ họa được định nghĩa trong Hệ tọa độ Thế giới (World Coordinate System, WCS) với đơn vị đo vật lý liên tục (như mét, kilomet hoặc micromet) và phạm vi tọa độ tùy ý. Ngược lại, thiết bị hiển thị như màn hình máy tính hoạt động trong Hệ tọa độ Thiết bị (Display Coordinate System, DCS) với đơn vị đo là các điểm ảnh số nguyên rời rạc (Pixels) và có gốc tọa độ thường đặt tại góc trên bên trái màn hình.

Phép biến đổi từ tọa độ thế giới sang tọa độ thiết bị (Window-to-Viewport Transformation) đảm nhận các vai trò trọng yếu:

- *Độc lập hóa Thiết bị và Ứng dụng:* Lập trình viên có thể mô hình hóa các vật thể hình học hoàn toàn tự nhiên theo kích thước thực nghiệm của bài toán mà không cần quan tâm đến việc chương trình sẽ được chạy trên màn hình độ phân giải Full HD (1920×1080), màn hình 4K hay màn hình hiển thị di động.
- *Định hình Khung nhìn và Tỉ lệ Thu phóng (Zoom & Pan):* Bằng cách thay đổi tọa độ của cửa sổ quan sát (Window) trong thế giới thực, hệ thống cho phép người dùng phóng to từng chi tiết vi mô hoặc quan sát toàn cảnh vĩ mô của bản vẽ mà không làm biến dạng dữ liệu hình học gốc.
- *Thực hiện Cơ chế Xén tỉa Đồ họa (Clipping):* Phép biến đổi đưa các tọa độ về không gian chuẩn hóa, cho phép các giải thuật xén tỉa như Cohen-Sutherland hay Sutherland-Hodgman loại bỏ hoàn toàn các thực thể nằm ngoài vùng quan sát trước khi tiến hành raster hóa, tiết kiệm hàng triệu chu kỳ tính toán lãng phí của GPU.
- *Hỗ trợ Cơ chế Đa Cổng nhìn (Multiple Viewports):* Cùng một dữ liệu mô hình thế giới thực có thể được ánh xạ đồng thời lên nhiều cổng nhìn (Viewport) khác nhau trên màn hình để cung cấp các góc quan sát đa chiều (hình chiếu đứng, hình chiếu bằng, hình chiếu cạnh và phối cảnh 3D) trong các phần mềm thiết kế chuyên nghiệp.

6.7 Tính Cần thiết của Cơ chế Đồng bộ Hai chiều Giữa Ánh xạ Cửa sổ - Cổng nhìn (Win-to-View) và Cổng nhìn - Cửa sổ (View-to-Win)

Một hệ thống đồ họa tương tác hoàn chỉnh không thể chỉ vận hành theo một chiều từ dữ liệu mô hình ra màn hình hiển thị. Bên cạnh luồng ánh xạ thuận từ cửa sổ sang cổng nhìn (*Win-to-View*), hệ thống bắt buộc phải duy trì một luồng ánh xạ nghịch đồng bộ từ cổng nhìn ngược trở lại cửa sổ thế giới thực (*View-to-Win*).

1. Bản chất Toán học của Hai Phép Ánh xạ: Giả sử cửa sổ thế giới thực được giới hạn bởi $[x_{w\,\min}, x_{w\,\max}] \times [y_{w\,\min}, y_{w\,\max}]$ và cổng nhìn hiển thị trên màn hình được giới hạn bởi $[x_{v\,\min}, x_{v\,\max}] \times [y_{v\,\min}, y_{v\,\max}]$. Phép ánh xạ thuận đưa một điểm $P_w(x_w, y_w)$ trong thế giới thực sang điểm $P_v(x_v, y_v)$ trên màn hình tuân theo phương trình tỉ lệ:

$$\begin{cases} x_v = x_{v\,\min} + (x_w - x_{w\,\min}) \cdot s_x \\ y_v = y_{v\,\min} + (y_w - y_{w\,\min}) \cdot s_y \end{cases} \tag{136}$$

trong đó hệ số co giãn tỉ lệ là:

$$s_x = \frac{x_{v\,\max} - x_{v\,\min}}{x_{w\,\max} - x_{w\,\min}}, \quad s_y = \frac{y_{v\,\max} - y_{v\,\min}}{y_{w\,\max} - y_{w\,\min}} \tag{137}$$

Ngược lại, khi người dùng thực hiện một tương tác chuột tại tọa độ pixel (x_v, y_v) trên màn hình, hệ thống phải kích hoạt phép ánh xạ nghịch (*View-to-Win*) để xác định vị trí hình học thực $P_w(x_w, y_w)$:

$$\begin{cases} x_w = x_{w\,\min} + (x_v - x_{v\,\min}) \cdot \frac{1}{s_x} \\ y_w = y_{w\,\min} + (y_v - y_{v\,\min}) \cdot \frac{1}{s_y} \end{cases} \tag{138}$$

2. Sự Cần thiết Tuyệt đối của Tính Đồng bộ Hai chiều: Hai phép biến đổi trên là nghịch đảo hàm số chặt chẽ của nhau: $\mathbf{M}_{\text{view-to-win}} = \mathbf{M}^{-1}_{\text{win-to-view}}$. Việc đồng bộ hóa hoàn hảo giữa hai phép ánh xạ này là điều kiện tiên quyết vì các lý do sau:

- *Đảm bảo Độ chính xác trong Thao tác Chọn Đối tượng (Picking & Selection):* Khi người dùng nhấp chuột để chọn một đỉnh hoặc một đường cong, hệ thống phải chuyển tọa độ pixel của chuột về tọa độ thế giới thực để so khớp khoảng cách với các đối tượng hình học. Nếu có sự sai lệch hoặc không đồng bộ tham số giữa hai phép biến đổi, tia chọn (Picking Ray) sẽ bị lệch khỏi mục tiêu, khiến thao tác tương tác bị trượt hoặc chọn nhầm thực thể lân cận.
- *Bảo toàn Tính Nhất quán khi Vẽ Trực tiếp (Direct Manipulation):* Trong các ứng dụng CAD hay đồ họa vector, khi người dùng kéo thả chuột để tạo một điểm mút mới hay di chuyển một đường cong Bézier, tọa độ chuyển động của chuột phải được phiên dịch tức thời và chính xác sang không gian mô hình. Sự đồng bộ đảm bảo rằng điểm được vẽ ra trên màn hình luôn bám sát vị trí con trỏ chuột mà không xuất hiện độ trễ hay sai số tích lũy không gian.
- *Bảo toàn Tỉ lệ Khung hình (Aspect Ratio Preservation):* Khi cửa sổ ứng dụng bị người dùng kéo dãn thay đổi kích thước, nếu hệ số s_x và s_y không được đồng bộ điều chỉnh đồng thời trên cả hai chiều ánh xạ, các hình tròn trong thế giới thực sẽ bị méo mó thành hình ellipse khi hiển thị, và các thao tác tương tác chuột trên màn hình sẽ tạo ra những dịch chuyển méo mó phi tuyến trong thế giới thực.

6.8 Xây dựng Ma trận Biến đổi Quay quanh một Trục Bất kỳ trong Không gian 3 Chiều

Trong không gian ba chiều, việc quay một đối tượng quanh một trục tọa độ chuẩn (Ox, Oy, Oz) được thực hiện thông qua các ma trận cơ sở kinh điển. Tuy nhiên, bài toán tổng quát yêu cầu thực hiện phép quay một góc θ quanh một trục định hướng bất kỳ trong không gian, được xác định bởi điểm xuất phát $P_1(x_1, y_1, z_1)$ và điểm kết thúc $P_2(x_2, y_2, z_2)$ (Hearn & Baker, 2004).

Vector chỉ phương của trục quay được xác định bởi:

$$\mathbf{V} = \vec{P_1 P_2} = (x_2 - x_1, y_2 - y_1, z_2 - z_1) \tag{139}$$

Chuẩn hóa vector chỉ phương để thu được vector đơn vị $\mathbf{u}$:

$$\mathbf{u} = \frac{\mathbf{V}}{\|\mathbf{V}\|} = (a, b, c), \quad \text{với } a^2 + b^2 + c^2 = 1 \tag{140}$$

Để xây dựng ma trận quay tổng hợp $\mathbf{R}(\mathbf{V}, \theta)$, ta áp dụng nguyên lý phân rã hình học liên hoàn gồm 5 bước tiêu chuẩn (minh họa tại Slide bài giảng số 9):

1. *Bước 1: Tịnh tiến trục quay về gốc tọa độ ($P_1 \to O$):* Dịch chuyển toàn bộ không gian sao cho điểm P_1 chuyển về trùng với gốc tọa độ $O(0, 0, 0)$, sử dụng ma trận tịnh tiến:

$$\mathbf{T}(P_1 \to O) = \begin{bmatrix} 1 & 0 & 0 & -x_1 \\ 0 & 1 & 0 & -y_1 \\ 0 & 0 & 1 & -z_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{141}$$

2. *Bước 2: Xoay trục quay trùng khít với trục tọa độ chuẩn Oz:* Quá trình này được phân rã thành hai phép quay liên tiếp:
   - *Bước 2a: Quay quanh trục Ox một góc α để đưa vector $\mathbf{u}$ vào mặt phẳng tọa độ Oxz:* Phép chiếu của vector $\mathbf{u}$ lên mặt phẳng Oyz là vector $\mathbf{u}' = (0, b, c)$ có độ dài $d = \sqrt{b^2 + c^2}$. Phép quay quanh trục Ox đưa $\mathbf{u}'$ về trùng với trục Oz, ma trận quay tương ứng là:
   $$\mathbf{R}_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c/d & -b/d & 0 \\ 0 & b/d & c/d & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{142}$$
   Sau bước này, vector $\mathbf{u}$ biến thành vector $\mathbf{u}'' = (a, 0, d)$ nằm trọn vẹn trong mặt phẳng Oxz.
   - *Bước 2b: Quay quanh trục Oy một góc $-\beta$ để đưa vector $\mathbf{u}''$ trùng khít hoàn toàn với trục Oz:* Vector $\mathbf{u}''$ hợp với trục Oz một góc β thỏa mãn $\cos\beta = d$ và $\sin\beta = -a$. Phép quay quanh trục Oy được xác lập bởi ma trận:
   $$\mathbf{R}_y(\beta) = \begin{bmatrix} d & 0 & -a & 0 \\ 0 & 1 & 0 & 0 \\ a & 0 & d & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{143}$$

3. *Bước 3: Thực hiện phép quay mong muốn quanh trục chuẩn Oz một góc θ:*
   $$\mathbf{R}_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{144}$$

4. *Bước 4: Thực hiện các phép quay ngược để khôi phục hướng ban đầu của trục quay:* Áp dụng lần lượt ma trận nghịch đảo của phép quay quanh trục Oy và phép quay quanh trục Ox:
   $$\mathbf{R}_y^{-1}(\beta) = \mathbf{R}_y(-\beta) = \begin{bmatrix} d & 0 & a & 0 \\ 0 & 1 & 0 & 0 \\ -a & 0 & d & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{R}_x^{-1}(\alpha) = \mathbf{R}_x(-\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c/d & b/d & 0 \\ 0 & -b/d & c/d & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{145}$$

5. *Bước 5: Tịnh tiến ngược toàn bộ không gian để đưa trục quay về vị trí ban đầu ($O \to P_1$):*
   $$\mathbf{T}^{-1}(P_1 \to O) = \mathbf{T}(O \to P_1) = \begin{bmatrix} 1 & 0 & 0 & x_1 \\ 0 & 1 & 0 & y_1 \\ 0 & 0 & 1 & z_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{146}$$

Ma trận biến đổi quay tổng quát hoàn chỉnh $\mathbf{R}(\mathbf{V}, \theta)$ được thiết lập thông qua tích chuỗi của 7 ma trận cơ sở:

$$\mathbf{R}(\mathbf{V}, \theta) = \mathbf{T}(P_1) \cdot \mathbf{R}_x^{-1}(\alpha) \cdot \mathbf{R}_y^{-1}(\beta) \cdot \mathbf{R}_z(\theta) \cdot \mathbf{R}_y(\beta) \cdot \mathbf{R}_x(\alpha) \cdot \mathbf{T}(-P_1) \tag{147}$$

Đặt $\mathbf{A}(\mathbf{V}) = \mathbf{R}_x(\alpha) \cdot \mathbf{R}_y(\beta)$ là ma trận canh chỉnh trục quay trùng với trục Oz, công thức trên được viết lại dưới dạng cô đọng:

$$\mathbf{R}(\mathbf{V}, \theta) = \mathbf{T}(P_1) \cdot \mathbf{A}^{-1}(\mathbf{V}) \cdot \mathbf{R}_z(\theta) \cdot \mathbf{A}(\mathbf{V}) \cdot \mathbf{T}(-P_1) \tag{148}$$

6.9 Chứng minh Sự Đồng nhất của Hai Góc α trong Bài toán Quay quanh Trục 3D (Slide 28)

Trong tài liệu bài giảng (Lecture 9, trang 28), việc xác định góc quay quanh trục Ox được minh họa thông qua hai sơ đồ hình học kề nhau: sơ đồ bên trái thể hiện góc hợp bởi vector hình chiếu $\mathbf{u}'$ và vector đơn vị trục Oz, trong khi sơ đồ bên phải thể hiện góc quay của trục tọa độ để đưa không gian

về trạng thái canh chỉnh. Cần chứng minh rằng hai góc được ký hiệu cùng bằng α trong hai sơ đồ này thực chất là hoàn toàn đồng nhất về mặt giải tích và hình học.

1. Thiết lập Hình học: Xét vector đơn vị chỉ phương của trục quay $\mathbf{u} = (a, b, c)$ xuất phát từ gốc tọa độ O. Ta có thể phân tích duy nhất vector $\mathbf{u}$ thành tổng của hai thành phần trực giao:

$$\mathbf{u} = a\mathbf{u}_x + \mathbf{u}' \tag{149}$$

trong đó $\mathbf{u}_x = (1, 0, 0)$ là vector đơn vị dọc theo trục Ox, và $\mathbf{u}' = (0, b, c)$ là vector hình chiếu trực giao của $\mathbf{u}$ lên mặt phẳng tọa độ Oyz. Độ dài của vector hình chiếu là $d = \|\mathbf{u}'\| = \sqrt{b^2 + c^2}$.

Trong sơ đồ bên trái (Slide 28), góc α được định nghĩa là góc phẳng nằm trong mặt phẳng Oyz, hợp bởi vector hình chiếu $\mathbf{u}' = (0, b, c)$ và vector đơn vị của trục Oz là $\mathbf{u}_z = (0, 0, 1)$. Áp dụng định nghĩa tích vô hướng trong mặt phẳng Oyz:

$$\cos\alpha = \frac{\mathbf{u}' \cdot \mathbf{u}_z}{\|\mathbf{u}'\|\|\mathbf{u}_z\|} = \frac{0 \cdot 0 + b \cdot 0 + c \cdot 1}{d \cdot 1} = \frac{c}{d} \tag{150}$$

2. Phân tích Góc Quay Không gian trong Sơ đồ bên Phải: Trong sơ đồ bên phải (Slide 28a), mục tiêu hình học là thực hiện một phép quay không gian ba chiều quanh trục Ox một góc θ_x sao cho vector $\mathbf{u}$ sau phép quay nằm trọn vẹn trong mặt phẳng tọa độ Oxz. Một vector bất kỳ nằm trong mặt phẳng Oxz khi và chỉ khi thành phần tung độ y của nó triệt tiêu hoàn toàn ($y = 0$).

Xét tác động của phép quay quanh trục Ox một góc θ_x lên vector $\mathbf{u}$:

$$\mathbf{u}_{\text{new}} = \mathbf{R}_x(\theta_x)\mathbf{u} = \mathbf{R}_x(\theta_x)\left(a\mathbf{u}_x + \mathbf{u}'\right) = a\mathbf{u}_x + \mathbf{R}_x(\theta_x)\mathbf{u}' \tag{151}$$

Bởi vì trục quay là Ox, thành phần $a\mathbf{u}_x$ hoàn toàn bất biến trước phép quay. Do đó, yêu cầu vector $\mathbf{u}_{\text{new}}$ nằm trong mặt phẳng Oxz tương đương tuyệt đối với yêu cầu:

$$\mathbf{R}_x(\theta_x)\mathbf{u}' \text{ phải nằm dọc theo trục } Oz \iff \mathbf{R}_x(\theta_x)\begin{bmatrix}0\\b\\c\end{bmatrix} = \begin{bmatrix}0\\0\\d\end{bmatrix} \tag{152}$$

Xét phép quay trong mặt phẳng Oyz: Tọa độ của vector $\mathbf{u}'$ trong hệ trục (Oy, Oz) là $(b, c) = (d\sin\alpha, d\cos\alpha)$. Phép quay một góc θ_x biến tọa độ này thành:

$$\begin{bmatrix}y_{\text{new}}\\z_{\text{new}}\end{bmatrix} = \begin{bmatrix}\cos\theta_x & -\sin\theta_x\\ \sin\theta_x & \cos\theta_x\end{bmatrix}\begin{bmatrix}b\\c\end{bmatrix} = \begin{bmatrix}b\cos\theta_x - c\sin\theta_x\\ b\sin\theta_x + c\cos\theta_x\end{bmatrix} \tag{153}$$

Để thành phần tung độ triệt tiêu ($y_{\text{new}} = 0$):

$$b\cos\theta_x - c\sin\theta_x = 0 \iff \tan\theta_x = \frac{b}{c} \tag{154}$$

Mặt khác, từ tam giác vuông tạo bởi vector hình chiếu $\mathbf{u}'(0, b, c)$ và trục Oz, ta có:

$$\tan\alpha = \frac{b}{c} \tag{155}$$

Do cả hai góc đều nằm trong khoảng xác định của phép quay hướng dương quanh trục Ox, ta suy ra trực tiếp:

$$\theta_x = \alpha \tag{156}$$

Điều này chứng minh hoàn toàn tính tương đương chặt chẽ: Góc quay cần thiết quanh trục Ox để đưa vector $\mathbf{u}$ vào mặt phẳng Oxz chính là góc phẳng α hợp bởi vector hình chiếu $\mathbf{u}'$ và trục Oz.

6.10 Cơ sở Hiệu năng và Ứng dụng Thực tiễn của Tích Vô hướng và Tích Hữu hướng trong Đồ họa

Trong toàn bộ kho tàng đại số tuyến tính ứng dụng cho đồ họa máy tính, tích vô hướng (Dot Product) và tích hữu hướng (Cross Product) là hai phép toán có tần suất sử dụng cao nhất và mang lại hiệu quả tính toán vượt trội nhất.

1. Cơ sở Hiệu năng Tính toán:

- *Tích vô hướng* $\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z$*:* Phép toán này chỉ tiêu tốn đúng 3 phép nhân và 2 phép cộng dấu phẩy động trên CPU/GPU. Mặc dù chi phí tính toán cực kỳ thấp, giá trị vô hướng thu được lại cung cấp ngay lập tức thông tin góc giữa hai vector ($\cos\theta = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$) và dấu của nó xác định điểm kiểm tra nằm ở phía trước, phía sau hay nằm trên một mặt phẳng siêu việt mà không cần gọi hàm lượng giác nghịch đảo tiêu tốn tài nguyên.
- *Tích hữu hướng* $\mathbf{a} \times \mathbf{b} = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$*:* Phép toán này tiêu tốn 6 phép nhân và 3 phép trừ, cho kết quả là một vector trực giao đồng thời với cả hai vector thành phần, có độ lớn bằng diện tích hình bình hành căng bởi hai vector đó.

2. Danh mục Ứng dụng Thực tiễn Tiêu biểu trong Đồ họa:

- *Tính toán Vector Pháp tuyến Bề mặt (Surface Normal Computation):* Với mọi đa giác phẳng hoặc tam giác 3D tạo bởi ba đỉnh V_0, V_1, V_2, vector pháp tuyến đơn vị của mặt được tính trực tiếp thông qua tích hữu hướng:

$$\mathbf{N} = \frac{(V_1 - V_0) \times (V_2 - V_0)}{\|(V_1 - V_0) \times (V_2 - V_0)\|} \tag{157}$$

- *Mô hình Chiếu sáng và Đánh bóng Bề mặt (Illumination & Shading):* Trong mô hình phản xạ Lambertian, cường độ ánh sáng khuếch tán tỉ lệ thuận với tích vô hướng giữa pháp tuyến bề mặt $\mathbf{N}$ và vector hướng sáng $\mathbf{L}$: $I_{\text{diff}} = I_l k_d \max(0, \mathbf{N} \cdot \mathbf{L})$. Trong mô hình Phong và Blinn-Phong, phản xạ gương được xác định bằng lũy thừa của tích vô hướng: $(\mathbf{R} \cdot \mathbf{V})^n$ hoặc $(\mathbf{N} \cdot \mathbf{H})^n$.
- *Khử Mặt khuất Bằng Phép Kiểm tra Mặt sau (Back-face Culling):* Một tam giác có pháp tuyến ngoài $\mathbf{N}$ sẽ quay lưng về phía người quan sát nếu tích vô hướng giữa pháp tuyến và vector hướng nhìn $\mathbf{V}_{\text{view}}$ thỏa mãn $\mathbf{N} \cdot \mathbf{V}_{\text{view}} \geq 0$. Kỹ thuật này loại bỏ ngay lập tức 50% số lượng tam giác trước khi gửi vào đường ống raster hóa.
- *Thiết lập Hệ Khung nhìn Camera (Camera Basis Construction):* Cho trước vị trí camera $\mathbf{E}$, điểm ngắm $\mathbf{T}$ và vector hướng lên $\mathbf{V}_{\text{up}}$, ba trục tọa độ trực chuẩn của camera được kiến tạo thuần túy bằng các phép tích hữu hướng liên hoàn:

$$\mathbf{w} = \frac{\mathbf{E} - \mathbf{T}}{\|\mathbf{E} - \mathbf{T}\|}, \quad \mathbf{u} = \frac{\mathbf{V}_{\text{up}} \times \mathbf{w}}{\|\mathbf{V}_{\text{up}} \times \mathbf{w}\|}, \quad \mathbf{v} = \mathbf{w} \times \mathbf{u} \tag{158}$$

- *Thuật toán Xén tỉa Đoạn thẳng Cyrus-Beck:* Xác định giao điểm giữa tia và đa giác lồi thông qua dấu của tích vô hướng giữa vector chỉ phương đoạn thẳng và vector pháp tuyến trong của từng cạnh.
- *Kiểm tra Giao điểm Tia - Tam giác (Möller-Trumbore Algorithm):* Sử dụng kết hợp tích hữu hướng và tích vô hướng (tích hỗn tạp Scalar Triple Product) để giải trực tiếp hệ phương trình tọa độ tỉ cự (Barycentric Coordinates) mà không cần tính phương trình mặt phẳng chứa tam giác.

6.11 Chứng minh Biểu thức Tích Hữu hướng $\vec{u}' \times \vec{u}_z = b \cdot \vec{u}_x$ (Slide 29)

Trong tài liệu bài giảng (Lecture 9, Slide 29), khi giải bài toán tìm $\sin\alpha$ phục vụ cho việc thiết lập ma trận quay $\mathbf{R}_x(\alpha)$, tài liệu đưa ra hai biểu thức đồng thời:

$$\begin{cases} \mathbf{u}' \times \mathbf{u}_z = \mathbf{u}_x \cdot \|\mathbf{u}'\| \cdot \|\mathbf{u}_z\| \cdot \sin\alpha \\ \mathbf{u}' \times \mathbf{u}_z = \mathbf{u}_x \cdot b \end{cases} \implies \sin\alpha = \frac{b}{d} \tag{159}$$

Dưới đây là chứng minh giải tích chi tiết cho biểu thức tích hữu hướng $\mathbf{u}' \times \mathbf{u}_z = \mathbf{u}_x \cdot b$.

1. Thiết lập Tọa độ Đại số của các Vector: Trong không gian ba chiều Euclid với hệ tọa độ trực chuẩn ba chiều bàn tay phải $\{O; \mathbf{u}_x, \mathbf{u}_y, \mathbf{u}_z\}$, ta có:

$$\mathbf{u}' = (0, b, c) = 0 \cdot \mathbf{u}_x + b \cdot \mathbf{u}_y + c \cdot \mathbf{u}_z \tag{160}$$

$$\mathbf{u}_z = (0, 0, 1) = 0 \cdot \mathbf{u}_x + 0 \cdot \mathbf{u}_y + 1 \cdot \mathbf{u}_z \tag{161}$$

2. Khai triển Định thức Tích Hữu hướng: Theo định nghĩa đại số của tích hữu hướng giữa hai vector trong không gian $\mathbb{R}^3$, ta thiết lập định thức hình thức cấp 3:

$$\mathbf{u}' \times \mathbf{u}_z = \begin{vmatrix} \mathbf{u}_x & \mathbf{u}_y & \mathbf{u}_z \\ 0 & b & c \\ 0 & 0 & 1 \end{vmatrix} \tag{162}$$

Khai triển định thức theo hàng thứ nhất:

$$\begin{aligned} \mathbf{u}' \times \mathbf{u}_z &= \mathbf{u}_x \begin{vmatrix} b & c \\ 0 & 1 \end{vmatrix} - \mathbf{u}_y \begin{vmatrix} 0 & c \\ 0 & 1 \end{vmatrix} + \mathbf{u}_z \begin{vmatrix} 0 & b \\ 0 & 0 \end{vmatrix} \\ &= \mathbf{u}_x(b \cdot 1 - c \cdot 0) - \mathbf{u}_y(0 \cdot 1 - c \cdot 0) + \mathbf{u}_z(0 \cdot 0 - b \cdot 0) \\ &= \mathbf{u}_x \cdot b - \mathbf{u}_y \cdot 0 + \mathbf{u}_z \cdot 0 \\ &= b \cdot \mathbf{u}_x \end{aligned} \tag{163}$$

3. Đối chiếu với Định nghĩa Hình học của Tích Hữu hướng: Theo định nghĩa hình học, tích hữu hướng của hai vector $\mathbf{u}'$ và $\mathbf{u}_z$ là một vector có:

- Phương trực giao với mặt phẳng chứa $\mathbf{u}'$ và $\mathbf{u}_z$: Vì cả hai vector đều nằm trong mặt phẳng tọa độ Oyz ($x = 0$), vector tích hữu hướng bắt buộc phải có phương song song hoặc trùng với trục hoành Ox, tức cùng phương với vector đơn vị $\mathbf{u}_x$.
- Chiều tuân theo quy tắc bàn tay phải: Khi quay ngón tay từ $\mathbf{u}'$ sang $\mathbf{u}_z$, ngón cái chỉ dọc theo chiều dương của trục Ox, tức cùng hướng với $\mathbf{u}_x$ khi $b > 0$.
- Độ lớn bằng diện tích hình bình hành:

$$\|\mathbf{u}' \times \mathbf{u}_z\| = \|\mathbf{u}'\| \cdot \|\mathbf{u}_z\| \cdot \sin\alpha = d \cdot 1 \cdot \sin\alpha \tag{164}$$

So sánh độ lớn từ kết quả khai triển đại số $\|\mathbf{u}' \times \mathbf{u}_z\| = \|b \cdot \mathbf{u}_x\| = |b| = b$ (với giả thiết vị trí góc phần tư thứ nhất $b \geq 0$), ta thu được phương trình:

$$d \cdot \sin\alpha = b \implies \sin\alpha = \frac{b}{d} \tag{165}$$

Biểu thức này kết hợp hoàn hảo với công thức tích vô hướng $\cos\alpha = \frac{c}{d}$ ở phương trình equation 150, cung cấp đầy đủ hai thành phần lượng giác chính xác để điền trực tiếp vào ma trận quay $\mathbf{R}_x(\alpha)$ tại phương trình equation 142 mà không phát sinh bất kỳ phép tính góc lượng giác thực nghiệm nào.

7 TUẦN 7: MÔ HÌNH HÓA HÌNH HỌC THAM SỐ 3D, CẤU TRÚC DỮ LIỆU BẢN VẼ VÀ KỸ THUẬT MẶT CONG

7.1 THUẬT TOÁN VÀ CÀI ĐẶT TÍNH TOÁN x_k THEO PHƯƠNG PHÁP SAI PHÂN QUY NẠP

Trong kỹ thuật raster hóa và tô màu đa giác theo dòng quét (Scan-line Polygon Fill), tại mỗi bước chuyển đổi giữa hai dòng quét liên tiếp từ y_k lên $y_{k+1} = y_k + 1$, thuật toán cần xác định chính xác hoành độ giao điểm mới x_{k+1} của dòng quét với từng cạnh kích hoạt. Một đoạn thẳng nối hai điểm mút $A(x_1, y_1)$ và $B(x_2, y_2)$ có phương trình giải tích dạng:

$$x = m^{-1}(y - y_1) + x_1 \tag{166}$$

trong đó hệ số nghịch đảo độ dốc là $m^{-1} = \frac{\Delta x}{\Delta y} = \frac{x_2 - x_1}{y_2 - y_1}$.

Nếu tính toán trực tiếp hoành độ giao điểm tại mỗi dòng quét bằng phép nhân hoặc chia số thực, hệ thống vừa tiêu tốn chu kỳ xử lý, vừa tích lũy sai số làm tròn số thực. Phương pháp sai phân tiến quy nạp (Forward Differencing) giải quyết triệt để vấn đề này bằng cách thiết lập công thức truy hồi:

$$x_{k+1} = x_k + \frac{1}{m} = x_k + \frac{\Delta x}{\Delta y} \tag{167}$$

Để khử hoàn toàn phép tính số thực, ta phân tích số nguyên Δx thành phép chia có dư cho Δy:

$$\Delta x = q \cdot \Delta y + r, \quad \text{với } q = \left\lfloor \frac{\Delta x}{\Delta y} \right\rfloor,\ 0 \le r < \Delta y \tag{168}$$

Tại mỗi bước quét, phần nguyên của hoành độ tăng tối thiểu một lượng q. Phần dư r được tích lũy vào một biến số nguyên tích lũy (Accumulator). Khi bộ tích lũy vượt quá ngưỡng Δy, hoành độ x được cộng thêm 1 đơn vị và biến tích lũy được trừ bớt Δy.

Đoạn mã C++ chuẩn dưới đây minh họa giải thuật tính toán x_k bằng sai phân số nguyên thuần túy:

```
#include <iostream>
#include <cmath>

struct EdgeInterpolator {
    int x_current;      // Hoành độ nguyên hiện tại
    int y_current;      // Tung độ dòng quét hiện tại
    int y_end;          // Tung độ kết thúc
    int step_x;         // Bước nhảy cơ sở q
    int remainder;      // Phần dư r
    int denominator;    // Mẫu số delta_y
    int accumulator;    // Bộ tích lũy phần dư

    EdgeInterpolator(int x1, int y1, int x2, int y2) {
        if (y1 > y2) { std::swap(x1, x2); std::swap(y1, y2); }
        x_current = x1;
        y_current = y1;
        y_end = y2;
        int dx = x2 - x1;
        int dy = y2 - y1;
        denominator = dy;
        if (dy == 0) {
            step_x = 0; remainder = 0; accumulator = 0;
            return;
        }
        int sign = (dx >= 0) ? 1 : -1;
        int abs_dx = std::abs(dx);
        step_x = sign * (abs_dx / dy);
        remainder = abs_dx % dy;
        if (sign < 0 && remainder != 0) {
```

```
            step_x -= 1;
            remainder = dy - remainder;
        }
        accumulator = 0;
    }

    bool step() {
        if (y_current >= y_end) return false;
        y_current++;
        x_current += step_x;
        accumulator += remainder;
        if (accumulator >= denominator) {
            x_current += 1;
            accumulator -= denominator;
        }
        return true;
    }
};
```

7.2 Thuật toán Tìm Bước Sai phân Tối ưu Thỏa mãn Tính Liên tục 8-Lân cận

Một đường cong hoặc đoạn thẳng rời rạc được gọi là liên tục trong lân cận 8 pixel (8-connected continuity) khi và chỉ khi giữa hai pixel kế tiếp (x_k, y_k) và (x_{k+1}, y_{k+1}) trên màn hình, khoảng cách Chebyshev không vượt quá 1 đơn vị:

$$\max\left(|x_{k+1} - x_k|, |y_{k+1} - y_k|\right) \leq 1 \tag{169}$$

Đồng thời, để tránh lãng phí bộ nhớ và vẽ đè pixel không cần thiết, hai pixel liên tiếp không được phép trùng nhau hoàn toàn: $|x_{k+1} - x_k| + |y_{k+1} - y_k| \geq 1$.

Đối với một đường cong tham số tổng quát $\mathbf{P}(t) = (x(t), y(t))$ với $t \in [0, 1]$, bài toán đặt ra là xác định bước nhảy tham số Δt_k sao cho khi $t_{k+1} = t_k + \Delta t_k$, bước dịch chuyển tọa độ thỏa mãn chính xác tiêu chuẩn lân cận 8. Khai triển chuỗi Taylor bậc nhất cho tọa độ điểm:

$$x(t + \Delta t) \approx x(t) + x'(t)\Delta t, \quad y(t + \Delta t) \approx y(t) + y'(t)\Delta t \tag{170}$$

Áp dụng điều kiện liên tục lân cận 8 tại phương trình equation 169:

$$\max\left(|x'(t)|\Delta t, |y'(t)|\Delta t\right) \leq 1 \implies \Delta t \leq \frac{1}{\max\left(|x'(t)|, |y'(t)|\right)} \tag{171}$$

Giá trị bước sai phân tối ưu tại mỗi thời điểm tham số t được chọn bằng cận trên lớn nhất:

$$\Delta t_{\text{opt}}(t) = \frac{1}{\max\left(|x'(t)|, |y'(t)|\right)} \tag{172}$$

Chương trình C++ dưới đây thực thi cơ chế tìm bước sai phân thích nghi (Adaptive Forward Differencing) cho một đường cong tham số tùy ý, đảm bảo tính liên tục 8-lân cận hoàn hảo:

```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>

struct Point2D { int x, y; };

template <typename CurveFunc, typename DerivFunc>
std::vector<Point2D> rasterizeWith8Connectivity(CurveFunc P, DerivFunc dP) {
    std::vector<Point2D> pixels;
    double t = 0.0;
    auto [x0, y0] = P(0.0);
    Point2D last_pixel = { static_cast<int>(std::round(x0)),
```

```
                            static_cast<int>(std::round(y0)) };
    pixels.push_back(last_pixel);

    while (t < 1.0) {
        auto [dx_dt, dy_dt] = dP(t);
        double max_deriv = std::max(std::abs(dx_dt), std::abs(dy_dt));
        double delta_t = (max_deriv > 1e-6) ? (1.0 / max_deriv) : 0.01;
        if (t + delta_t > 1.0) delta_t = 1.0 - t;
        t += delta_t;

        auto [xt, yt] = P(t);
        Point2D current_pixel = { static_cast<int>(std::round(xt)),
                                  static_cast<int>(std::round(yt)) };

        // Kiểm tra điều kiện lân cận 8
        int step_x = std::abs(current_pixel.x - last_pixel.x);
        int step_y = std::abs(current_pixel.y - last_pixel.y);
        if (step_x > 1 || step_y > 1) {
            // Bước nhảy vượt quá 1 pixel, tinh chỉnh giảm nửa delta_t
            t -= delta_t;
            delta_t /= 2.0;
            t += delta_t;
            auto [x_fix, y_fix] = P(t);
            current_pixel = { static_cast<int>(std::round(x_fix)),
                              static_cast<int>(std::round(y_fix)) };
        }
        if (current_pixel.x != last_pixel.x || current_pixel.y != last_pixel.y) {
            pixels.push_back(current_pixel);
            last_pixel = current_pixel;
        }
    }
    return pixels;
}
```

7.3 Tiêu chuẩn Phân định Vị trí Tương quan Trái - Phải của Điểm so với Vector qua Tích Hữu hướng

Trong không gian hình học phẳng hai chiều, bài toán xác định một điểm kiểm tra $P(x_p, y_p)$ nằm ở phía bên trái, phía bên phải, hay nằm trực tiếp trên đường thẳng định hướng đi từ $A(x_a, y_a)$ đến $B(x_b, y_b)$ là bài toán nền tảng của hình học tính toán (Computational Geometry).

Xây dựng hai vector trong mặt phẳng:

$$\mathbf{V} = \vec{AB} = (x_b - x_a, y_b - y_a) \tag{173}$$

$$\mathbf{W} = \vec{AP} = (x_p - x_a, y_p - y_a) \tag{174}$$

Nhúng hai vector 2D này vào không gian ba chiều bằng cách gán thành phần cao độ $z = 0$: $\mathbf{V} = (x_b - x_a, y_b - y_a, 0)$ và $\mathbf{W} = (x_p - x_a, y_p - y_a, 0)$. Tích hữu hướng giữa $\mathbf{V}$ và $\mathbf{W}$ là một vector hoàn toàn định hướng dọc theo trục cao độ chuẩn Oz:

$$\mathbf{V} \times \mathbf{W} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ x_b - x_a & y_b - y_a & 0 \\ x_p - x_a & y_p - y_a & 0 \end{vmatrix} = [(x_b - x_a)(y_p - y_a) - (y_b - y_a)(x_p - x_a)]\,\mathbf{k} \tag{175}$$

Ta định nghĩa đại lượng vô hướng định hướng (Perp Dot Product hoặc 2D Cross Product):

$$D(A, B, P) = (x_b - x_a)(y_p - y_a) - (y_b - y_a)(x_p - x_a) \tag{176}$$

Theo định nghĩa hình học, độ lớn và dấu của $D(A, B, P)$ liên hệ trực tiếp với góc quay có hướng $\theta \in [-\pi, \pi]$ từ vector $\mathbf{V}$ sang vector $\mathbf{W}$ theo hệ thức $D = \|\mathbf{V}\|\|\mathbf{W}\| \sin\theta$. Quy tắc bàn tay phải cho phép thiết lập tiêu chuẩn phân loại xác định như sau:

- *Nếu* $D(A, B, P) > 0$*:* Vector $\mathbf{W}$ quay ngược chiều kim đồng hồ so với vector $\mathbf{V}$ ($\sin\theta > 0$). Điểm P nằm nghiêm ngặt ở phía bên trái (Left Side) của đường thẳng định hướng AB.
- *Nếu* $D(A, B, P) < 0$*:* Vector $\mathbf{W}$ quay thuận chiều kim đồng hồ so với vector $\mathbf{V}$ ($\sin\theta < 0$). Điểm P nằm nghiêm ngặt ở phía bên phải (Right Side) của đường thẳng định hướng AB.
- *Nếu* $D(A, B, P) = 0$*:* Hai vector $\mathbf{V}$ và $\mathbf{W}$ cộng tuyến ($\theta = 0$ hoặc $\theta = \pi$). Điểm P nằm trực tiếp trên đường thẳng đi qua hai điểm A và B.

Tiêu chuẩn này có ý nghĩa thực tế sống còn: Đại lượng $D(A, B, P)$ chính là hai lần diện tích có hướng của tam giác $\triangle ABP$. Nó là nhân lõi trong giải thuật xây dựng bao lồi Graham Scan, giải thuật xén tỉa Sutherland-Hodgman, và kiểm tra điểm nằm trong đa giác lồi trong thời gian tuyến tính $O(n)$.

7.4 Khảo sát Hệ thống các Ứng dụng Toàn diện của Tích Vô hướng và Tích Hữu hướng trong Đồ họa

Trong toàn bộ kiến trúc đường ống đồ họa, tích vô hướng và tích hữu hướng đóng vai trò là hai công cụ hình học vạn năng, hiện diện ở tất cả các tầng từ mô hình hóa đến chiếu sáng và kết xuất. Bảng 15 tổng hợp đối sánh toàn diện về đặc tính toán học và phạm vi ứng dụng của hai phép toán này.

Bảng 15: Tổng hợp so sánh đặc tính và ứng dụng của Tích Vô hướng và Tích Hữu hướng

Tiêu chí So sánh	**Tích Vô hướng (Dot Product)**	**Tích Hữu hướng (Cross Product)**
Bản chất Đầu ra	Đại lượng vô hướng $s \in \mathbb{R}$.	Vector trực giao $\mathbf{w} \in \mathbb{R}^3$.
Ý nghĩa Hình học	Đo lường góc, hình chiếu trực giao, khoảng cách mặt phẳng.	Đo lường diện tích, xác định phương trực giao, kiểm tra tính định hướng.
Độ phức tạp Tính toán	3 phép nhân, 2 phép cộng dấu phẩy động.	6 phép nhân, 3 phép trừ dấu phẩy động.
Ứng dụng Chiếu sáng	Tính cường độ tán xạ Lambert ($\mathbf{N}\cdot\mathbf{L}$), phản xạ gương Phong/Blinn-Phong.	Tính vector pháp tuyến phục vụ chiếu sáng nhẵn Gouraud và Phong.
Ứng dụng Khử Mặt khuất	Phép thử mặt sau Back-face Culling ($\mathbf{N} \cdot \mathbf{V}_{\text{view}} \geq 0$).	Xác định hướng quay đỉnh thuận/nghịch kim đồng hồ của tam giác 3D.
Ứng dụng Xén tỉa Đồ họa	Thuật toán Cyrus-Beck và Liang-Barsky xác định đoạn tia nằm trong biên.	Xác định nửa mặt phẳng chứa điểm trong Sutherland-Hodgman.
Ứng dụng Điều hướng	Tính góc xoay giữa hai hướng quan sát của camera.	Thiết lập hệ trục trực chuẩn Camera $(\mathbf{u}, \mathbf{v}, \mathbf{w})$ từ vector chỉ ngắm.
Ứng dụng Truy vết Tia	Tính giao điểm tia với mặt cầu, mặt phẳng và khối bao AABB.	Giải hệ phương trình tọa độ tỉ cự tam giác (Möller-Trumbore).

7.5 Phương pháp Tính Chiều dài Cung của Đường cong Bézier

Trong các hệ thống thiết kế đồ họa vector và hoạt hình máy tính, việc xác định chính xác chiều dài cung (Arc Length) của một đường cong Bézier tham số $\mathbf{P}(t)$ với $t \in [0, 1]$ là yêu cầu bắt buộc để phục vụ việc phân bố điểm đều, điều khiển vận tốc chuyển động của đối tượng theo thời gian thực (Constant Speed Motion Control) và đo đạc chiều dài biên dạng.

Về mặt giải tích vi tích phân, chiều dài cung L của đường cong từ $t = 0$ đến $t = 1$ được định nghĩa bằng tích phân đường loại một:

$$L = \int_0^1 \|\mathbf{P}'(t)\|\, dt = \int_0^1 \sqrt{x'(t)^2 + y'(t)^2 + z'(t)^2}\, dt \tag{177}$$

Đối với đường cong Bézier bậc ba, đạo hàm $\mathbf{P}'(t)$ là một đa thức vector bậc hai. Biểu thức dưới dấu căn thức do đó là một đa thức bậc bốn theo biến t. Về mặt lý thuyết hàm số, tích phân của căn bậc hai của đa thức bậc bốn không thể biểu diễn dưới dạng các hàm sơ cấp thông thường mà thuộc lớp tích phân elliptic. Do đó, đồ họa máy tính sử dụng hai kỹ thuật tính toán xấp xỉ số học tiêu chuẩn:

1. Phương pháp Cầu phương Gauss-Legendre (Gauss-Legendre Quadrature): Phương pháp này xấp xỉ tích phân giải tích bằng tổng có trọng số của các điểm đánh giá tối ưu:

$$L \approx \sum_{i=1}^{n} w_i \|\mathbf{P}'(t_i)\| \tag{178}$$

với $n = 5$ điểm cầu phương, sai số tính toán nhỏ hơn 10^{-6}, hoàn toàn đáp ứng độ chính xác cấp độ pixel.

2. Phương pháp Phân chia Thích nghi (Adaptive Subdivision): Dựa trên thuật toán De Casteljau, đường cong được chia đôi tại $t = 0.5$ thành hai đoạn cong trái và phải. Ta so sánh chiều dài dây cung nối hai mút $L_{\text{chord}} = \|\mathbf{P}_3 - \mathbf{P}_0\|$ với chiều dài đa giác điều khiển $L_{\text{poly}} = \sum_{i=0}^{2} \|\mathbf{P}_{i+1} - \mathbf{P}_i\|$. Nếu hiệu số $|L_{\text{poly}} - L_{\text{chord}}| < \epsilon$, đường cong đủ phẳng và chiều dài được xấp xỉ bằng $\frac{L_{\text{chord}} + L_{\text{poly}}}{2}$. Ngược lại, giải thuật tiếp tục phân chia đệ quy.

Dưới đây là mã nguồn C++ hoàn chỉnh triển khai phương pháp tích phân Gauss-Legendre 5 điểm:

```
#include <iostream>
#include <cmath>
#include <array>

struct Vector2D {
    double x, y;
    double length() const { return std::sqrt(x * x + y * y); }
};

struct CubicBezier {
    Vector2D p0, p1, p2, p3;

    Vector2D derivative(double t) const {
        double u = 1.0 - t;
        double c0 = -3.0 * u * u;
        double c1 = 3.0 * u * u - 6.0 * u * t;
        double c2 = 6.0 * u * t - 3.0 * t * t;
        double c3 = 3.0 * t * t;
        return { c0 * p0.x + c1 * p1.x + c2 * p2.x + c3 * p3.x,
                 c0 * p0.y + c1 * p1.y + c2 * p2.y + c3 * p3.y };
    }

    double computeArcLengthGauss5() const {
        // Tọa độ nghiệm Legendre chuẩn hóa trên [0, 1] và trọng số Gauss tương ứng
        const std::array<double, 5> t_vals = {
            0.046910077030668, 0.230765344947158, 0.500000000000000,
            0.769234655052842, 0.953089922969332
        };
        const std::array<double, 5> weights = {
            0.118463442528095, 0.239314335249683, 0.284444444444444,
            0.239314335249683, 0.118463442528095
        };

        double length = 0.0;
        for (size_t i = 0; i < 5; ++i) {
            length += weights[i] * derivative(t_vals[i]).length();
        }
        return length;
    }
};
```

7.6 Thiết kế Cấu trúc Dữ liệu Bản vẽ Đồ họa Hỗ trợ Toàn diện Thao tác CRUD và Ánh xạ Hai chiều

Để xây dựng một ứng dụng vẽ kỹ thuật hoàn chỉnh (tương tự AutoCAD mini hoặc phần mềm xử lý vector), cấu trúc dữ liệu bản vẽ đóng vai trò là kiến trúc trung tâm quản lý mô hình dữ liệu. Bản vẽ phải hỗ trợ trọn vẹn 5 tiêu chí tác vụ CRUD chuẩn mực: Tạo mới (Create), Đọc tệp (Open), Lưu trữ (Save), Xóa bỏ (Delete), Hiệu chỉnh thuộc tính (Modify), đồng thời tích hợp sẵn mô đun biến đổi hai chiều giữa thế giới thực và màn hình hiển thị.

Kiến trúc hướng đối tượng tối ưu được tổ chức thành các lớp sau:

- *Lớp Thực thể Cơ sở* `GraphicEntity`*:* Khai báo giao diện trừu tượng với các phương thức thuần ảo: kết xuất `draw()`, tuần tự hóa dữ liệu `serialize()`, giải tuần tự hóa `deserialize()`, kiểm tra va chạm chuột `hitTest()`, và trả về hình chữ nhật bao `getBoundingBox()`.
- *Các Lớp Kế thừa Thực thể Cụ thể:* `LineEntity`, `CircleEntity`, `BezierEntity`, `PolygonEntity`. Mỗi thực thể lưu trữ tọa độ đỉnh gốc trong hệ tọa độ thế giới (WCS) và các thuộc tính đồ họa như màu sắc viền, độ dày nét và màu tô.
- *Lớp Bản vẽ* `DrawingDocument`*:* Quản lý danh sách các con trỏ thực thể đa hình, cung cấp chỉ mục định danh định danh (UUID), hỗ trợ cơ chế quản lý lịch sử phục vụ tác vụ Hoàn tác/Làm lại (Undo/Redo), và xuất/nhập dữ liệu định dạng tệp chuẩn (JSON hoặc tệp nhị phân).
- *Lớp Ánh xạ Không gian* `ViewportTransformer`*:* Chịu trách nhiệm đồng bộ hóa hai chiều giữa Cửa sổ Thế giới (W) và Cổng nhìn Màn hình (V), tự động duy trì tỉ lệ khung hình để chống biến dạng hình học.

Khung mã C++ minh họa kiến trúc lõi được thiết kế như sau:

```
#include <iostream>
#include <vector>
#include <memory>
#include <string>
#include <fstream>

struct Point2D { double x, y; };
struct ScreenPoint { int x, y; };

class ViewportTransformer {
public:
    double xw_min, xw_max, yw_min, yw_max;
    int xv_min, xv_max, yv_min, yv_max;

    ScreenPoint worldToScreen(const Point2D& pw) const {
        double sx = (xv_max - xv_min) / (xw_max - xw_min);
        double sy = (yv_max - yv_min) / (yw_max - yw_min);
        return { static_cast<int>(xv_min + (pw.x - xw_min) * sx),
                 static_cast<int>(yv_max - (pw.y -
yw_min) * sy) }; // Lật trục Y màn hình
    }

    Point2D screenToWorld(const ScreenPoint& ps) const {
        double sx = (xw_max - xw_min) / (xv_max - xv_min);
        double sy = (yw_max - yw_min) / (yv_max - yv_min);
        return { xw_min + (ps.x - xv_min) * sx,
                 yw_min + (yv_max - ps.y) * sy };
    }
};

class GraphicEntity {
```

```
public:
    int id;
    std::string color;
    virtual ~GraphicEntity() = default;
    virtual void draw(const ViewportTransformer& trans) const = 0;
    virtual bool hitTest(const Point2D& pt, double tolerance) const = 0;
    virtual void serialize(std::ostream& out) const = 0;
};

class LineEntity : public GraphicEntity {
public:
    Point2D start, end;
    LineEntity(int id, Point2D p1, Point2D p2) : start(p1), end(p2) { this-
>id = id; }
    void draw(const ViewportTransformer& trans) const override {
        ScreenPoint s1 = trans.worldToScreen(start);
        ScreenPoint s2 = trans.worldToScreen(end);
        // Gọi hàm raster hóa đường thẳng Bresenham từ s1 đến s2
    }
    bool hitTest(const Point2D& pt, double tol) const override {
        // Tính khoảng cách từ pt đến đoạn thẳng start-end
        return false;
    }
    void serialize(std::ostream& out) const override {
        out << "LINE " << id << " " << start.x << " " << start.y << " "
            << end.x << " " << end.y << "\n";
    }
};

class DrawingDocument {
private:
    std::vector<std::unique_ptr<GraphicEntity>> entities;
    int next_id = 1;

public:
    // Create
    int addLine(Point2D p1, Point2D p2) {
        int id = next_id++;
        entities.push_back(std::make_unique<LineEntity>(id, p1, p2));
        return id;
    }
    // Delete
    bool removeEntity(int id) {
        auto it = std::remove_if(entities.begin(), entities.end(),
            [id](const auto& e) { return e->id == id; });
        if (it != entities.end()) {
            entities.erase(it, entities.end());
            return true;
        }
        return false;
    }
    // Save
    void save(const std::string& filename) const {
        std::ofstream file(filename);
        for (const auto& e : entities) e->serialize(file);
    }
    // Open
    void open(const std::string& filename) {
        entities.clear();
```

```
        std::ifstream file(filename);
        std::string type;
        while (file >> type) {
            if (type == "LINE") {
                int id; Point2D p1, p2;
                file >> id >> p1.x >> p1.y >> p2.x >> p2.y;
                entities.push_back(std::make_unique<LineEntity>(id, p1, p2));
                if (id >= next_id) next_id = id + 1;
            }
        }
    }
};
```

7.7 Thuật toán và Cài đặt Vẽ Mô hình Khung dây (Wireframe) Quả cầu với Lưới Tứ giác

Mô hình khung dây (Wireframe Model) biểu diễn các đối tượng hình học 3D dưới dạng tập hợp các đỉnh nối với nhau bởi các cạnh thẳng rời rạc. Đối với mặt cầu tâm tại gốc tọa độ $O(0,0,0)$ với bán kính R, ta sử dụng hệ tọa độ cầu tham số hóa theo vĩ độ $\phi \in [-\pi/2, \pi/2]$ và kinh độ $\theta \in [0, 2\pi]$:

$$\begin{cases} x(\theta,\phi) = R\cos\phi\cos\theta \\ y(\theta,\phi) = R\cos\phi\sin\theta \\ z(\theta,\phi) = R\sin\phi \end{cases} \tag{179}$$

Để kiến tạo lưới khung dây dạng lưới vuông (Quadrilateral Grid Mesh), ta phân hoạch miền vĩ độ thành M dải song song (Vĩ tuyến) và miền kinh độ thành N cung hướng tâm (Kinh tuyến). Tại mỗi điểm nút (i,j) với $i \in [0,M]$ và $j \in [0,N]$:

$$\phi_i = -\frac{\pi}{2} + i \cdot \frac{\pi}{M}, \quad \theta_j = j \cdot \frac{2\pi}{N} \tag{180}$$

Mỗi ô lưới tứ giác được tạo thành bởi 4 đỉnh: $P(i,j)$, $P(i+1,j)$, $P(i+1,j+1)$, $P(i,j+1)$. Khung dây được vẽ bằng cách kết nối các cạnh ngang theo vĩ tuyến $P(i,j) \to P(i,j+1)$ và các cạnh dọc theo kinh tuyến $P(i,j) \to P(i+1,j)$. Để chiếu các điểm 3D lên màn hình 2D, ta áp dụng phép chiếu phối cảnh chuẩn với khoảng cách mắt quan sát d:

$$x_{\text{proj}} = d \cdot \frac{x}{z + z_{\text{cam}}}, \quad y_{\text{proj}} = d \cdot \frac{y}{z + z_{\text{cam}}} \tag{181}$$

Chương trình C++ dưới đây thực hiện toàn bộ quy trình tính toán và xuất tập cạnh khung dây của quả cầu:

```
#include <iostream>
#include <vector>
#include <cmath>

struct Point3D { double x, y, z; };
struct Edge3D { int p1_idx, p2_idx; };

class WireframeSphere {
public:
    double radius;
    int rings;      // Số vĩ tuyến M
    int sectors;    // Số kinh tuyến N
    std::vector<Point3D> vertices;
    std::vector<Edge3D> edges;

    WireframeSphere(double r, int m, int n) : radius(r), rings(m), sectors(n) {
        generateMesh();
    }
```

```
    void generateMesh() {
        const double PI = std::acos(-1.0);
        // 1. Sinh tập đỉnh
        for (int i = 0; i <= rings; ++i) {
            double phi = -PI / 2.0 + i * (PI / rings);
            for (int j = 0; j <= sectors; ++j) {
                double theta = j * (2.0 * PI / sectors);
                Point3D pt;
                pt.x = radius * std::cos(phi) * std::cos(theta);
                pt.y = radius * std::cos(phi) * std::sin(theta);
                pt.z = radius * std::sin(phi);
                vertices.push_back(pt);
            }
        }
        // 2. Nối cạnh lưới tứ giác
        for (int i = 0; i < rings; ++i) {
            for (int j = 0; j < sectors; ++j) {
                int current = i * (sectors + 1) + j;
                int next_ring = (i + 1) * (sectors + 1) + j;
                // Cạnh kinh tuyến dọc
                edges.push_back({ current, next_ring });
                // Cạnh vĩ tuyến ngang
                edges.push_back({ current, current + 1 });
            }
        }
    }
};
```

7.8 Xác định và Tính toán Trường Vector Pháp tuyến của Quả cầu

Trường vector pháp tuyến bề mặt (Normal Vector Field) là đại lượng hình học vi phân trọng yếu xác định hướng trực giao tại mọi điểm trên mặt cong, đóng vai trò quyết định trong việc tính toán mô hình phản xạ ánh sáng (Shading) và phát hiện va chạm.

Xét mặt cầu tâm tại $C(x_c, y_c, z_c)$ bán kính R dưới hai góc nhìn giải tích:

1. Tiếp cận thông qua Phương trình Ẩn (Implicit Surface): Phương trình ẩn của mặt cầu là $F(x, y, z) = (x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 - R^2 = 0$. Theo định lý gradient trong giải tích đa biến, gradient của hàm vô hướng ∇F luôn trực giao với mặt mức $F = 0$:

$$\nabla F(x, y, z) = \begin{bmatrix} \frac{\partial F}{\partial x} \\ \frac{\partial F}{\partial y} \\ \frac{\partial F}{\partial z} \end{bmatrix} = \begin{bmatrix} 2(x - x_c) \\ 2(y - y_c) \\ 2(z - z_c) \end{bmatrix} \tag{182}$$

Độ lớn của vector gradient tại mọi điểm trên mặt cầu là:

$$\|\nabla F\| = \sqrt{4(x - x_c)^2 + 4(y - y_c)^2 + 4(z - z_c)^2} = 2\sqrt{R^2} = 2R \tag{183}$$

Do đó, trường vector pháp tuyến đơn vị hướng ra ngoài tại điểm $P(x, y, z)$ là:

$$\mathbf{N}(P) = \frac{\nabla F}{\|\nabla F\|} = \begin{bmatrix} \frac{x - x_c}{R} \\ \frac{y - y_c}{R} \\ \frac{z - z_c}{R} \end{bmatrix} = \frac{P - C}{R} \tag{184}$$

2. Tiếp cận thông qua Phương trình Tham số (Parametric Surface): Sử dụng dạng tham số mặt cầu tại phương trình equation 179, hai vector tiếp tuyến cục bộ tại điểm (θ, ϕ) được xác định thông

qua các đạo hàm riêng:

$$\mathbf{T}_\theta = \frac{\partial \mathbf{S}}{\partial \theta} = \begin{bmatrix} -R\cos\phi\sin\theta \\ R\cos\phi\cos\theta \\ 0 \end{bmatrix} \tag{185}$$

$$\mathbf{T}_\phi = \frac{\partial \mathbf{S}}{\partial \phi} = \begin{bmatrix} -R\sin\phi\cos\theta \\ -R\sin\phi\sin\theta \\ R\cos\phi \end{bmatrix} \tag{186}$$

Vector pháp tuyến chưa chuẩn hóa là tích hữu hướng giữa hai vector tiếp tuyến:

$$\begin{aligned} \mathbf{n}(\theta,\phi) = \mathbf{T}_\theta \times \mathbf{T}_\phi &= \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ -R\cos\phi\sin\theta & R\cos\phi\cos\theta & 0 \\ -R\sin\phi\cos\theta & -R\sin\phi\sin\theta & R\cos\phi \end{vmatrix} \\ &= \mathbf{i}\left(R^2\cos^2\phi\cos\theta\right) - \mathbf{j}\left(-R^2\cos^2\phi\sin\theta\right) + \mathbf{k}\left(R^2\cos\phi\sin\phi\sin^2\theta + R^2\cos\phi\sin\phi\cos^2\theta\right) \\ &= R^2\cos\phi\begin{bmatrix} \cos\phi\cos\theta \\ \cos\phi\sin\theta \\ \sin\phi \end{bmatrix} \end{aligned} \tag{187}$$

Độ lớn của vector này là $\|\mathbf{n}(\theta,\phi)\| = R^2|\cos\phi|$. Khi chuẩn hóa chia cho độ lớn (với $\phi \in [-\pi/2, \pi/2]$ nên $\cos\phi \geq 0$), ta thu được trường pháp tuyến đơn vị:

$$\mathbf{N}(\theta,\phi) = \begin{bmatrix} \cos\phi\cos\theta \\ \cos\phi\sin\theta \\ \sin\phi \end{bmatrix} \tag{188}$$

Kết quả này hoàn toàn trùng khớp với công thức equation 184. Điều này khẳng định một tính chất hình học đẹp đẽ: Tại bất kỳ điểm nào trên mặt cầu, vector pháp tuyến luôn có phương xuyên tâm tuyệt đối, hướng thẳng từ tâm mặt cầu đi qua điểm đang xét ra vô cực.

7.9 Thuật toán và Cài đặt Tạo Mặt Trụ Trải (TabSurf) với Đường sinh Bézier Bậc Ba

Trong thiết kế hình học có máy tính hỗ trợ (CAD), Mặt Trụ Trải (Tabulated Cylinder, thường gọi là *TabSurf*) là một mặt kẻ suy biến đặc biệt, được kiến tạo bằng cách tịnh tiến một đường cong phẳng hoặc đường cong không gian tùy ý $G(u)$ dọc theo một vector chỉ hướng $\mathbf{v}$ trong không gian.

Xét đường sinh $G(u)$ là một đường cong Bézier bậc ba định nghĩa bởi 4 điểm điều khiển P_0, P_1, P_2, P_3:

$$G(u) = (1-u)^3 P_0 + 3u(1-u)^2 P_1 + 3u^2(1-u)P_2 + u^3 P_3, \quad u \in [0,1] \tag{189}$$

Cho vector dịch chuyển định hướng $\mathbf{v} = (v_x, v_y, v_z)$. Phương trình tham số hai biến của mặt cong $S(u,v)$ có dạng:

$$S(u,v) = G(u) + v \cdot \mathbf{v}, \quad u \in [0,1],\ v \in [0,1] \tag{190}$$

Mặt $S(u,v)$ có các đặc tính hình học quan trọng:

- Với mỗi giá trị v cố định, thiết diện $u \mapsto S(u,v)$ là một đường cong Bézier bậc ba tịnh tiến hoàn toàn đồng dạng với đường sinh gốc $G(u)$.
- Với mỗi giá trị u cố định, đường đẳng tham số $v \mapsto S(u,v)$ là một đoạn thẳng thẳng tắp (được gọi là đường sinh hay Ruling) song song với vector $\mathbf{v}$.

Chương trình C++ sau đây tạo lưới khung dây cho mặt TabSurf với độ phân giải $N \times M$ ô lưới:

```
#include <iostream>
#include <vector>

struct Point3D {
    double x, y, z;
    Point3D operator+(const Point3D& o) const { return {x + o.x, y + o.y, z + o.z}; }
```

```
    Point3D operator*(double s) const { return {x * s, y * s, z * s}; }
};

class TabulatedCylinder {
public:
    Point3D p0, p1, p2, p3; // 4 điểm điều khiển của đường Bézier bậc 3
    Point3D dir_vector;     // Vector tịnh tiến v

    Point3D evaluateBezier(double u) const {
        double c0 = (1 - u) * (1 - u) * (1 - u);
        double c1 = 3 * u * (1 - u) * (1 - u);
        double c2 = 3 * u * u * (1 - u);
        double c3 = u * u * u;
        return p0 * c0 + p1 * c1 + p2 * c2 + p3 * c3;
    }

    Point3D evaluateSurface(double u, double v) const {
        return evaluateBezier(u) + (dir_vector * v);
    }

    void generateWireframe(int num_u, int num_v, std::vector<Point3D>& grid) const {
        grid.resize((num_u + 1) * (num_v + 1));
        for (int i = 0; i <= num_u; ++i) {
            double u = static_cast<double>(i) / num_u;
            for (int j = 0; j <= num_v; ++j) {
                double v = static_cast<double>(j) / num_v;
                grid[i * (num_v + 1) + j] = evaluateSurface(u, v);
            }
        }
    }
};
```

7.10 Thuật toán và Cài đặt Vẽ Mặt Kẻ (RuleSurf) Nối Hai Biên Cong Bézier Bậc Ba Khác nhau

Mặt Kẻ (Ruled Surface, ký hiệu là *RuleSurf*) là một lớp mặt cong tổng quát được hình thành bằng cách quét một đoạn thẳng nối giữa hai điểm tương ứng trên hai đường biên dạng độc lập $C_1(u)$ và $C_2(u)$ khi tham số u biến thiên từ 0 đến 1 (Foley et al., 1996).

Xét trường hợp hai biên dạng là hai đường cong Bézier bậc ba hoàn toàn khác nhau trong không gian ba chiều:

$$C_1(u) = \sum_{k=0}^{3} B_{k,3}(u) P_k, \quad u \in [0,1] \tag{191}$$

$$C_2(u) = \sum_{k=0}^{3} B_{k,3}(u) Q_k, \quad u \in [0,1] \tag{192}$$

với $\{P_k\}_{k=0}^{3}$ và $\{Q_k\}_{k=0}^{3}$ là hai bộ điểm điều khiển độc lập. Phương trình tham số nội suy tuyến tính của mặt kẻ $S(u,v)$ được xác định bởi:

$$S(u,v) = (1-v)C_1(u) + vC_2(u), \quad u \in [0,1],\ v \in [0,1] \tag{193}$$

Tại mọi giá trị tham số $u = u_0 \in [0,1]$ cố định, quỹ đạo của điểm khi v biến thiên từ 0 đến 1 là một đoạn thẳng nối trực tiếp từ $C_1(u_0)$ tới $C_2(u_0)$. Ngược lại, tại mọi giá trị $v = v_0$ cố định, thiết diện cắt ngang là một đường cong Bézier bậc ba có các điểm điều khiển là tổ hợp lồi: $R_k(v_0) = (1-v_0)P_k + v_0 Q_k$ với $k = 0, \ldots, 3$.

Dưới đây là mã nguồn C++ xây dựng lưới khung dây cho mặt RuleSurf:

```
#include <iostream>
#include <vector>

struct Point3D {
    double x, y, z;
    Point3D operator+(const Point3D& o) const { return {x + o.x, y + o.y, z + o.z}; }
    Point3D operator*(double s) const { return {x * s, y * s, z * s}; }
};

class RuledSurface {
public:
    std::vector<Point3D> ctrl_P; // 4 điểm điều khiển biên 1
    std::vector<Point3D> ctrl_Q; // 4 điểm điều khiển biên 2

    Point3D evalBezier(const std::vector<Point3D>& cp, double u) const {
        double c0 = (1 - u) * (1 - u) * (1 - u);
        double c1 = 3 * u * (1 - u) * (1 - u);
        double c2 = 3 * u * u * (1 - u);
        double c3 = u * u * u;
        return cp[0] * c0 + cp[1] * c1 + cp[2] * c2 + cp[3] * c3;
    }

    Point3D evaluateSurface(double u, double v) const {
        Point3D pt_c1 = evalBezier(ctrl_P, u);
        Point3D pt_c2 = evalBezier(ctrl_Q, u);
        return pt_c1 * (1.0 - v) + pt_c2 * v;
    }

    void generateWireframe(int steps_u, int steps_v,
                           std::vector<std::pair<Point3D, Point3D>>& wireframe_lines)
        // Sinh các đường sinh thẳng (Rulings) nối dọc theo tham số v
        for (int i = 0; i <= steps_u; ++i) {
            double u = static_cast<double>(i) / steps_u;
            Point3D start = evaluateSurface(u, 0.0);
            Point3D end = evaluateSurface(u, 1.0);
            wireframe_lines.push_back({start, end});
        }
        // Sinh các đường cong ngang dọc theo tham số u
        for (int j = 0; j <= steps_v; ++j) {
            double v = static_cast<double>(j) / steps_v;
            for (int i = 0; i < steps_u; ++i) {
                double u1 = static_cast<double>(i) / steps_u;
                double u2 = static_cast<double>(i + 1) / steps_u;
                wireframe_lines.push_back({ evaluateSurface(u1, v), evaluateSurface(u
            }
        }
    }
};
```

7.11 Bản chất Hình học của Mặt Vá Coons (EdgeSurf) và Chứng minh Lượng dư Triệt tiêu Bằng P_3

Mặt Vá Coons (thường được gọi là lệnh *EdgeSurf* trong các hệ thống CAD) do Steven Anson Coons đề xuất, là một trong những phát minh quan trọng nhất của mô hình hóa hình học. Bài toán đặt ra là: Cho trước bốn đường cong biên không gian tùy ý nối tiếp nhau khép kín tạo thành một khung biên 4 cạnh, hãy xây dựng một hàm mặt cong trơn tru $S(u, v)$ với $(u, v) \in [0, 1] \times [0, 1]$ sao cho bề mặt nội suy chính xác tuyệt đối cả 4 đường cong biên đó.

1. Thiết lập Bốn Đường cong Biên và Bốn Đỉnh Góc: Giả sử bốn đường cong biên được ký hiệu là:

$$\text{Biên đáy } (v=0): \quad C_1(u) = S(u,0) \tag{194}$$
$$\text{Biên đỉnh } (v=1): \quad C_2(u) = S(u,1) \tag{195}$$
$$\text{Biên trái } (u=0): \quad D_1(v) = S(0,v) \tag{196}$$
$$\text{Biên phải } (u=1): \quad D_2(v) = S(1,v) \tag{197}$$

Để bốn đường cong này tạo thành một đường bao liên tục khép kín, chúng phải thỏa mãn điều kiện tương thích tại 4 đỉnh góc:

$$P_{00} = C_1(0) = D_1(0) = S(0,0) \tag{198}$$
$$P_{10} = C_1(1) = D_2(0) = S(1,0) \tag{199}$$
$$P_{01} = C_2(0) = D_1(1) = S(0,1) \tag{200}$$
$$P_{11} = C_2(1) = D_2(1) = S(1,1) \tag{201}$$

2. Xây dựng Các Mặt Kẻ Thành phần: Để nội suy hai đường cong biên đối diện $C_1(u)$ và $C_2(u)$, ta kiến tạo mặt kẻ thứ nhất $P_1(u,v)$:

$$P_1(u,v) = (1-v)C_1(u) + vC_2(u) \tag{202}$$

Mặt kẻ $P_1(u,v)$ khớp chính xác với hai biên $C_1(u)$ và $C_2(u)$, nhưng tại hai biên bên $u=0$ và $u=1$, nó suy biến thành hai đoạn thẳng nối các góc thay vì đi theo đường cong $D_1(v)$ và $D_2(v)$.

Hoàn toàn tương tự, để nội suy hai đường biên đối diện $D_1(v)$ và $D_2(v)$, ta kiến tạo mặt kẻ thứ hai $P_2(u,v)$:

$$P_2(u,v) = (1-u)D_1(v) + uD_2(v) \tag{203}$$

3. Xuất hiện Sai số và Bản chất của Lượng Dư P_3: Nếu ta thực hiện phép cộng đơn giản hai mặt kẻ thành phần $S_{\text{test}}(u,v) = P_1(u,v) + P_2(u,v)$, hãy khảo sát giá trị của bề mặt này tại biên đáy $v=0$:

$$\begin{aligned} S_{\text{test}}(u,0) &= P_1(u,0) + P_2(u,0) \\ &= [(1-0)C_1(u) + 0 \cdot C_2(u)] + [(1-u)D_1(0) + uD_2(0)] \\ &= C_1(u) + [(1-u)P_{00} + uP_{10}] \end{aligned} \tag{204}$$

Nhận xét mấu chốt: Kết quả không bằng đường biên mong muốn $C_1(u)$ mà bị đội lên một lượng thừa chính bằng đoạn thẳng nội suy tuyến tính giữa hai đỉnh góc: $(1-u)P_{00} + uP_{10}$. Hiện tượng dư thừa này xuất hiện là do bốn đỉnh góc $P_{00}, P_{10}, P_{01}, P_{11}$ đã bị tính hai lần (một lần trong mặt kẻ P_1 và một lần nữa trong mặt kẻ P_2).

Để triệt tiêu lượng dư thừa này trên toàn bộ miền xác định, ta bắt buộc phải trừ đi một mặt nội suy song tuyến tính (Bilinear Interpolation Surface) tạo bởi 4 đỉnh góc, được ký hiệu là $P_3(u,v)$:

$$P_3(u,v) = \begin{bmatrix} 1-u & u \end{bmatrix} \begin{bmatrix} P_{00} & P_{01} \\ P_{10} & P_{11} \end{bmatrix} \begin{bmatrix} 1-v \\ v \end{bmatrix} = (1-u)(1-v)P_{00} + u(1-v)P_{10} + (1-u)vP_{01} + uvP_{11} \tag{205}$$

Phương trình hoàn chỉnh của Mặt Vá Coons (EdgeSurf) là:

$$S(u,v) = P_1(u,v) + P_2(u,v) - P_3(u,v) \tag{206}$$

4. Chứng minh Lượng Dư Bị Triệt Tiêu Tuyệt Đối tại Toàn Bộ Bốn Biên:

- *Kiểm tra tại biên đáy $v=0$:*

  $$P_1(u,0) = C_1(u) \tag{207}$$
  $$P_2(u,0) = (1-u)D_1(0) + uD_2(0) = (1-u)P_{00} + uP_{10} \tag{208}$$
  $$P_3(u,0) = (1-u)(1-0)P_{00} + u(1-0)P_{10} + 0 + 0 = (1-u)P_{00} + uP_{10} \tag{209}$$

  Thay vào phương trình equation 206:

  $$S(u,0) = C_1(u) + [(1-u)P_{00} + uP_{10}] - [(1-u)P_{00} + uP_{10}] = C_1(u) \tag{210}$$

  Lượng dư P_3 triệt tiêu hoàn toàn phần thừa của P_2, trả lại chính xác biên $C_1(u)$.

- *Kiểm tra tại biên đỉnh $v = 1$:*

$$P_1(u,1) = C_2(u) \tag{211}$$
$$P_2(u,1) = (1-u)D_1(1) + uD_2(1) = (1-u)P_{01} + uP_{11} \tag{212}$$
$$P_3(u,1) = (1-u)P_{01} + uP_{11} \tag{213}$$

  Suy ra $S(u,1) = C_2(u) + P_2(u,1) - P_3(u,1) = C_2(u)$.

- *Kiểm tra tại biên trái $u = 0$ và biên phải $u = 1$:* Hoàn toàn tương tự, tại $u = 0$ ta có $P_1(0,v) = P_3(0,v) = (1-v)P_{00} + vP_{01}$, do đó $S(0,v) = P_1(0,v) + D_1(v) - P_3(0,v) = D_1(v)$. Tại $u = 1$, ta có $S(1,v) = D_2(v)$.

Chứng minh giải tích trên khẳng định rằng: Việc trừ đi lượng dư $P_3(u,v)$ là điều kiện cần và đủ để khử lượng cộng gộp hai lần của các đỉnh góc, đảm bảo tính liên tục hình học và tính khớp biên tuyệt đối của mặt vá Coons trên toàn bộ không gian tham số.

TÀI LIỆU

Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew J. Ballard, Joshua Bambrick, et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630(8016):493–500, 2024. doi: 10.1038/s41586-024-07487-w.

Herbert Bay, Andreas Ess, Tinne Tuytelaars, and Luc Van Gool. Speeded-up robust features (SURF). *Computer Vision and Image Understanding*, 110(3):346–359, 2008. doi: 10.1016/j.cviu.2007.09.014.

Pierre Bézier. *Numerical Control: Mathematics and Applications*. John Wiley & Sons, 1972.

Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

Jack E. Bresenham. Algorithm for computer control of a digital plotter. *IBM Systems Journal*, 4(1):25–30, 1965. doi: 10.1147/sj.41.0025.

Jack E. Bresenham. A linear algorithm for incremental digital display of circular arcs. *Communications of the ACM*, 20(2):100–106, 1977. doi: 10.1145/359423.359432.

Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision (ECCV)*, pp. 213–229. Springer, 2020.

Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995. doi: 10.1007/BF00994018.

Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven C. H. Hoi. Instructblip: Towards general-purpose vision-language models with instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 49250–49267, 2023.

Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 1, pp. 886–893, 2005. doi: 10.1109/CVPR.2005.177.

Paul de Casteljau. Courbes et surfaces à pôles. Technical report, André Citroën Automobiles SA, Paris, 1959.

Henghui Ding, Chang Liu, Shuting He, Kaining Ying, Xudong Jiang, Chen Change Loy, and Yu-Gang Jiang. Mevis: A multi-modal dataset for referring motion expression video segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(12):11400–11416, 2025. doi: 10.1109/TPAMI.2025.3600507.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

James D. Foley, Andries van Dam, Steven K. Feiner, and John F. Hughes. *Computer Graphics: Principles and Practice*. Addison-Wesley, 2nd edition, 1996.

Kevin Frans, L. B. Soros, and Olaf Witkowski. CLIPDraw: Exploring text-to-drawing synthesis through language-image encoders. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pp. 5207–5218, 2022.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pp. 2672–2680, 2014.

Tianrui Guan, Fuxiao Liu, Xiyang Wu, Ruiqi Xian, Zongxia Li, Xiaoyu Liu, Xijun Wang, Lichang Chen, Furong Huang, Yaser Yacoob, Dinesh Manocha, and Tianyi Zhou. Hallusionbench: An advanced diagnostic suite for entangled language hallucination and visual illusion in large vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14375–14385, 2024.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016. doi: 10.1109/CVPR.2016.90.

Donald Hearn and M. Pauline Baker. *Computer Graphics with OpenGL*. Prentice Hall, 3rd edition, 2004.

Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations (ICLR)*, 2019.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 6840–6851, 2020.

Ajay Jain, Amber Xie, and Pieter Abbeel. VectorFusion: Text-to-SVG by abstracting pixel diffusion priors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2441–2450, 2023.

Hervé Jégou, Matthieu Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.

Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations (ICLR)*, 2014.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, pp. 1097–1105, 2012.

Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2):83–97, 1955. doi: 10.1002/nav.3800020109.

Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International Conference on Machine Learning*, pp. 12888–12900. PMLR, 2022.

Lin Li, Haoyan Guan, Jianing Qiu, and Michael Spratling. One prompt word is enough to boost adversarial robustness for pre-trained vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 24676–24685, 2024.

Tzu-Mao Li, Michal Lukáč, Michaël Gharbi, and Frédo Durand. Differentiable vector graphics rasterization for editing and learning. *ACM Transactions on Graphics (TOG)*, 39(6):193:1–193:15, 2020. doi: 10.1145/3414685.3417871.

Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. Evaluating object hallucination in large vision-language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 292–305, 2023.

Yuhong Li, Xiaofan Zhang, and Deming Chen. CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1091–1100, 2018.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 2980–2988, 2017.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 34892–34916, 2023.

Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. SSD: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*, pp. 21–37. Springer, 2016.

Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin Transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 10012–10022, 2021.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004. doi: 10.1023/B:VISI.0000029664.99615.94.

OpenAI. GPT-4V(ision) system card. *OpenAI Technical Report*, 2023. URL `https://openai.com/research/gpt-4v-system-card`.

William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4195–4205, 2023.

Jean Piaget. *The Psychology of Intelligence*. Routledge & Kegan Paul, London, 1950. URL `http://www.jstor.org/stable/2088298`. Translated by Malcolm Piercy and D. E. Berlyne; Original French edition 1947.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, pp. 8748–8763. PMLR, 2021.

Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.

Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, pp. 91–99, 2015.

Hamid Rezatofighi, Nathan Tsoi, Jianwei Geng, Rui Deng, Stefan Leutenegger, and Silvio Savarese. Generalized intersection over union: A metric and a loss for bounding box regression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 658–666, 2019.

Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951. doi: 10.1214/aoms/1177729586.

Juan A. Rodriguez and et al. StarVector: Generating scalable vector graphics with multimodal large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015.

Boyu Wang, Huidong Liu, Dimitris Samaras, and Minh Hoai. Distribution matching for crowd counting. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 1595–1607, 2020.

Ximing Xing and et al. LLM4SVG: Unleashing the power of large language models in scalable vector graphics generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

Ximing Xing, Chuang Wang, Haitao Zhou, Zhihao Hu, Chongxuan Li, Dong Xu, and Qian Yu. SVGDreamer: Text guided SVG generation with diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4546–4555, 2024.

Chuanqi Zang, Hanqing Wang, Mingtao Pei, and Wei Liang. Discovering the real association: Multimodal causal reasoning in video question answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 19027–19036, 2023.

Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. ByteTrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision (ECCV)*, pp. 1–21. Springer, 2022.

Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, and Dongwei Ren. Distance-IoU loss: Faster and better learning for bounding box regression. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 34, pp. 12993–12900, 2020.